

软件工程的来龙去脉

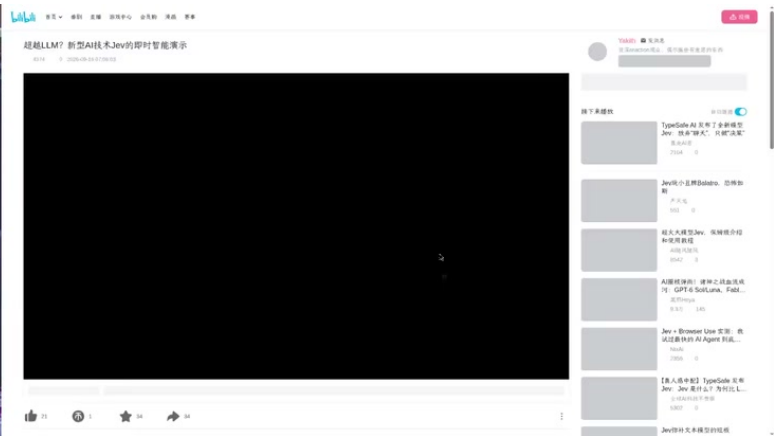
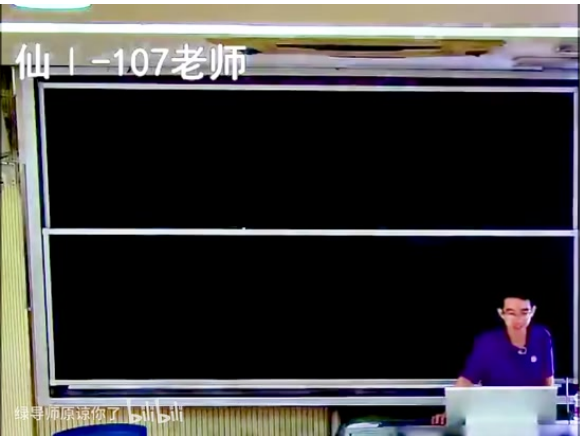
生成式软件工程 · 南京大学
夸克原文图文转录

本稿保留夸克导出的原文、语序和段落，不做摘要或术语改写，尚未逐句听校。时间标记来自 TXT 的段落起点，不代表逐句字幕或精确结束时间。截图按时间归入最近的前置段落，图注保留实际画面时间。

全片 01:39:59 | 147 个段落 | 162 张截图
来源：绿导师原谅你了 · BV1Nyeq6qEt8

00:00:00

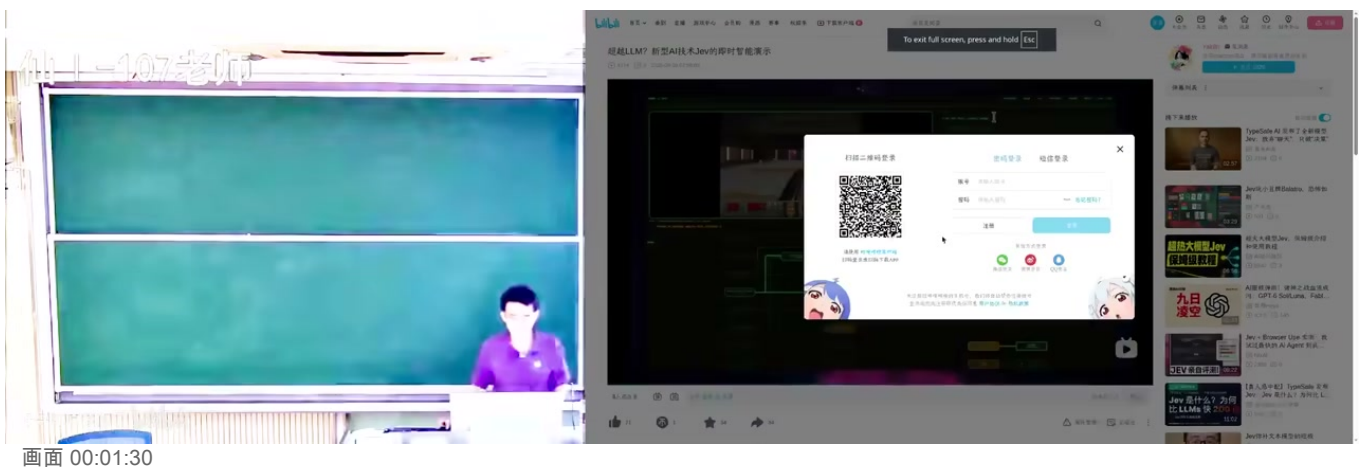
好，我们继续讲生成式软件工程。上一节课我们讲了一点版本管理，然后今天我会正式开始软件工程部分的内容，所以现在算是真正的软件工程课了。虽然当然版本管理也是软件工程。当然在上课之前，我可以先分享一些最近一周发生的新闻，当然因为这个热度已经高到一定程度了，所以我相信大家多多少少都已经看过了，对吧？比如说这个type CVI最新发布了一个可以实时玩doom的这样的一个模型，算是基础模型。我觉得它我猜盲猜它是一个prefer only的一个就像只能出一个token的transformer，它可能是别的架构，然后它可以输入一个，比如说输入一个json，在这里应该他把他这个毁灭战士的状态变成一个真相文件，然后让他给一个决策。然后他大概可以在毫秒级或者百毫秒级，就基本上是一个准人类实时的这样的，你看底下有一个决策然后来做一个输出一个头，然后这个token应该他微调过了，你可以理解成微调或者训练过了，反正他不说人话，对吧？他不说人话，但是他可以做决策，就像训练一个神经网络，但是一个通用的介于大学模型和以前的那种，比如说决策数，你们可能学过一个非常小的模型之间，但是他就可以真的可以干，你可以看它真的是可以玩游戏的。





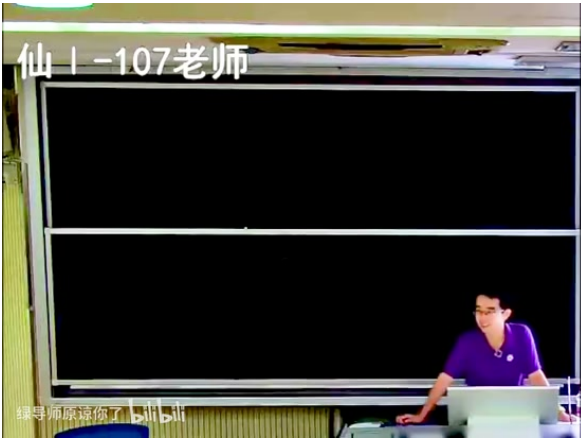
00:01:30

然后反正这个新闻火了，然后他他给自己一个名字叫system one，叫真正人类意义上的实施决策。其实这不是一个很新的技术，而且你们过去如果关注过这个赛道的话，会看到确实也有一些人试图做这种非常快的模型。但是这次火了以后，他就正式把这个赛道打开了。



00:01:54

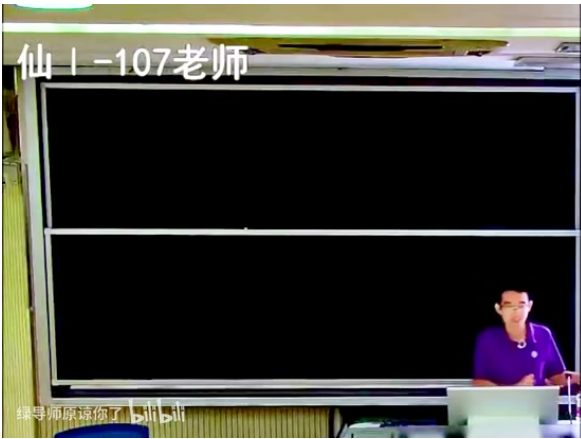
把这个赛道打开意味着什么呢？你们作为AI native时代的年轻人，如果有一个模型可以在100毫秒，当然你乐观一点，如果模型可以裁的更小一点，可以在50毫秒或者更短的时间里面做实时的决策。那你可以用它来干什么呢？你可以用它来感知这个周围的环境。比如说为什么人和人沟通，和人和机器沟通，你会感觉到有很大的差别，对吧？前两天好像我看到一个大家在传说和AI演员对话，然后那个A演员突然就开始说广东话了，然后另外两个就在他们英语对话的时候突然就说广东话。而且这个AI的生成的数字，人和人之间是有很大的延迟的。



画面 00:02:00

00:02:40

但如果你有一个100毫秒、200毫秒的小的决策模型，你可以做什么呢？你可以让他做决策，然后实时生成脸上的微表情，根据你说的话，然后这样的话你就可以得到了一个非常有实时性人感的东西，这是一个非常好的，以前的大圆模型在那个延迟下他可能有这个能力做，但是做的就不是那么好。但是你今天就可以得，你看他能打打游戏，他就可以生成这个有人感的人，然后他可以甚至可以哄这个小朋友睡觉，对吧？婴儿睡不着的难题就彻底解决了。所以他开了一个非常大的赛道。至少这次火了以后，我们看到这件事，当然也许已经之前已经也有人也有人看到。



画面 00:03:00

Review & Comments

摧毁性的事情一直都在发生

- Typesafe AI 的 [Jev](#)
 - 我猜是一个 prefill only 的 transformer 😊
 - 官方披露：一次并行输出全部概率，无需逐 token 自回归生成。见 [技术介绍](#)。
 - System I (人类意义上的实时决策)

感受一下 AI 进化的速度

- 很难想象 [Silicon Valley](#) 里的 Pied Piper 到今天会怎么样
- 以及我们每个人在就业市场上面临的挑战

Review & Comments

摧毁性的事情一直都在发生

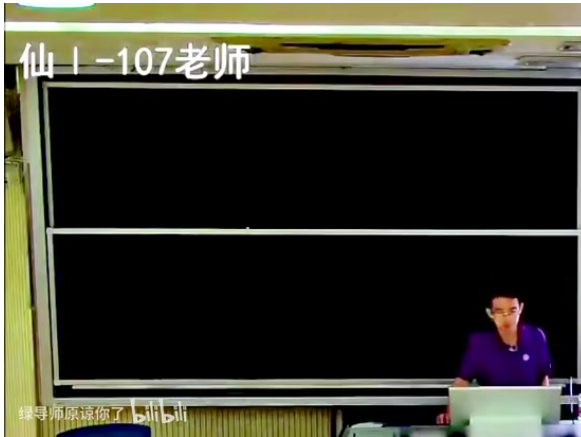
- Typesafe AI 的 [Jev](#)
 - 我猜是一个 prefill only 的 transformer 😊
 - 官方披露：一次并行输出全部概率，无需逐 token 自回归生成。见 [技术介绍](#)。
 - System I (人类意义上的实时决策)

感受一下 AI 进化的速度

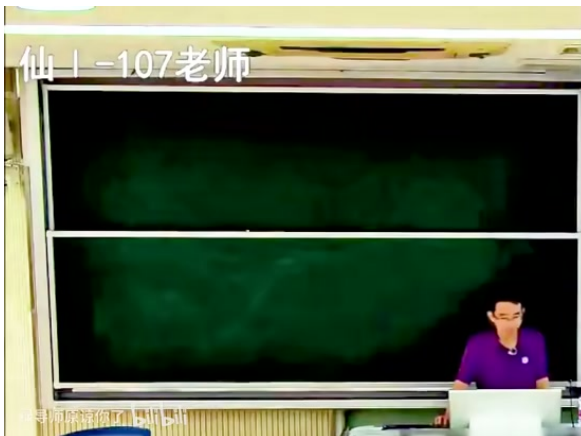
- 很难想象 [Silicon Valley](#) 里的 Pied Piper 到今天会怎么样
- 以及我们每个人在就业市场上面临的挑战

00:03:24

对，所以其实我要说的是AI I的进化速度其实非常的快。这个视频完全是由gp t6生成的，大概是从你看到从1956年到58年到1960年，感知与规划机器走出实验室。然后到70年到90年的人工智能的寒冬。然后这个寒冬到97年的时候，我们是用搜索算法第一次击败了人类的世界冠军。97年你看这个经历了很长时间的的发展。然后到后面你突然发现从2012年开始，这个Alex night开始，这个世界就发生了很大的变化，发展的速度真的是始料未及。到2016年那个时候我在现场看直播，2017年的时候我们也关注，那时候我在读PD我也在关注像磁向量、这个transformer、bir啊这些可以用来干什么有趣的事情。



画面 00:03:25



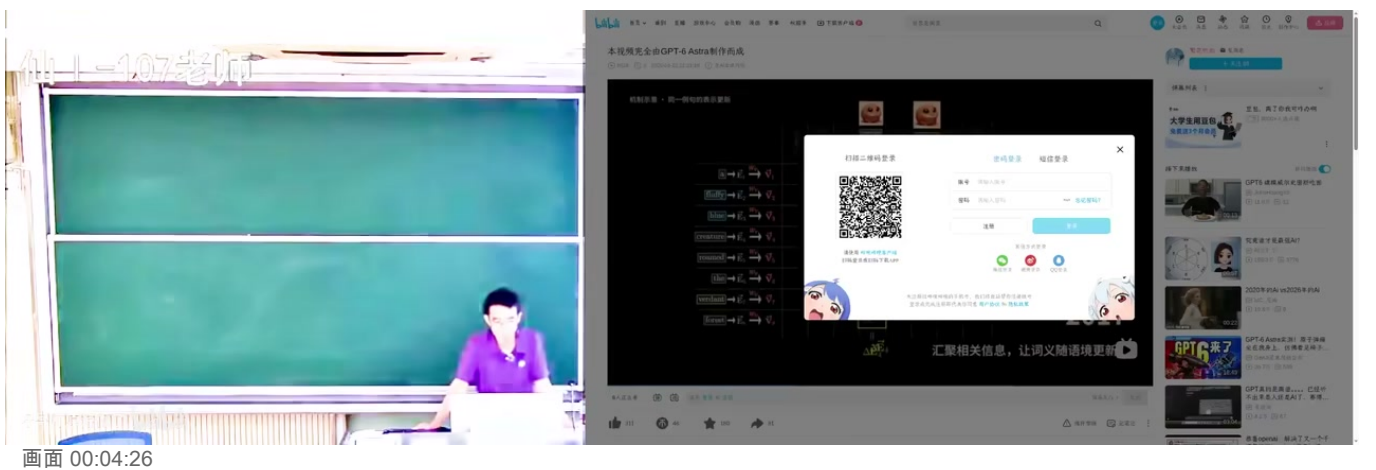
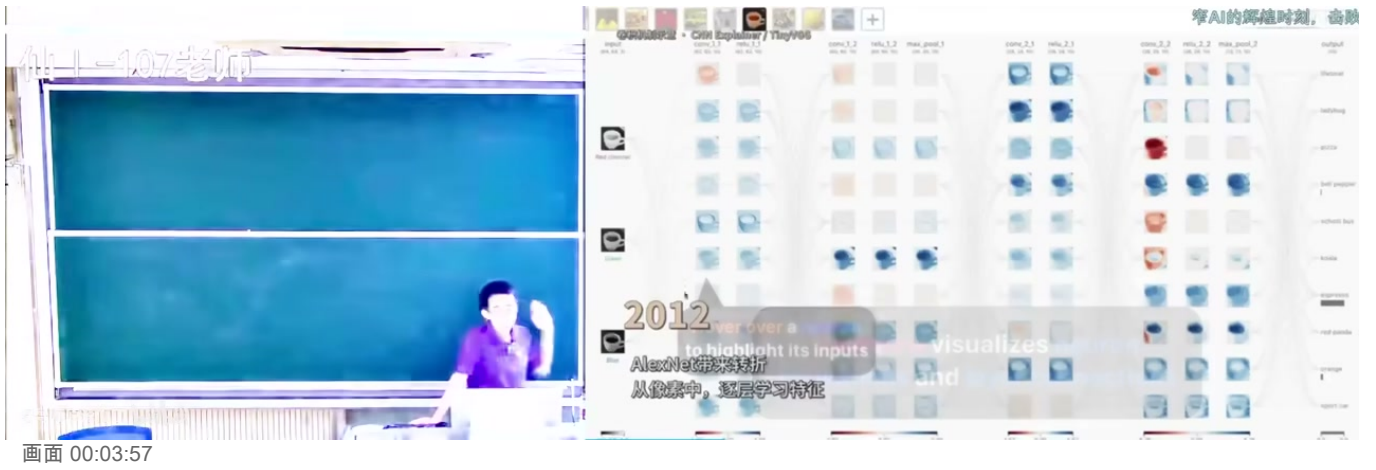
画面 00:03:28



画面 00:03:32

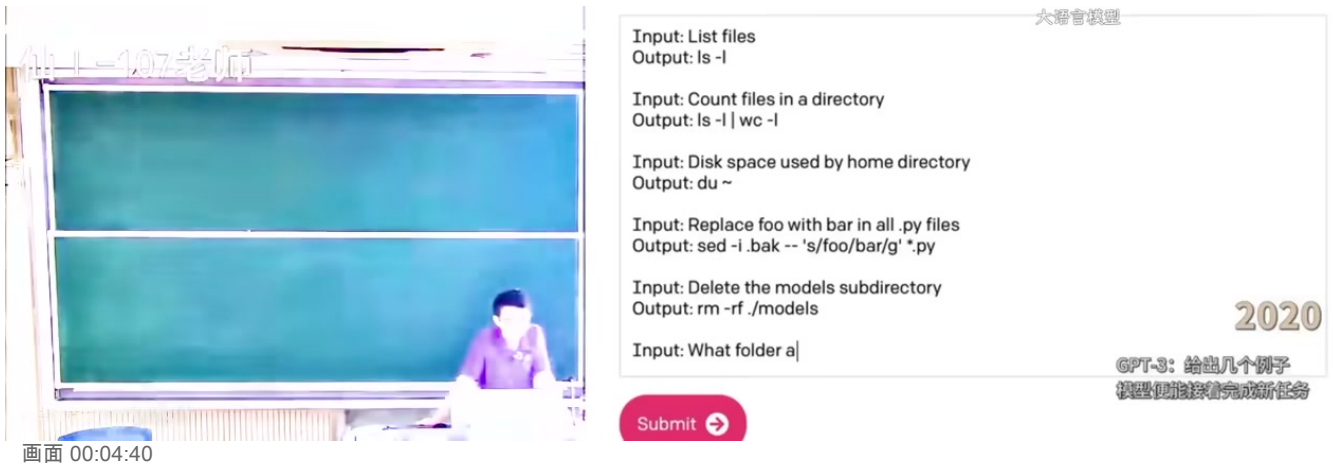


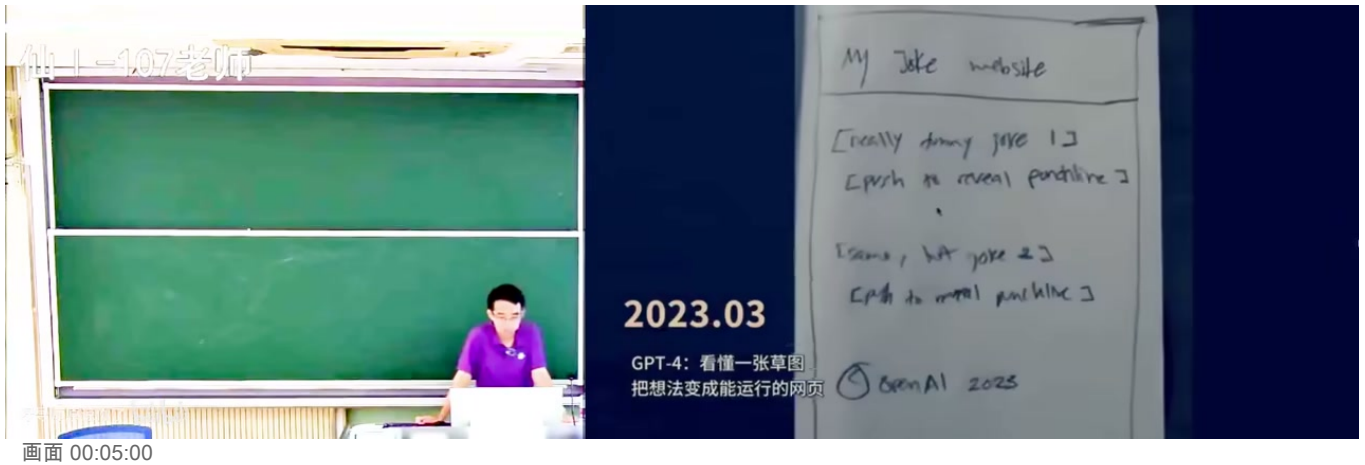




00:04:30

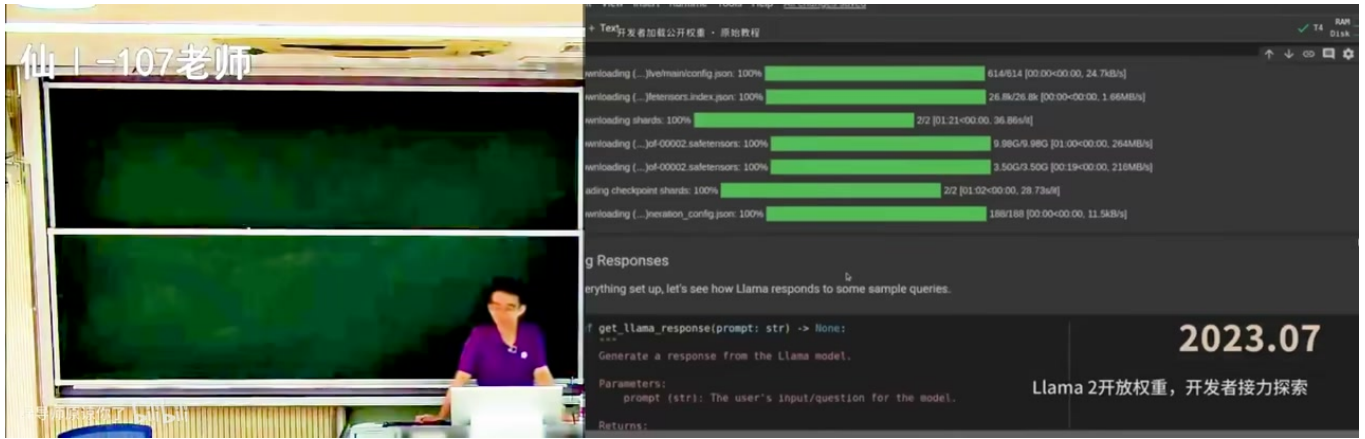
你看短短的2017年那个时候，你们是在上高中吗？还是在上初中？都没有。现在已经2026年了，这已经是接近十年以前的事情了。然后到2020年的时候，他们训练了集地3。然后集地3当时是一个大家都觉得很吃惊，我们为什么要做一个那么大参数的模型，对吧？然后直到2022年的时候，我们知道为什么需要什么了。这个到现在也不过是四年的时间，很快的从2022年到2023年，他就可以看图了。



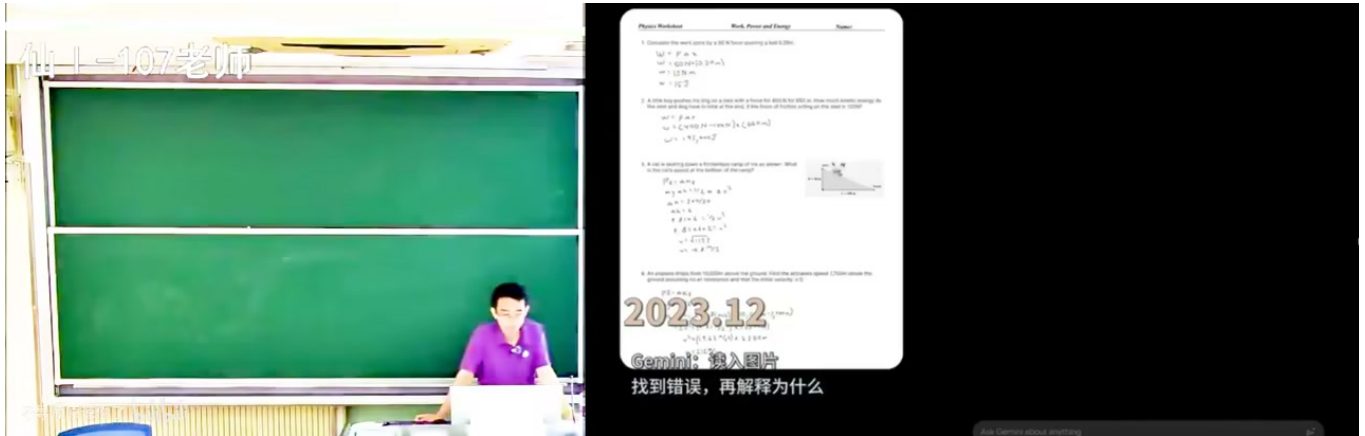


00:05:05

GP4O在生成的时候，它可以生成就可以识别语音，它可以多个模态，然后到开源模型MI。生成视频你们很难想象，就今天你看到这个抖音上面全是这个短AI生成的短视频，然后我们计算机学院楼下的那个板上也是AI生成的。就2024年的时候索尔才第一次发布对吧？2024年9月的时候才有了天耀thought的o one的模型。2024年9月你还能想象从2024年9月到现在？这个GB6s tra就在这这么短的时间里面发生了这么多的事情。第一个开源的这个deepone。



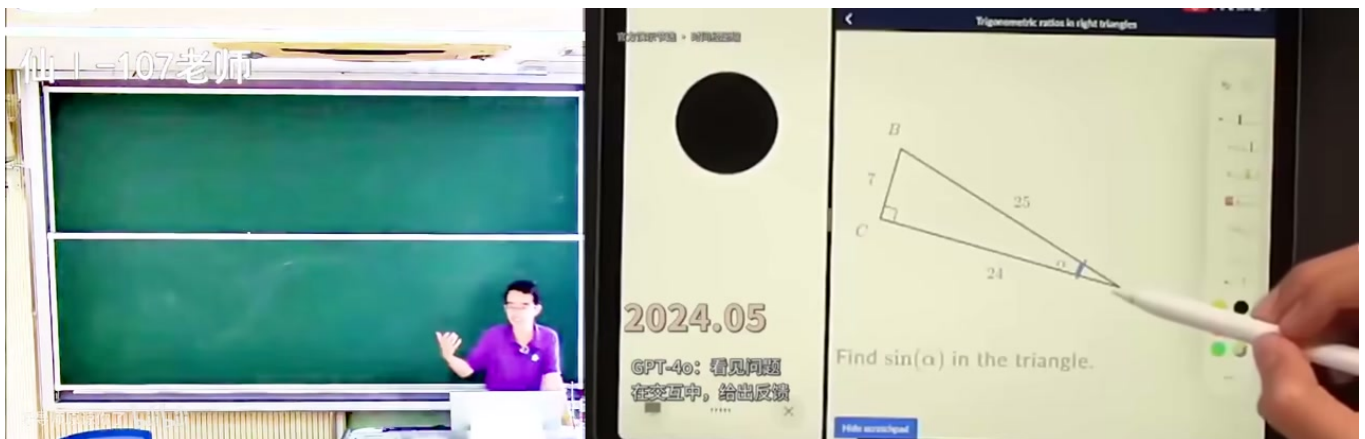
画面 00:05:06



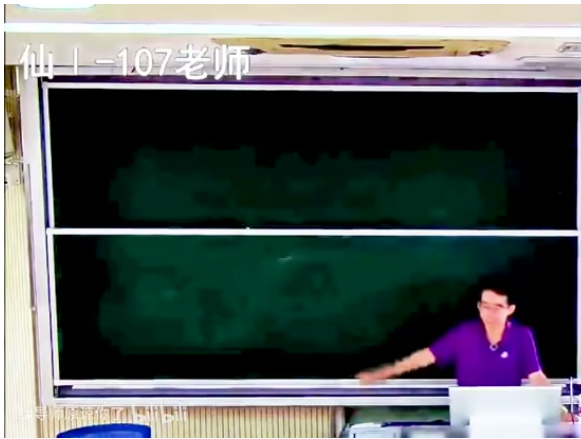
画面 00:05:10



画面 00:05:16



画面 00:05:22



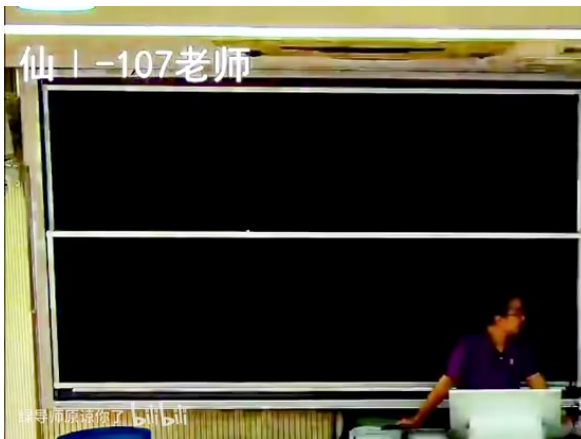
画面 00:05:26



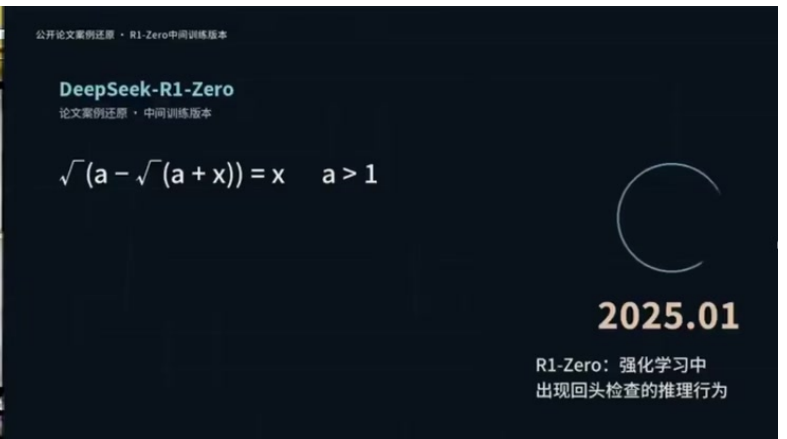
画面 00:05:30



画面 00:05:33

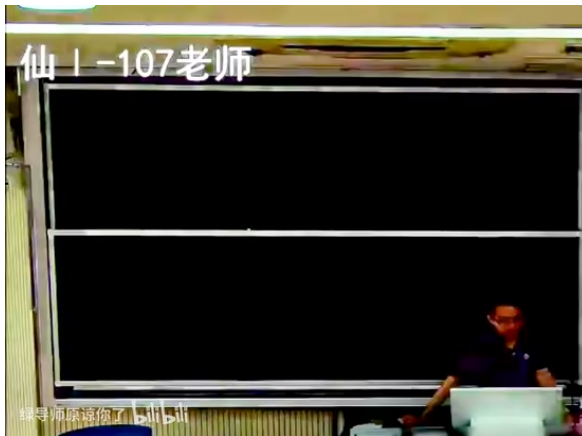
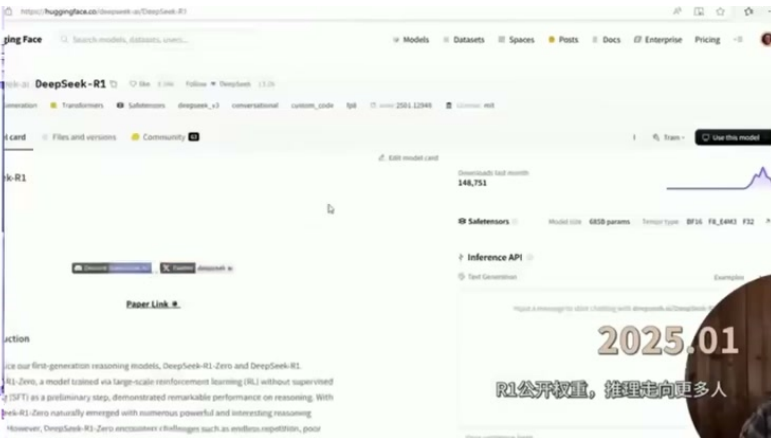


画面 00:05:37





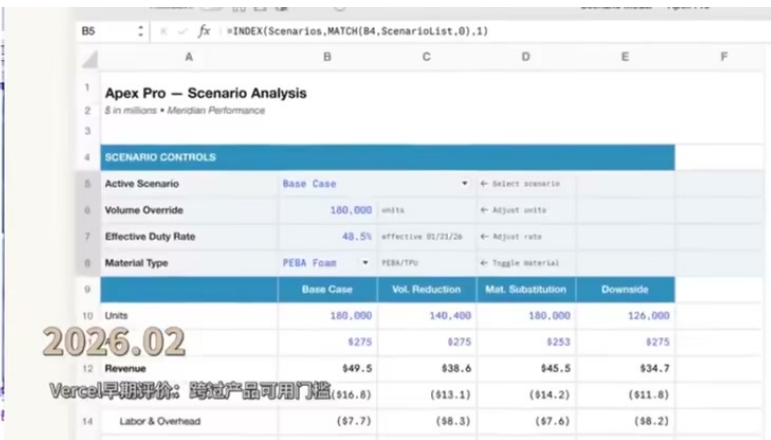
画面 00:05:42



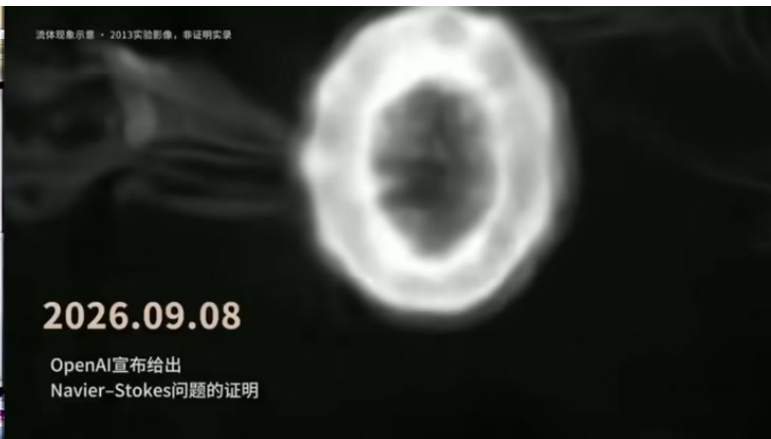
画面 00:05:51



画面 00:05:55



画面 00:05:58



00:06:00

对，这是刚刚发生的事情，对吧？Millenia problem, 然后就这么被暴力大力出奇迹给解决了。



画面 00:06:15

FINITE TIME BLOWUP FOR NAVIER-STOKES	
OPENAI	
ABSTRACT. For every positive viscosity, we construct a solution of the three-dimensional incompressible Navier-Stokes equations that starts from rest and develops unbounded velocity in finite time while maintaining uniformly bounded kinetic energy.	
CONTENTS	
1. Introduction	1
2. Physical description of the blowup	3
3. Proof outline	6
4. Constructing the leading order flow	24
5. Correcting the base flow to every order	45
6. Auxiliary torus and separation of oscillatory supports	62
7. Oscillatory realization and correction of the residual stress	73
8. Compactly supported mean corrections	88



画面 00:06:20



00:06:21

对，最后的彩蛋是这个视频完全是是，当然不知道是哪位博主直接用GPUS制作的。然后我也会想，如果我们有一个比较好比较大的素材库，其实本身比如说U tube或者哔哩哔哩，你本身可以下载，你可以用工具来下载。你就拥有了这个世界上所有的所有东西的入口。然后你创作创造任何东西都会变得很容易，对吧？



画面 00:06:28



00:06:46

然后你们有没有同学看过这个剧，这个Silicon Valley，这是个D1S1的101的名场面。他大概是几个几个伙伴们，他们要去做一件改变世界的事情。千百年来我们这种书呆子一直都被欺负，对吧？在人工智能时代终于可以出人头地。Ok anyway这个不重要，但是可以看到的是这个世界的发展在加速，就这世界发展的速度可能比大家想象的要快。所以可能我们要可能要回过头来想一想，在就业市场上面临的挑战。这也同样意味着这个世界上有很多的机遇，对吧？比如说这个jef带火的这样的一种我能看到的生态，不知道是他一瞬间就会到来，还是可能要等几年再等技术成熟一下，也许要等到硬件跟上，但是我们不知道对吧？你看到今年春晚的时候，那个没有人感的那个人形机器人。然后你到明年的时候，你就能看到一个人非常有人感，能够实时做出非常棒的微表情的机器人，有点可怕。

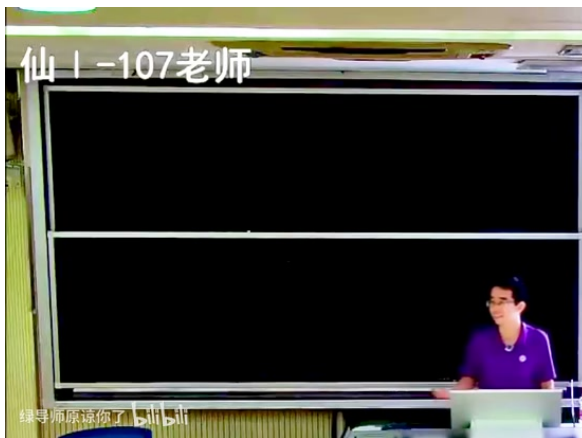
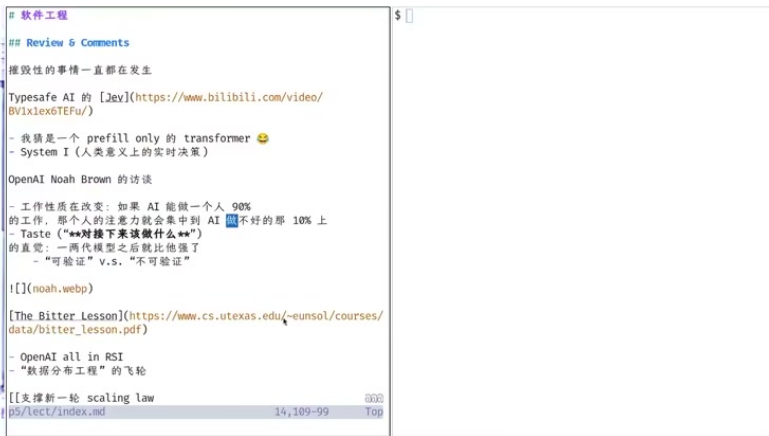




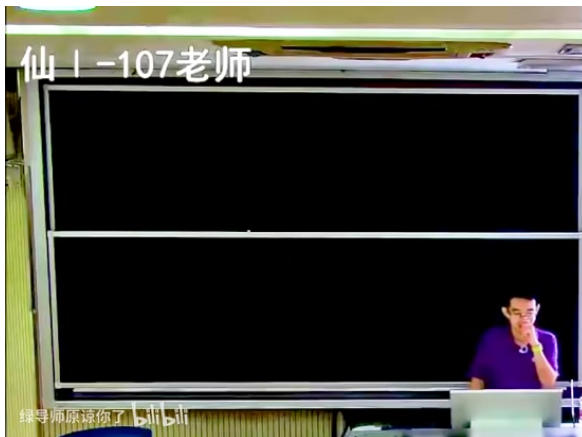
画面 00:07:00



画面 00:07:19



画面 00:08:00



画面 00:08:04



00:08:05

对，然后最近也有一个no Brown的访谈，来自OpenAI I他跟我的判断可能我不知道他泄露了多少，就是他觉得可能虽然现在的AI模型的短板是也是品味。也就是说他对这个问题的定义是说模型对接下来应该做什么的直觉做的还不够好，我们说模型被训练成了这个prompt的舔狗，你你做什么他就会盯着你那个就疯狂的做。然后同时他也有一点点这种reward hacking的倾向，会快速用最快速最脏的方式把你想要的任务完成。因为对他来说，他把任务完成才得到reward了，那他就不会不断的会往这个方向去强化。有的时候他生产的东西从长期主义角度来看，他是可能不满意的这就是一个对接下来该做什么的直觉，对吧？他现在因为强化宣训练，有学习有反馈的信号，他就会执着的去做立即有反馈的东西。但是同样我觉得没有什么道理，根本道理我们不能用更多的合成的数据去告诉模型，对吧？你你你在做这件事情的时候，你可以想的更远一点，无非就是你圈soft的时候，那头几个token决定你要你要你要做什么，对吧？



工作性质在改变

OpenAI Noam Brown 的访谈

- 如果 AI 能做一个人 **90%** 的工作，那个人的注意力就会集中到 AI 做不好的 **10%** 上
- Taste (“对接下来该做什么”的直觉)：他预计，一两代模型之后就比他强了
 - “可验证” v.s. “不可验证”



2/28



The Bitter Lesson

OpenAI All In RSI

- The Bitter Lesson
 - “数据分布工程”的飞轮

支撑新一轮 Scaling Law：数据分布工程

- 学什么、何时学：在什么阶段，接触哪些经验
- 怎样学：比例、难度、顺序与训练方式
- 持续调整：根据学习进展、跨域迁移与泛化表现，改变训练分布
 - DoReMi 的实验：某领域的训练占比降低后，该领域的困惑度仍然可以改善

3/28

00:09:23

然后他也提到了可验证和不可验证的边界。比如说你觉得可能品味是一个不可验证的东西，但也许没有那么难。后但觉得数学是一个可验证的东西，是可能验证也没有那么的容易。因为你就算是把它formalize了，你的auformalization再从你的一个自然语言描述的问题到这个形式语言描述的问题之间还是有gap的。就像前段时间有一个很有意思的笑话，有人声称证明了哪一个黎曼猜想还是P等于NP还是P不等于NP但是后来大家发现他那个形式化证明里面，等于说他定义那个问题就是一个nothing不存在的定义。然后他说我证明了一个就就证明了一等于1这样的东西，然后最后他写了大概几千页的证明，然后最后证明一等于1。有这个感觉。



画面 00:10:00

00:10:07

Ok, 所以对我觉得对我们的启发来说是, 比如open I他说他要in IRRSI, 然后那我们就回头再想, 其实the better lesson到今天就是scaling law, 到今天可能还没有停止。就比如说我看到g p 6它发布以后, 我最大的感受就是可能欧I摸到了一个门道。这个门道我不知道这个应该叫什么, 大概就叫我我可能叫他数据分布工程。也就是说吉P6它有我我前面说过, 它有非常惊人的空间推理能力, 对吧? 它它可以理解一个图片里面的空间关系, 然后从而他在画图方面, 在生成模型方面都做的都非常好。毫无疑问是因为他在训练的时候肯定加入了这部分的数据, 可能基模训练的时候就加了这些数据。

工作性质在改变

OpenAI Noam Brown 的访谈

- 如果 AI 能做一个人 **90%** 的工作, 那个人的注意力就会集中到 AI 做不好的 **10%** 上
- Taste ("对接下来该做什么"的直觉): 他预计, 一两代模型之后就比他强了
 - "可验证" v.s. "不可验证"



2/3



画面 00:10:07

The Bitter Lesson

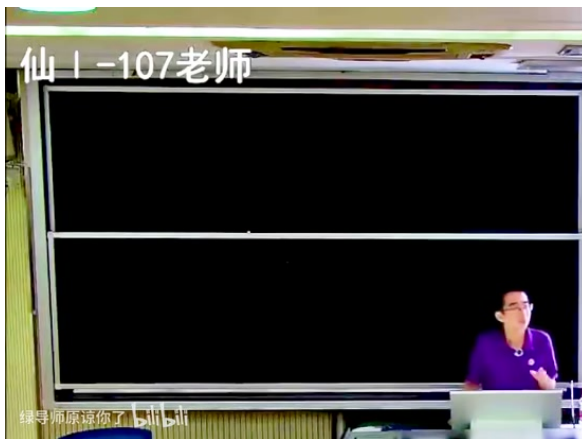
OpenAI All In RSI

- [The Bitter Lesson](#)
 - "数据分布工程"的飞轮

支撑新一轮 Scaling Law: 数据分布工程

- ④ **学什么、何时学**: 在什么阶段, 接触哪些经验
- ④ **怎样学**: 比例、难度、顺序与训练方式
- ④ **持续调整**: 根据学习进展、跨域迁移与泛化表现, 改变训练分布
 - ④ **DoReMi** 的实验: 某领域的训练占比降低后, 该领域的困惑度仍然可以改善

3/3



画面 00:11:00

The Bitter Lesson

OpenAI All In RSI

- [The Bitter Lesson](#)
 - "数据分布工程"的飞轮

支撑新一轮 Scaling Law: 数据分布工程

- ④ **学什么、何时学**: 在什么阶段, 接触哪些经验
- ④ **怎样学**: 比例、难度、顺序与训练方式
- ④ **持续调整**: 根据学习进展、跨域迁移与泛化表现, 改变训练分布
 - ④ **DoReMi** 的实验: 某领域的训练占比降低后, 该领域的困惑度仍然可以改善

3/3

00:11:05

然后同时他在后训练的时候，强化学习的时候，也有更多的数据帮助他去理解这个空间，给他反馈说这样做是好，这样做是不好，或者对和错。另外一个维度的sclaw就变成了以前我们说好大模型law师，我有更多的训练数据，它就可以寂寞，就可以变得更好。所以去买书，然后去烧书对吧？他只要买了书，烧了书，他有了绝版的数据，他就可以比别人别人更强。然后在强化学习的时代，我觉得他们看到一点，如果我们有一个机模，他已经做的很好了，我们可以再加一个他没有见过的领域，然后强化学习，然后他就会在所有的任务上都做的更好，这是一件非常恐怖的事情。

00:11:49

你看到就刚才我们看的那个视频说，模型是一代一代怎么怎么发展过来的。那最早的时候模型就完全就是语料对吧？它是这个语言的token large language model。然后他是先训练了语言，然后再训练了可能视觉的多模态，训练了一部分，然后他训练的数学？你看这人工智能就是大圆模型，它发展的脉络是先从语言到数学语言，然后再到多模态。他学会了数学以后，他才再学会空间感知的关系，然后它可能就不需要那么多数据。

仙！-107老师

画面 00:12:00

The Bitter Lesson

OpenAI All In RSI

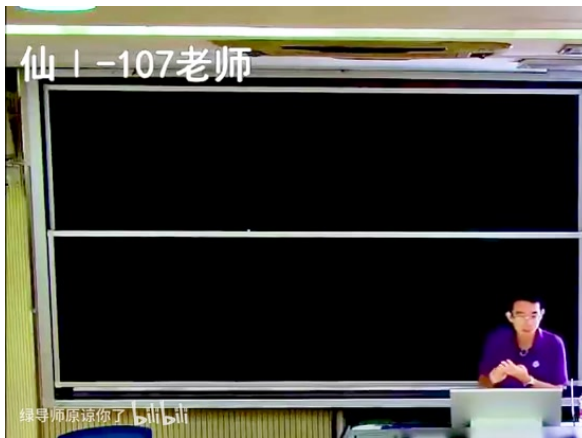
- [The Bitter Lesson](#)
 - “数据分布工程”的飞轮

支撑新一轮 **Scaling Law**：数据分布工程

- 学什么、何时学：在什么阶段，接触哪些经验
- 怎样学：比例、难度、顺序与训练方式
- 持续调整：根据学习进展、跨域迁移与泛化表现，改变训练分布
 - DoReMi 的实验：某领域的训练占比降低后，该领域的困惑度仍然可以改善

00:12:23

因为如果你从零开始训练一个比如说矩型智能的模型，它很容易什么呢？你会先要有大量的模拟器数据，然后模拟器数据我们的模型是一个rehacker，它就很容易根据你的模拟器里面的，比如说光照的那种渲染的一点非常细微的，比如说那个脚上的光像素点是亮还是不亮，它就能够得到这个rehack的反馈信号，它不能泛化。但是如果你是一个大模型，一个基座很大的基座模型，它既会语言又会数学，还会写写各写程序。如果你在这样的一个好的基机动模型上面，再给他做空间感知的训练，可能就不需要那么多数据了。所以每家一个就就我猜测你每加一个新的人类的领域，它需要的数据不是更多，反而是更少，它就能达到一个泛华的效果，这是件非常恐怖的事情。然后人跟模型是反过来的，人是先学的世界模型。



画面 00:13:00

00:13:13

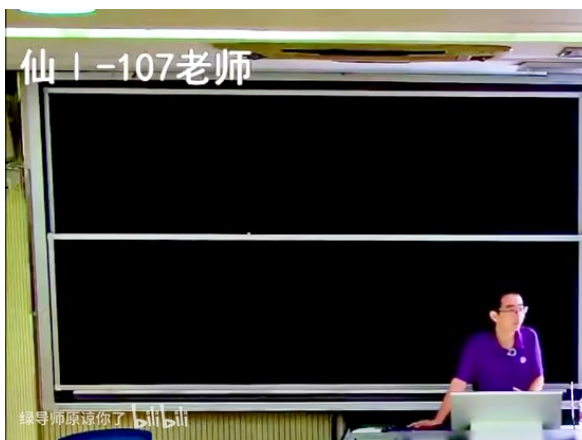
我们在刚长刚婴儿出生的时候，我们就和这个世界交互，先学的是世界模型。然后你看他要到训练一年、两年、三年的世界模型以后，他才能开始说话。你要幼儿园的时候，小朋友都还说话，都不利索。它是先有世界模型，再训练语言，再训练数学，再比如说变成我的读一个PhD训练这个research test，然后大圆模型反过来。但我们看到的是不管按照什么顺序训练，只要有足够多的数据，你有scaling law，这个模型就能成对吧？

00:13:45

大模型成功了，人也成功了。所以他们可能看到了这个数据分布工程上的这样的一个飞轮。他只要决定在什么时候给模型，按照什么样的比例注入数据。

00:13:57

模型的架构不重要，算法不重要，只要训练不崩掉，只要这个训推能够一致，对吧？我不能训练的时候，一个参数推理的时候，它这个参数变了。只要这些工程性的问题能解决，它只要有十倍100倍的数据，这个问题就解决了。可能算法就你用一个更好的算法，用更好的模型结构，可能会能够有一定程度的提升。但是对他来说，只要这个飞轮还在，只要他有钱，他就可以无限的scale下去。所以这是一个非常恐怖的事情。积木厂说一方面我们比如说二今年投稿的数量到达新的巅峰了，对吧？学术界更多的用这个AI I stop p来充满这个，对，因为同学们都想发论文，你们如果想保研的话，你们都一定会想发一篇论文。



画面 00:14:00

The Bitter Lesson

OpenAI All In RSI

- [The Bitter Lesson](#)
 - “数据分布工程”的飞轮

支撑新一轮 Scaling Law: 数据分布工程

- ④ 学什么、何时学：在什么阶段，接触哪些经验
- ④ 怎样学：比例、难度、顺序与训练方式
- ④ 持续调整：根据学习进展、跨域迁移与泛化表现，改变训练分布
 - ④ DoReMi 的实验：某领域的训练占比降低后，该领域的困惑度仍然可以改善

5/18

The Bitter Lesson

OpenAI All In RSI

- [The Bitter Lesson](#)
 - “数据分布工程”的飞轮

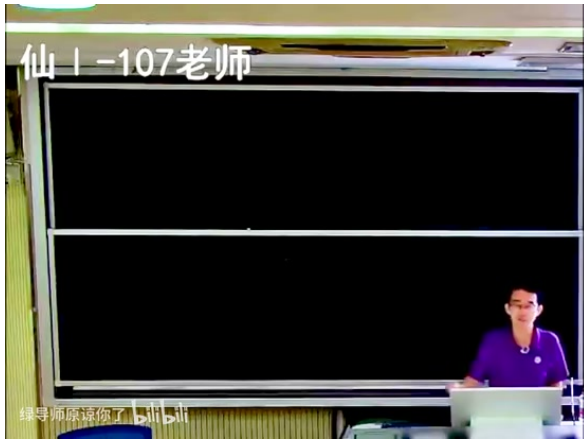
支撑新一轮 Scaling Law: 数据分布工程

- ④ 学什么、何时学：在什么阶段，接触哪些经验
- ④ 怎样学：比例、难度、顺序与训练方式
- ④ 持续调整：根据学习进展、跨域迁移与泛化表现，改变训练分布
 - ④ DoReMi 的实验：某领域的训练占比降低后，该领域的困惑度仍然可以改善

5/18

00:14:41

然后另外一个方面是这个积木厂已经不care这些东西。他说你们在一个小规模上做任何实验对他来说都没有意义。因为他看到了另外一个维度的law，然后你不在积木厂就没有没有任何资源可以做这件事。你对这个分布，比如说在什么时候加进这个数据一无所知。然后你就算是能做一些实验，在一些小规模上有一些非常酷的想法，可能你也不需要写一篇paper对吧？你只要写一个blo然后，你今天只要写一个blog post，然后他们的那个AI看到这个blopost就可以他们到他们的数据上去试一试对吧？



画面 00:15:00

The Bitter Lesson

OpenAI All In RSI

• The Bitter Lesson

- “数据分布工程”的飞轮

支撑新一轮 Scaling Law: 数据分布工程

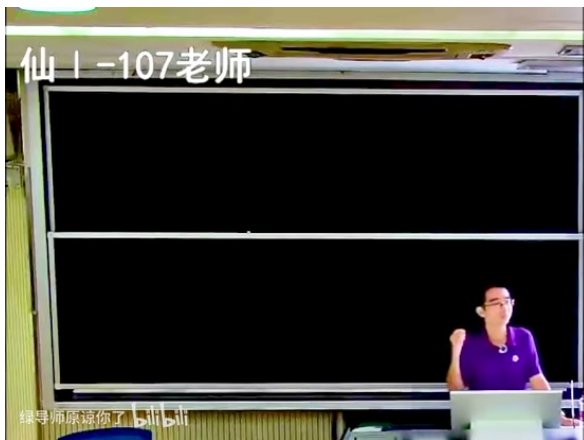
- 学什么、何时学: 在什么阶段, 接触哪些经验
- 怎样学: 比例、难度、顺序与训练方式
- 持续调整: 根据学习进展、跨域迁移与泛化表现, 改变训练分布
 - DoReMi 的实验: 某领域的训练占比降低后, 该领域的困惑度仍然可以改善

00:15:14

他们有RSI的loop，就他们先要构成一个人工智能上面recursive improvement的一个循环，然后他靠这个人工智能循环，他就可以去做任何事儿，然后飞轮就登起来了，有点恐怖，我觉得这是一个他们要做自动化的能力，能力产生。一旦这个闭环被他们探索的基本上接近收敛，而且在AI的帮助下，这个闭环我觉得肯定会很快的收敛。因为A比任何一个人类都要强，他就可以去发现找一个新领域，对吧？我不知道下一个领域是什么，情感计算还是这种化学。

00:15:52

因为我听说看到消息说anropic要做一个White lab，去做这个生物工程的实验。那也许生物工程或者比如说甚至预测这个老鼠的行为？你就成为了一个下一个sk的领域。然后他在这个领域上面可以建立反馈，优化训练分布，就把这个领域以一个插件的形式插进他的筋模训练里，然后这个飞轮转起来，这就非常恐怖了，对吧？



画面 00:16:00

数据分布工程的飞轮

自动化能力生产过程

- 增强后的模型参与寻找领域、构建环境与验证器，并优化数据分布与训练方法。

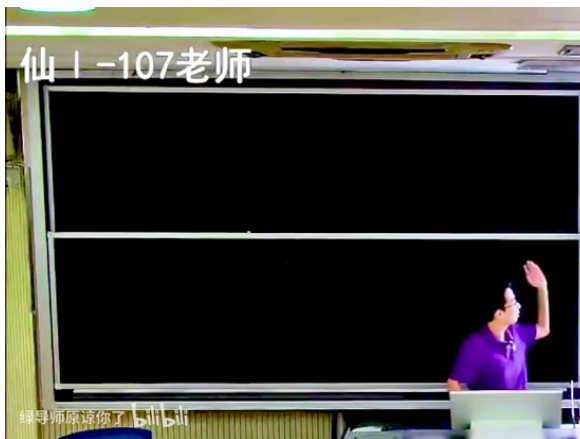


00:16:22

他每次就是说我要新建一个实验室，投资几十亿美元新建一个实验室，然后他的模型能力就往上抬一个台阶。这是一个几乎可预见能够有收益的。而不是像我们现在学术界，比如说写个paper，就是我随机的想一个idea，这个有没有用不知道。然后我要去发一篇paper，然后接受这个perefview。然后我觉得这个体系马上就要崩溃了。大每个同学都可以想一想，在这个崩溃的体系下，我们还能还能剩下什么？

00:16:51

对，然后我也理了一些，可能我跟AI聊了一会儿，然后比如说这些至少这些都是可以作为一个小的可能是一个小的领域来做验证的。比如说你可以在系统架构上面去训练品味，让他做出好的系统设计。然后很快？可能到我这个学期结束的时候，他们突然推出了下一版的模型，把系统设计加进去了。后然后你就看到我我经常在课上说你笑话AI生成的这个，说你看它价格多么糟糕，我加一个需求他就崩溃了。



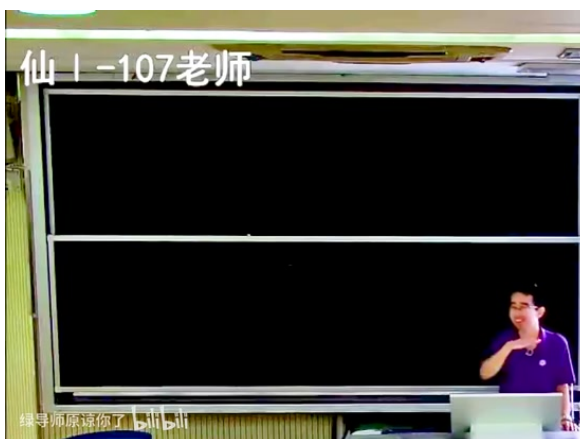
增加可训练领域

把更多智力活动变成可以规模化训练的对象

- 空间变换、图形规则归纳、几何构造
 - 生成器与约束检查：构造任务，验证结果
- 系统架构、并发协议、故障诊断
 - 运行、模型检查、故障注入与需求变化：检验方案
- 主动实验、记忆管理、需求澄清
 - 后续预测与任务结果：评价此前的选择

00:17:31

后但这些事情可能就永远都不会发生了，对很有意思。但不管怎么样，我们还是需要上课的。需要上课是因为你你还是不能完全的依赖AI或者说你人类是一个竞争的社会，它和生物生我们就我们所在的环境，生物的进化很像是survival of the你是适者生存的。所以这个斩杀线就毫无疑问，这个斩杀线是不断的向上移动的。然后你每个人就会就是说我终于怎么样才能逃离这个斩杀线，然后在最后还没有被斩杀的时候搭上最后一班车。



Review: 版本管理

软件是 Intent、Spec、Impl 的组合物

- 以文件系统快照的形式存储 (版本管理)
- 所谓“软件开发”，就是不断推出新的版本，直到满足客户需求
 - 这其中很自然会遇到分叉、冲突.....
 - 于是你能理解 SVN、Git、Jujutsu、.....的设计原因

从零开始构造软件

- 上次起了个头：从空的项目 (模糊的 intent) 开始，我们可以做什么？

00:18:07

因为人类就我觉得从人类的角度来讲，人类肯定不希望自己消亡，对吧？不管是这还是open a还是deep他们，都希望的是靠人工智能去推动人类的进步。当然这个斩杀一定程度上会使很多人的生活没有生命变得没有意义。但是我们成为一个更好的人，我们就可以更好的找到这个意义，对吧？那这也是没有办法的事儿，我们每个人都是这样，包括我也是我也是这样。

00:18:33

Ok所以我们还是回到技术上的内容。我们上一次讲了版本管理，然后我花了好多次课来给大家说我们的软件。软件所面临的问题是我们的intent，就是你想要什么，我们的implementation。它这三个之间是有gap的。然就是因为有这个gap，所以构造软件就成为了一个世纪性的难题。

00:18:56

也有了软件工程，然后我们起了个头说。如果你现在就要从零开始构造一个系统，那我们就需要管理软件。所以我讲了一些这个版本控制的东西。



仙 I -107老师

Review: 版本管理

软件是 Intent、Spec、Impl 的组合物

- 以文件系统快照的形式存储 (版本管理)
- 所谓“软件开发”，就是不断推出新的版本，直到满足客户需求
 - 这其中很自然会遇到分叉、冲突.....
 - 于是你能理解 SVN、Git、Jujutsu、.....的设计原因

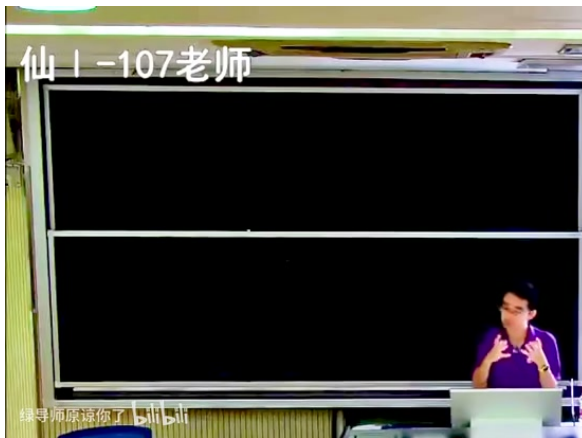
从零开始构造软件

- 上次起了个头：从空的项目 (模糊的 intent) 开始，我们可以做什么?

画面 00:19:00

00:19:08

接下来就是如果我们真的想要构造软件，而且版本管理也有了，那我们到底应该怎么怎么构造呢？当然我也起了一个头，有一个答案，就今天的答案是直接做一个minimum viable product，一个M1P一个最小原型。这件事情对大家来说是非常直观的对吧？我们做的多就错的多，我做的少我就错的少。所以我做一个最少的，但是又和我最终的那个product最接近的，那应该是收益比较大的。因为它可以帮助我去补上这个intenspecification implementation的这个gap，因为我做一个MVP首先是AI I生成的slop，所以它也不怎么花钱，这个硬盘配置我随时可以扔掉，这个不重要。但是它能够帮我我知道我的intent哪里没有弄对，然后我的比如说架构还有没有可以改进的空间，然后再做几个版本的迭代，最后再去再去出来。



画面 00:20:00

今天的答案：直接做一个 MVP

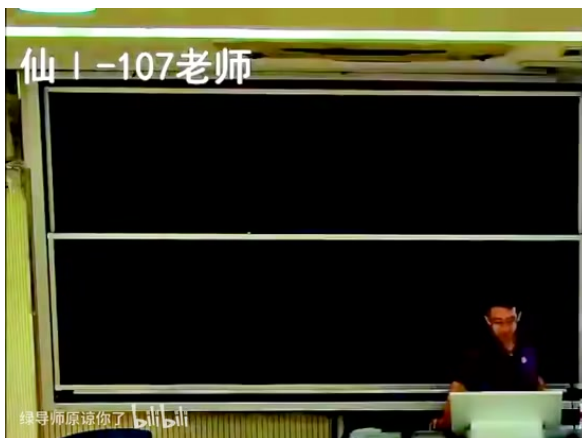
Office Assistant (GLM-5.3 直出)

- “软件”公司的彻底消亡 (Jev, 豆包手机, ...)
- 以后可以想象：说好需求，自动下单零部件，3D 打印定制件，回家 Lab 就建好了



00:20:06

比如说我那天晚上做了这样的一个东西做这样的一个东西，所有东西都是AI slot。我就是开着豆包输入法，然后开了几个终端，然后来回骂过来，骂我一圈一圈骂过来。这是一个和物理世界有交互的这个东西，对吧？我有一个标签打印机，它他就把标签就打出来了。本来就lab里面有一些之前买的各种各样的设备，然后每个设备大家都要用，大家都用那个设备自带的软件去使用，然后那个软件用起来就像吃屎一样。所以我们干脆就不要这个软件了，我就把所有的这些软件都屏蔽掉。当然这也是AI I做的，相当于做一个抽象对吧？这这其实是有sorry这其实是有一些软件工程智慧的。



画面 00:20:07



画面 00:20:54

今天的内容

但其实曾经的软件工程不是这样的 😊

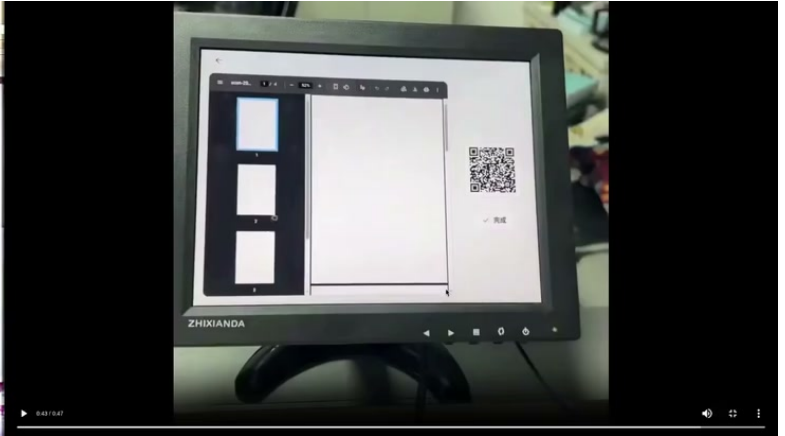
- 有必要了解一下“构造软件”的历史中，大家发现、发明了什么
 - 软件工程的诞生
 - 软件工程的智慧
 - UML

00:21:00

我们有一台计算机，然后这个计算机上总线上连了一些设备，这是操作系统课程，anyway不重要。扫描仪、打印机各种各样，你能想象的想象任何东西。然后这个码是可以扫的，这个我会就整个流程它会自动，你们不需要扫码。这个这是一个白的，它会上传到我们SS自己的那个那个主页上。我们为什么不能对它做一个抽象？它就是一个就我们说我上操作系统课的时候会说什么设备驱动？把一个设备抽象成read right，are you control的一个接口，那我能不能把它抽象成一个更好用的接口？比如说扫描仪，我就是个scan对吧？



画面 00:21:00

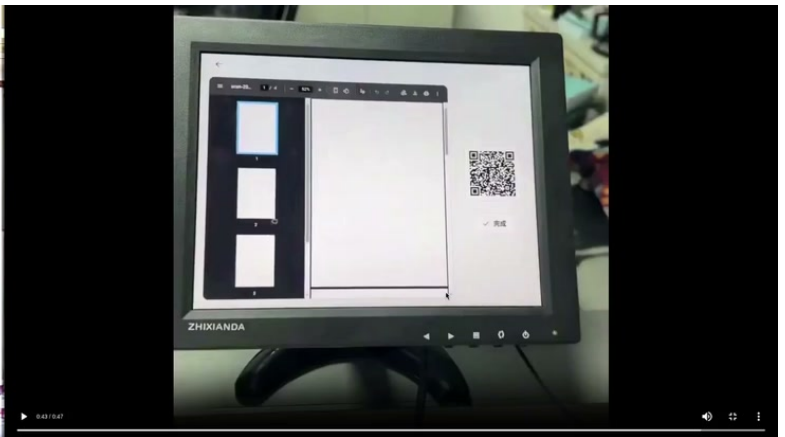


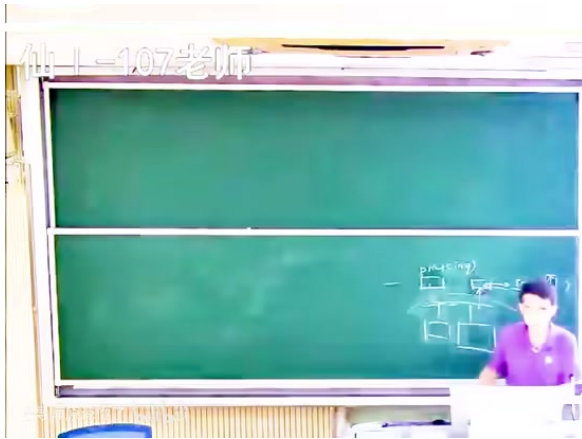
00:21:41

我希望的是我是正面向上还是下面向上，然后我要单面还是双面，我D是多少，我就给他一个更干净的接口，然后我的这个打印机也是的，我的接口就是一个print一个image对吧？当我有这个接口以后，我我我的实设计和实现就完全解开了。然后我的软件层就可以工作在这个接口层上。对，这是一个非常典型的软件工程里的你的设计，然后这样我就再也不用不需要去用那些不好用的软件了，对吧？然后这就是一个self service，然后你看到的是什么？AI I出现以后，而且我推意是用的是国产的模式，因为正好他我额度是空的，然后我就正好把5个小时额度基本上全部登完了。然后就得到看起来是一个非常可用的，就跟你在食堂或者宿舍楼下看到那个机器达到了某种产品级的效果的这样的东西。所以就是以后软件公司就是面临一个非常彻底的消亡，对吧？



画面 00:22:00

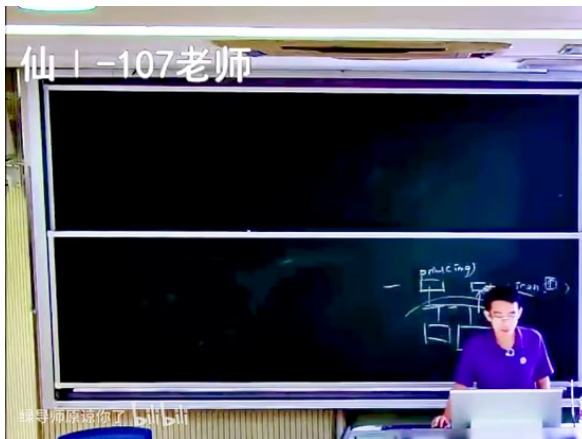




画面 00:22:17

00:22:47

你想他，我不知道你们有没有装过这种，比如说标签打印机都会给你一个驱动，给你一个软件，它的开发成本是非常高的。然后像我的这个扫描仪是佳能的扫描仪，然后这个扫描仪的成本软件成本也是非常高的。他要开发对吧？还要跨平台的驱动等等等。但是在今天这些通通都不需要了，这里有代码。我直接让他解释一下。



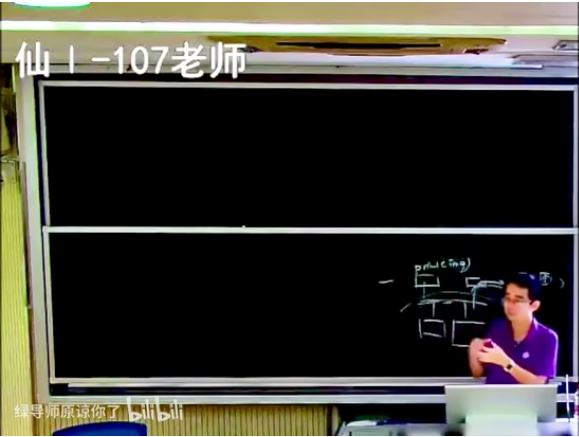
画面 00:23:00

00:23:24

打印机和。那我直接让AI帮我解释，实际上这个打印机，这个照片打印机是个非标的东西，就没有就我不知道它是什么，其实我不知道它是什么，也是相当于是一个杂牌的对吧？你看它是一个叫easy scan，不算是一个特别标准的设备。然后我就直接让告诉AI说，我就想要把这个打印机驱动起来，他都不知道。

00:23:56

就比如说这个打印机里面有一个选项，它是热转印的对，就那种标签。你们看到那种贴固定资产的标签，然后那个标签它是一个碳粉袋，然后那个碳粉袋会加热，就相当于是在有像素的位置加热，然后把那个碳粉烫上去，这样热转印的一个打印机。然后那个热转印打印机有一个参数是大概是加热的程度或者就是深度，就是图像的深度。然后我们不知道这个参数是多少，意思，应该设什么。



画面 00:24:00

00:24:27

然后AI I就不停的打标签，他先打一个上面有数字，比如说这是第15次尝试十五，然后十五然后问我是淡了还是浓了，他看不到。但这个完全如果接个摄像头他就可以不用，对吧？我把一个摄像头对着那个，他就不需要，甚至连我都不需要，他自己一张一张一张打，他就可以完全自动把把这个东西给完成，对吧？这是一个他一开始尝试的是试图用这个LP因为能识别到这个。然后后来他发现他可以直接用这个PCR的驱动去一个generic的打印，把它当成一个通用的打印机，然后来来来处理好，我就完全没有监管，我只是问他要一个结果，是一个可验收的结果。A阵就可以一轮一轮的去把这个事搞定。然后搞定了以后，然后在外部一个前端界面，我就是用嘴输出就可以。所以看到这个软件会发生根本性的变化。

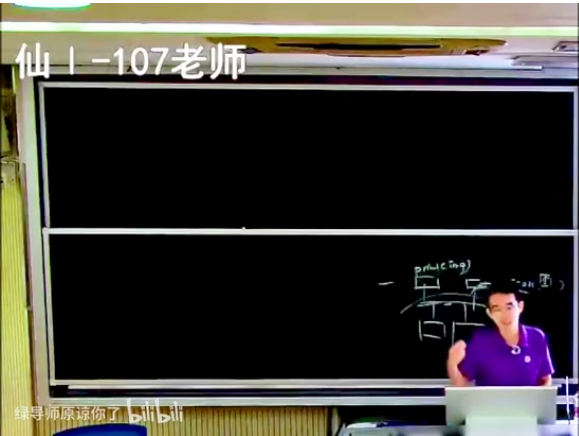
Thinking...

结论：两者互不连接，各自以 USB 连接到一台主机（Linux 机器），在软件里是两条独立的通路。

1. 物理/驱动层（各走各的）

	标签打印机	扫描仪
设备	EZSCAN / ZMIN GL70	Canon imageFORMULA R40
USB ID	03eb:9638	1083:1679
内核路径	usb/lp 驱动 → 字符设备 /dev/usb/lpN	libusb 直连 USB (SANE 的 canon_dr 后端)
应用访问	直接往 lpN 写裸字节 (ZMIPL1 指令+1bit 位图)，不经 CUPS	scanimage 经 SANE 读图
权限	impl/misc/udev/99-zmin-gl70.rules (usbmisc+usb 两条都设 0666)	impl/misc/udev/99-canon-r40.rules (USB 设 0666, 否则节点 root-only)

发现方式也不同：
- 打印机 zmprint.find_printer() 遍历 /sys/class/usbmisc/lp*
- 扫描仪 zmprint.find_printer() 遍历 /sys/class/usbmisc/lp*



画面 00:25:00

00:25:22

最近也发布了豆包手机，豆包手机也是一个我觉得非常好的工程产品，因为它它也体现了同样的软件工程智慧。他说的是我们这个app都不重要了，对吧？这些app都不重要。如果我们有一个jam这样的模型？我觉得J和豆包手机真的是非常配，就是一个决策，但是又是接近实时的。所以我觉得他走在前面，他赌这个模型，当然可能我们也都看到这一点了，他赌模型可以在有一个时间点快到可以和人一样快点手机。

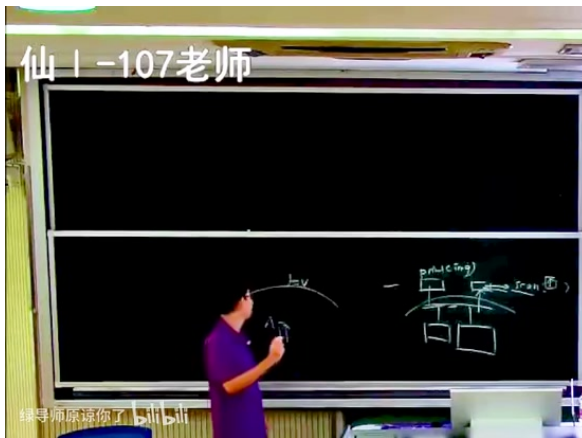
00:25:55

虽然当时他做这个豆包手机的时候还不能，但这件这一天很快就会来了。Ok所以他就把app抽象成了各种各样的接口，然后这个接口反正它有computer use去调用。我要点外卖对吧？我要点外卖他就直接好应用，就沦为了一个点外卖的接口，沦为了一个查询数据的接口。所以这些app是不是有一天全部都要死掉了，因为它只要提供接口就行了，对吧？

	标签打印机	扫描仪
设备	EZSCAN / ZMIN GL70	Canon imageFORMULA R40
USB ID	03eb:9638	1083:1679
内核路径	usb/lp 驱动 → 字符设备 /dev/usb/lpN	libusb 直连 USB (SANE 的 canon_dr 后端)
应用访问	直接往 lpN 写裸字节 (ZMIPL1 指令+1bit 位图)，不经 CUPS	scanimage 经 SANE 读图
权限	impl/misc/udev/99-zmin-gl70.rules (usbmisc+usb 两条都设 0666)	impl/misc/udev/99-canon-r40.rules (USB 设 0666, 否则节点 root-only)

发现方式也不同：
- 打印机 zmprint.find_printer() 遍历 /sys/class/usbmisc/lp* 读 /dev/usb/lpN，匹配 MFG:EZSCAN ;MDL:GL70，返回 /dev/usb/lpN。
- 扫描仪 scripts/scan/find_device() 用 scanimage -L: 为避开 net/esc/airsn 各超时 ~10s，它建了个临时 SANE_CONFIG_DIR，只启用 canon_dr 后端再探测，并优先取 libusb: 设备。

2. 应用层（谁把它们串起来）



画面 00:26:00

00:26:21

他提供一个app I是给是提供一个app是给人用的。但他如果只要提供一个app I就可以给agent用。所以你看
到比如说阿里在疯狂做这件事儿，阿里在疯狂的把它的所有旗下的服务都整合整像agent整合。

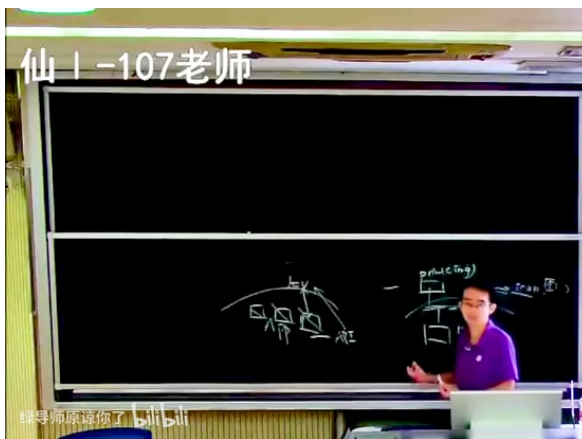
00:26:36

试图用一个千问这样的应当是千问，我觉得有点生不逢时，他他太想把这个产品推出了，然后这个时候他模型它的算力可能也不够，也没有他们可能自己也没有一个jff这样的智力又足够而且又足够快的模型来帮你完成各种各样的任务。所以拿出来像一个未来的半成品。但是这也意味着未来总有一天肯定会来的对吧？然后软件整个软件公司就彻底消亡。你想这个产业链上养活了多少人，多少外包公司，对吧？你在就这个标签打印机，我看标签打印机，对，我找一帧稍微好没有，没事，就这样的一个标签打印机，它为了推出就你想象你是一个你毕业了要去想要去创业，然后你想要去创业，你说我要卖一个就找到一个细分市场。比如说你发现每一个学校，都每个单位都需要一个这样的标签打印机。但这是一个不错的市场，你可能卖几百块钱一台，还是相当有有利润可赚的。

今天的答案：直接做一个 MVP

Office Assistant (GLM-5.3 直出)

- “软件”公司的彻底消亡 (lev, 豆包手机, ...)
- 以后可以想象：说好需求，自动下单零部件，3D 打印定制件，回家 Lab 就建好了



画面 00:27:00

今天的答案：直接做一个 MVP

Office Assistant (GLM-5.3 直出)

- “软件”公司的彻底消亡 (lev, 豆包手机, ...)
- 以后可以想象：说好需求，自动下单零部件，3D 打印定制件，回家 Lab 就建好了

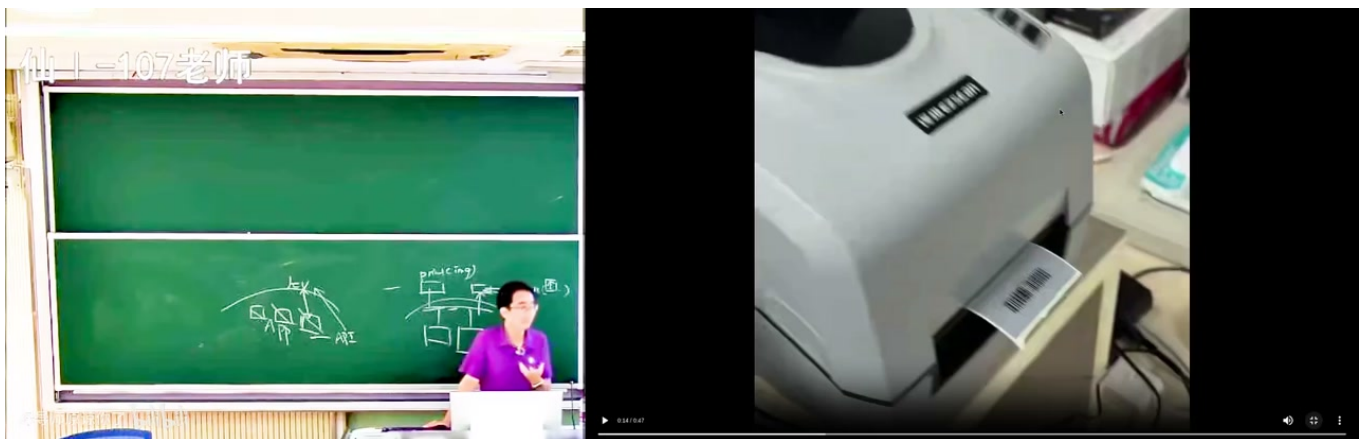




画面 00:27:18

00:27:40

硬件上因为中国供应链很好，中国你基本上这种成熟的硬件你肯定能找到供应链，然后你就需要软件，然后软件开发的成本在AI prai I的时代是非常高的。它需要驱动windows的驱动，然后windows驱动完了以后，它还需要做mac的驱动，对吧？因为因为你现在越来越多的人用mac，然后他说ma克没驱动，他就会会抱怨，然后你的产品可能就卖不了卖不了那么好。



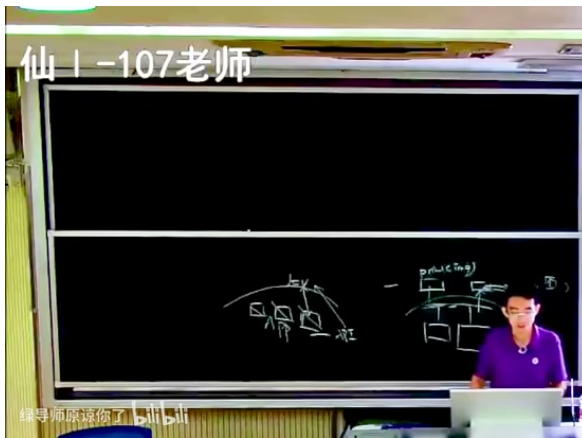
画面 00:28:00

00:28:07

但是到今天全部都直接被AI I杀死比赛，对吧？你可以看一下这个laelprint就是一个我都完全没管，这些都是AI I自己自己生成的，他自己prob了这个打印机的参数和我现在当前插入的这个标签完全是定制化的。也是说我想要什么以我最舒适的方式来帮我定制，不要通用对吧？因为以后生产软件，生产intelligence的成本会非常的低。



画面 00:28:08



画面 00:28:46

00:28:47

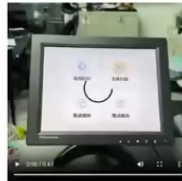
Ok好，当然这是未来会发生的事情。你们已经生在这个时代了，就是已经生在软件死掉了的时代了。那为什么我们还要讲软件工程呢？因为曾经的软件工程不是这样的。你们透过软件工程的历史，虽然今天确实是就你你们我我下一个实验，我现在还没有布置，我下一个实验就会要求大家把你们的第一个实验的那个 assistant 和一个 physical device 联系起来，就让它能够驱动一个任何一个你的可能是你的手表也可以是一个你你就你宿舍里面任何一个已有的东西，也许是一个你就想买一个，对吧？花200块钱买一个什么什么小设备，然后那个设备也是非标的，它可能也没有Linux驱动，但是你就想把它搞起来。比如说我的办公室里面有一个电子墨水瓶，然后电子那个电子墨水瓶它也是一个and卓ID的app烧写，然后那个app极度难用。当然我可以用豆包手机的方式来用computer use的方式来用它。

今天的答案：直接做一个 M

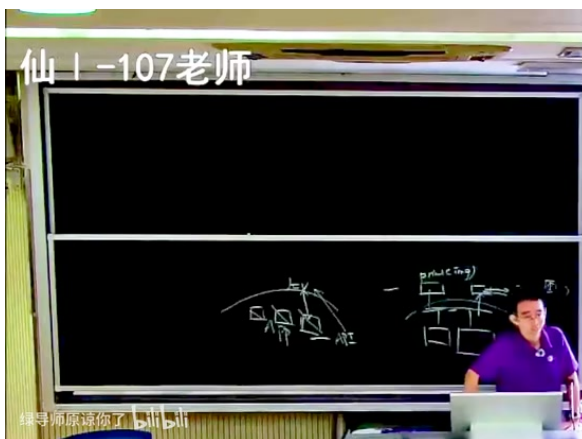
To exit full screen, press and hold [Esc]

Office Assistant (GLM-5.3 直出)

- “软件”公司的彻底消亡 (Jev, 豆包手机, ...)
- 以后可以想象：说好需求，自动下单零部件，3D 打印定制件，回家 Lab 就建好了



01/8



画面 00:29:00

00:29:52

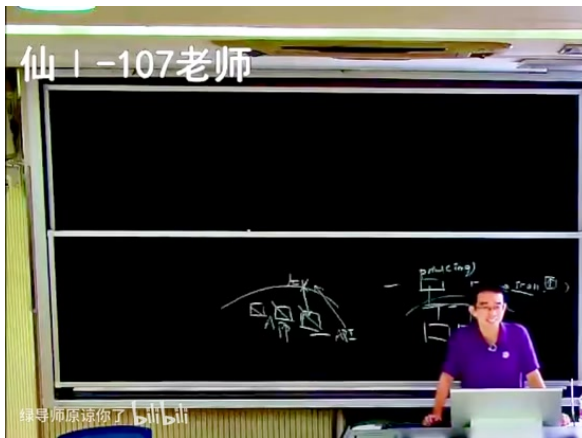
然后我当时干了另一件事儿，我用cloud o p 4.6，直接把那个android app的APK。丢进去，然后让他解包，让他逆向那个蓝牙的协议。就因为它是还是一个标准蓝牙设备，然后让他知道它是根据代码怎么写到这个，怎么把把这个屏写进去。然后我也得到了一个一样的东西，你们可以借助一下现在手头的模型来做这样事儿。

今天的内容

但其实曾经的软件工程不是这样的 😊

- 有必要了解一下“构造软件”的历史中，大家发现、发明了什么
 - 软件工程的诞生
 - 软件工程的智慧
 - UML

02/8



画面 00:30:00

今天的内容

但其实曾经的软件工程不是这样的 😊

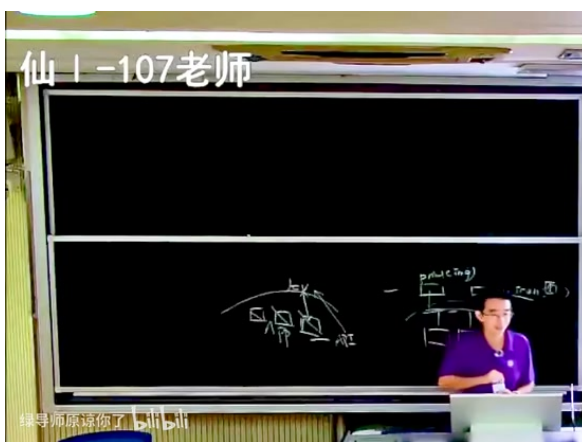
- 有必要了解一下“构造软件”的历史中，大家发现、发明了什么
 - 软件工程的诞生
 - 软件工程的智慧
 - UML

00:30:21

虽然今天我们已经生在这个时代了，但是软件工程的智慧其实是没有还是没有消失的。所以从今天开始我会讲一讲这些你看到的世界上你看到这些软件它到底是怎么来的。我们在软件构造的历史当中，大家发明了什么。然后用这些理念可以更好的指导你今天和agent的协作，成为一个更好的agent native的软件开发者。

00:30:47

对所以今天我来讲软件工程。首先软件工程是怎么诞生的？在软件还是一个新事物的年代，比如说我们回到1960年，虽然1960有点太早了，196年的时候硬件比马农要贵得多，对吧？它一个计算机就要占一个很大的房间，虽然不至于一层楼楼那么高，但是一个计算机确实会非常大。而且作为最先进的设备，它的制造成本也是很高的。尤其尤其现在这个厂家都懂得怎么样偷工减料，对吧？



画面 00:31:00

软件还是新生事物的年代

回到 1960s: 昂贵的码农

- 码农很贵：越早，编程工具越不成熟，码农越贵
 - 高智力需求的工种，成本极高
 - 当然那时候硬件更贵 😊
- 码农还经常缺少对客户的同理心、商业的理解等“软技能”，也不懂业务

把客户的需求翻译成软件系统

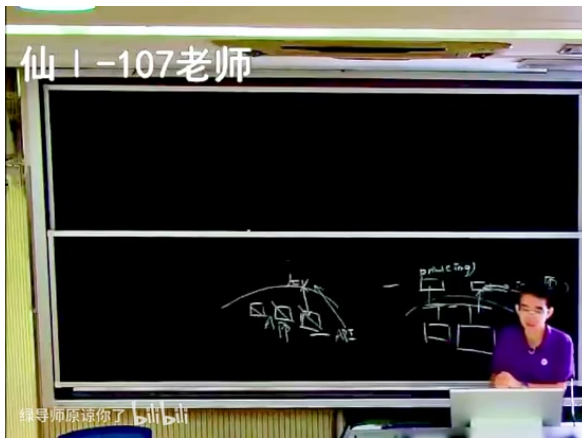
- 软件企业的任务是把客户的需求翻译成软件系统
 - 客户不理解计算机能做什么
 - Intent-spec-impl gap 天然存在
- ⚡ 这还是个年轻的职业: Dijkstra 回忆, 1957 年结婚登记时, “程序员”不被承认为职业, 最后登记成了“理论物理学家”。[本人回忆](#)

00:31:18

最新的消息是这个iphone 18 pro采用了QLC的闪存颗粒。好像也没有关系，因为iphone它的主张是你你应该把所有东西都存在I cloud上，对吧？所以你的数据坏了，他他他也就这样，他他不会不对你的数据负责。可能在保修期内那个闪存坏了，他就再给你换一个主板对吧？但是你是应该把重要的数据放到icd上。

00:31:44

1960年的时候，大家还不知道怎么造计算机的时候，是不敢偷工减料的。就像你刚开始造建筑的时候都是要猛堆料的，所以成本非常高。马龙当然也是很贵的，因为越早这个编程的工具越不成熟。马农是一个非常智力需求的工种。就更像一直到Silicon Valley那个电视剧里面都都仍然是仍然是这样的。



画面 00:32:00

00:32:10

我们看到今天看到了一些，可能你们现在没有了一些以前比如说以前写汇编写机器代码的时候大家的要求。比如说要画流程图，你们现在肯定不画流程图了。因为那个时候你写的是机器代码，你如果不画一流程图，你直接写机器代码，就跟今天大模型直接写汇编一样。你看着大模型可以做的就直接二进制序列写会变样，但你做不到对吧？你需要你需要的预训练才能才能做到这一点。同时不仅码农是一个很贵的东西，而且各位在985大学认为是经历了层层筛选的高智商人群。那么你们多多少少你数学训练多了以后，都会缺少对客户的同理心，对商业的理解等这样的软技能。

软件还是新生事物的年代

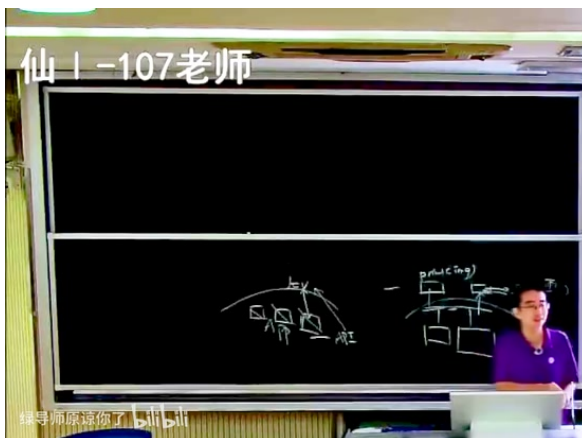
回到 1960s：昂贵的码农

- 码农很贵：越早，编程工具越不成熟，码农越贵
 - 高智力需求的工种，成本极高
 - 当然那时候硬件更贵 😊
- 码农还经常缺少对客户的同理心、商业的理解等“软技能”，也不懂业务

把客户的需求翻译成软件系统

- 软件企业的任务是把**客户**的需求翻译成软件系统
 - 客户不理解计算机能做什么
 - Intent-spec-impl gap 天然存在
- 📖 这还是个年轻的职业：Dijkstra 回忆，1957 年结婚登记时，“程序员”不被承认为职业，最后登记成了“理论物理学家”。[本人回忆](#)

5/7



画面 00:33:00

00:33:02

多多少少会有一点，不是说说大家就没有。其实我见过很多在各方面都很强的六边形战士。但我的观察是在理工科专业里面更多这种nerdy的人会多一些。他们可能不care业务，他只想要造世界上最好的操作系统，对吧？Lin那样的那样那样的人，他他他就不是一个很好的客户经理。但是软件企业依然需要承担把implementation spe和intent之间的gap给补上的这样的一个职责。

00:33:35

这就是公司，他作为一个公司，他他需要签订合同，说我希望1000万美元给你开发一个操作系统，对吧？这个天文数字在今天看对吧？说吧，我给你vivo 50，我也可以给你开发一个操作系统。

软件还是新生事物的年代

回到 1960s：昂贵的码农

- 码农很贵：越早，编程工具越不成熟，码农越贵
 - 高智力需求的工种，成本极高
 - 当然那时候硬件更贵 😊
- 码农还经常缺少对客户的同理心、商业的理解等“软技能”，也不懂业务

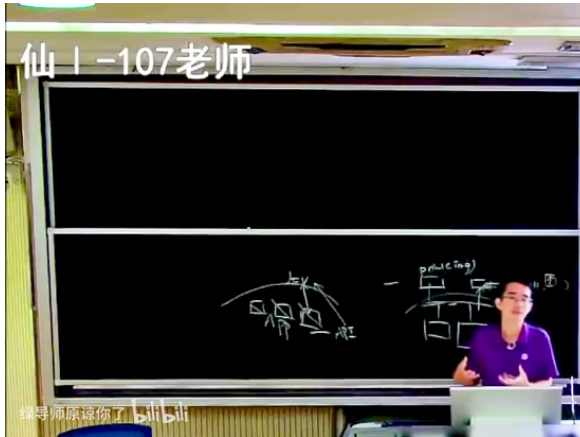
把客户的需求翻译成软件系统

- 软件企业的任务是把**客户**的需求翻译成软件系统
 - 客户不理解计算机能做什么
 - Intent-spec-impl gap 天然存在
- 📖 这还是个年轻的职业：Dijkstra 回忆，1957 年结婚登记时，“程序员”不被承认为职业，最后登记成了“理论物理学家”。[本人回忆](#)

5/7

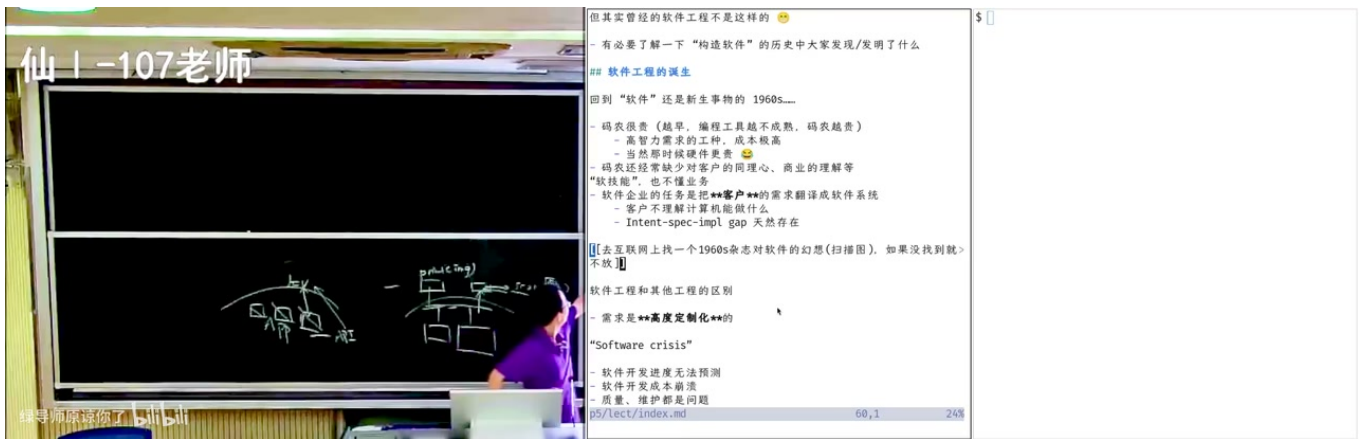
00:33:50

今天不是星期四，客户就是软件企业面对的客户。那个时候客户也不知道计算机能做什么，对吧？比如说打法的官员，当然那个时候比如说nasa可能他们工程师还会有一些概念，他们要把人类送上月球对吧？他面对的都是这样的需求。或者飞机上面要做一个飞控系统，那他们的客户可能也不完全知道他们想要什么，然后程序员还是一个非常年轻的职业。这个d1957年在登记结婚的时候要登记职业。然后这个程序员不被承认是一个职业，最后还登记成了这个理论物理学家。就在那个时候，客户已经对计算机或者说软件产生了非常过度的遐想。



00:34:41

然后这个是一个来自1968年杂志，我特意当然这你可以看一下在这儿。所以AI还是就是我我我的我的slice是AI Copilot生成的。我这里有一个两个中括号，然后这个中括号就代表了这里面的话是我交给AI的。然后AI看到了，他就会把它就就上面是我写的，就是上面这些是我自己写的，你看它基本上是verteam copy的，可能会修复一下我的type。然后像这些就是他自己发挥我的那个make size的skill，自己发挥。然后他就找到了这个boys life tomorrow school。

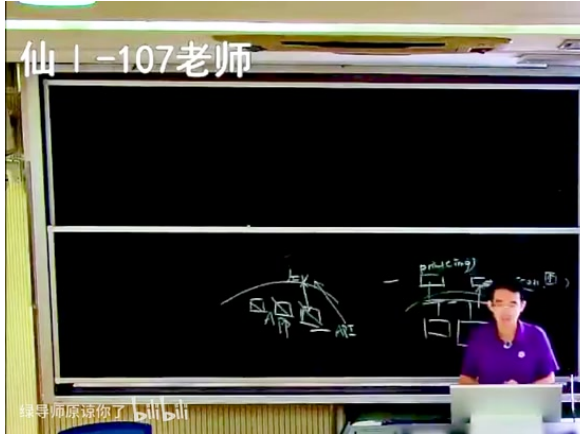


00:35:23

那时候大众其实已经对计算机开始了各种各样的想象，吧？人是怎么样这个终端？及时判分，针对薄弱环节辅导，让学生按自己的节奏学习。到今天都没有实现。

00:35:37

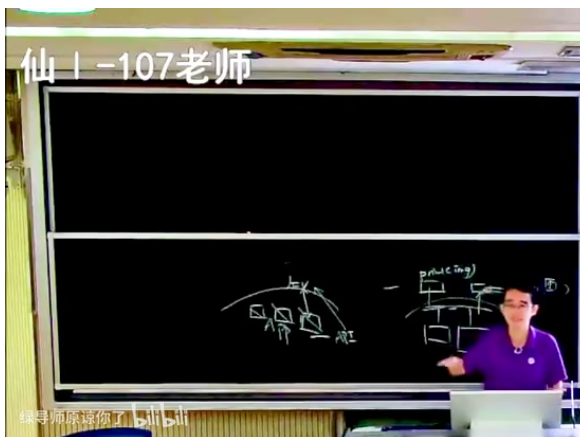
你们还在这里上两节课，每节课50分钟的这个课程。1968年的时候，你看boys life 9也号就已经畅想了这件事情了。所以这个gap就越来越大了，对吧？在那个时候的可能一个客户，他他想想软件公司给他画了一个大饼。然后这个客户说我要投1000万美元做一个什么跨时代的软件系统，但是他脑子里是这样的东西。然后你回到1968年的时候，程序员脑子里是什么？脑子里这是汇编对吧？Go to对吧？这个gap就毫无疑问是非常大的。



画面 00:36:00

00:36:18

然后就是因为有这个gap，我们的软件是要实现物理世界需求在信息世界里面的投影，使得软件工程和其他的比如说像桥梁工程？这个炼钢这样的工程有有很大的区别。他们的需求就是因为自然物理世界我们只能适应，我们不能改造。就它的结构？这个材料的强度有多少，那在他出厂的时候就已经确定了。然后这个楼要盖起来，它要盖几层，要容纳多少人人有多大，这都是由这个自然界选择下来的结果已经决定了的事情。那我们只能按照自然规律的桥，就应该做成这样一带一代一代迭代。所以他们是遵循比较强的模式，然后做一些小小改进，不断的往上创新，网上创新，一个一个彻底推导的一个大的改进和创新，对他们来说是非常困难的。



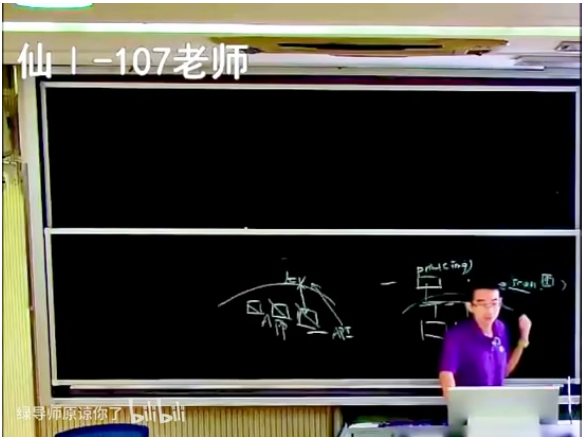
画面 00:37:00

00:37:12

是一直都是比较incremental的。但软件不一样，软件的需求完全是拍脑袋想出来，对吧？你看1968年的时候，大家可以想出这个东西来。需求因为是高度定制化的，所以把它投影到计算机世界，也是一个一直以来都认为是高度需要人类智慧的活动，对吧？我们一直说软件是人类智慧的巅峰，也确实是一个事实。

00:37:37

你看像这个操作系统对吧？编译器这些非常难的软件，你去学它都需要确实需要多年的专业专业积累。他很复杂，然后复杂以至于甚至你的制定就是想隐tent的人都没有想好这个各种各样的控制case应该怎么样，比如说他只想做一个选课系统，那选课系统到底应该是先得先到先得，还是抽签，还是按专业保留，这都是AI给我YY出来的。不满足先修课要求的时候，老师能不能特批，对吧？这种都很容易。



画面 00:38:00

软件工程和其他工程的区别

需求是高度定制化的

- 也因此一直认为是高度需要人类智慧的 (编程，谁学谁知道)
- 以“选课系统”为例，同一个功能名称，可能对应完全不同的规则
 - 名额先到先得、抽签，还是按专业保留？
 - 不满足先修课要求时，老师能否特批？谁负责批准？
 - 退课空出的名额交给候补队首，还是重新开放抢课？

程序也在执行客户的制度与例外

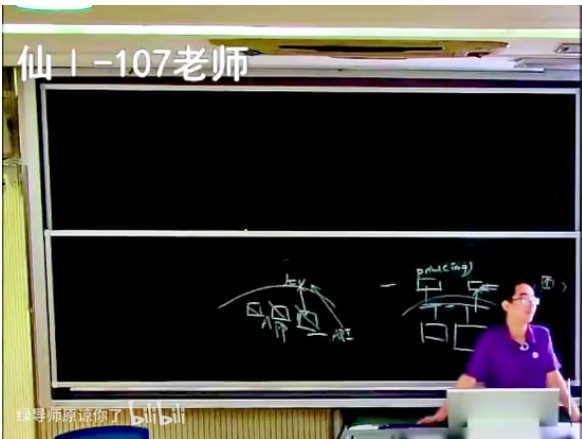
- 这些选择决定了谁能选上课。我们的理解：业务规则与责任边界，本身就是要构造的软件的一部分。

00:38:18

在需求就你在做需求的时候，领导说跟软件公司说，我们的这个课程是要有先修课程的对吧？那也是知道的，你们在系统里面都能看到，比如说你要选修操作系统，你必须要先修计算机系统。基础好，那就糟糕了。这是一个硬的限制还是一个软的限制？谁能override？最后你到最后一秒钟毕不了业的时候，所有的口子学校都能给你开，对吧？你最后就差这门课的时候，你马上就能选上这门课。这就是一个例外性的需求，对吧？

00:38:51

所以软件的需求是一个高度定制化，非常复杂的链接了连接了物理社会人的偏好等等的东西。所以做软件很难做软件在那个时候就大家就发现了进度、成本、质量是一起失控的。开发进度不能预测，软件开发成本崩溃，而且要延期延期交付。



画面 00:39:00

Software Crisis

进度、成本、质量一起失控

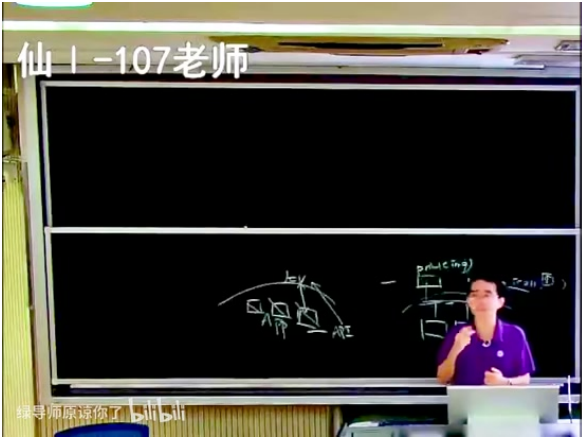
- 软件开发进度无法预测
- 软件开发成本崩溃
- 质量、维护都是问题

E. W. Dijkstra 的“暴论”

- 暴论：Go To Statement Considered Harmful (1968)
 - 挑战性的开场：程序员的水平随程序中 goto 密度上升而下降；背后的关切是人的推理能力有限，程序结构应帮助人理解执行过程。
- The Humble Programmer (1972 ACM Turing Lecture)
 - 硬件越强，社会越敢提出更大的目标；新的野心把软件开发推向能力边界。

00:39:10

然后那个时候其实scientist已经观察到了各种各样软件里面有趣的东西。比如说这个d的ton尼奖d呢也是非常有意思，叫go to statement considered har啊。我们今天不需要这个在让让让他让他读一下。Anyway这个大家都听说过go to是，但你们同时也看到这个linux内核里面有好多go to对吧？这是C的一个为数不多的一个比较edimatic的用法，就是你你最后要go to release，然后在release的时候你会逐个检查你在函数里面申请的所有的资源，然后一一释放。这样它就可以保证你就你在任何时候，你在中间要申请一个资源的时候，你都在release的地方加一个释放检查和释放。然后你在写代码的时候，你永远不能在中途退出，在中途想要退出时候必须go to release，那这个程序就能基本上就能正确了，对吧？



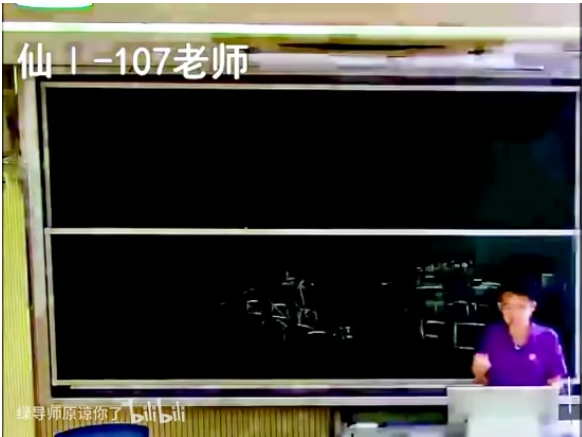
画面 00:40:00

00:40:23

他有一个非常就跟我说说的没有token要退群一样，他先发表一个暴论，他说程序员的水平随着go to的密度上升而下降。它其实背后是有一定的道理的对吧？他他不绝对对吧？

00:40:39

然后可台还各种告诉你应该怎么样用go to，这就对吧这就go to statement considered，这个大家在学编程的时候都学过对吧？因为大量的go to会让你的程序变得很难理解，所以他讨论的其实不是go to到底har不Hargo to可以不harful。如果作为一个正确的机制使用的话，只允许你这样用go to它就不har它挺好的。它解决了C语言里面一个很大的你随时申请从不释放的问题。你们在做OJ的时候留下的坏习惯，那他背后考虑的是这样的一个一个希望你能够写出人能读的代码。那只有你人能读，它才有可能能够维护的下去。从而ok让这个世界还能够正常的软件事界能够相对来说正常一点的去运转。Ok然后到1972年的时候也。



画面 00:40:39

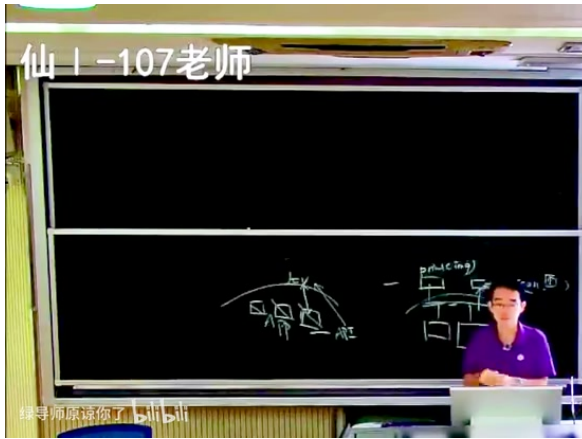
Software Crisis

进度、成本、质量一起失控

- 软件开发进度无法预测
- 软件开发成本崩溃
- 质量、维护都是问题

E. W. Dijkstra 的“暴论”

- 暴论: Go To Statement Considered Harmful (1968)
 - 挑战性的开场: 程序员的水平随程序中 goto 密度上升而下降; 背后的关切是人的推理能力有限, 程序结构应帮助人理解执行过程。
- The Humble Programmer (1972 ACM Turing Lecture)
 - 硬件越强, 社会越敢提出更大的目标; 新的野心把软件开发推向能力边界。



画面 00:41:00

已读取该 PDF (是一份扫描件, 我用 OCR 提取了全文)。这是计算机科学史上最著名的短文之一。

Dijkstra {Go To Statement Considered Harmful} (CACM 11(3), 1968) 摘要

背景与前言

- 作者多年的观察: 程序员水平与所写程序中 go to 语句的密度呈反比。
- 主张: go to 应从一切“高级语言”中废除 (也许纯机器码除外)。

核心论证: 静态文本 vs. 动态过程

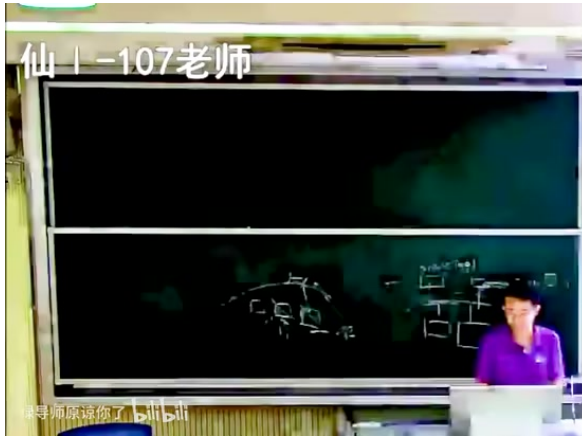
- 程序员的真正工作对象不是程序文本, 而是程序控制下随着时间展开的执行过程; 最终要满足规格的是这个动态过程。
- 人的智力擅长把握静态关系, 却弱于想象随时间演化的过程。因此应尽量缩小“静态程序”与“动态过程”之间的概念鸿沟, 让二者对应关系尽可能平凡。

如何刻画过程的推进 (坐标系)

- 绝赋值语句序列: 一个指向程序文本中的位置即可 — 称为文本索引 (textual index)。
- 加入条件、选择、case、条件表达式后, 单个文本索引仍够用。
- 加入**过程 (procedure) **后, 还需知道是“哪一次调用”, 于是用一串文本索引, 长度等于调用深度。
- 加入重复子句 (循环) 后, 文本索引不再充分, 需为每次进入循环配一个动态索引 (循环计数)。
- 结论: 过程推进总可由文本索引与动态索引的混合序列唯一刻画。

为什么需要这些独立坐标

- 这些索引的值不受程序员控制, 由程序书写方式或运行时动态产生 — 因而独立坐标。



画面 00:41:46

时, n 恰好等于实际人数减一。

go to 的危害

- 滥用 go to 使得“找到一个有意义的坐标系”变得极其困难。
- 也许还能用某种“归一化时钟” (自程序启动以来的动作计数) 唯一刻画推进, 但这种坐标毫无用处 — 想描述“ $n = \text{人数} - 1$ ”的那些点”会变得异常复杂。
- go to 太原始, 是在“邀请人把程序弄乱”。各种结构化子句正是对它加以**约束 (bridle) **的手段: 未来新增子句 (如 abortion 子句) 也应满足“能维持一个对程序员独立且有助益的坐标系”这一要求。

致谢与相关文献

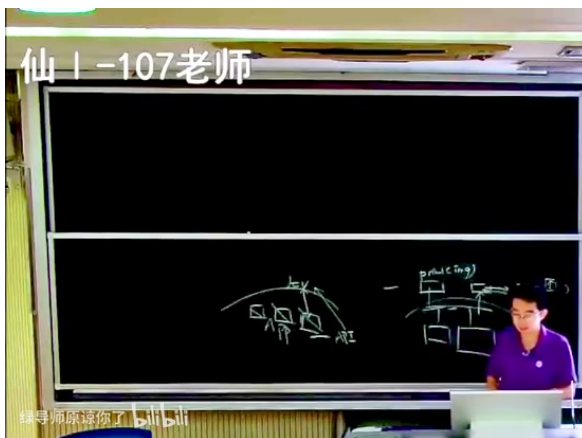
- 受到 Peter Landin, Christopher Strachey 影响: Heinz Zemanek 在 1959 年哥本哈根 pre-ALGOL 会议上已质疑 go to 是否应与赋值语句平起平坐。
- Wirth & Hoare 在推动 case 结构时也表达了类似观点
- Jacopini (Böhm & Jacopini) 证明了 go to 在逻辑上多余, 但机械性地把任意流程图转成无跳转版本并不值得推荐, 结果未必更清晰。

Recap: 抓住灾难的真正主线 — 它不是简单“讨厌 goto”, 而是“用文本索引/动态索引构成独立坐标系”来论证: 值必须相对于过程推进点才有意义, 而 goto 破坏了这种可理解性; 这正是结构化编程的理论出发点。工具上用了 curl 下载 + pdftotext 判断为扫描件 + pdftoppm 转图 + tesseract OCR 提取全文。

read
<https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf> and summary

00:41:54

有他的这个Turing lecture, 就是他得图灵奖以后的这个演讲, 叫的humble programmer。然后我觉得这就现在很好, 我们可以直接去消这个digest原始的文稿, 对吧? 我们可以在AI I的帮助下, 直接去阅读原始的文稿。



画面 00:42:00

Thinking...

```
$ cd /tmp 66 curl -sL -A "Mozilla/5.0" -o ewd340.html
"https://www.cs.utexas.edu/~EWD/transcriptions/EWD03xx/EWD340.html" 66 ls -la ewd340.html 66 file ewd340.html 66 echo
"..." 66 python3 -c '<<'EOF'
import re,html
s=open('/tmp/ewd340.html','encoding='utf-8','errors='replace')
.read()
print("chars:",len(s))
print(s[:800])
EOF (timeout 60s)

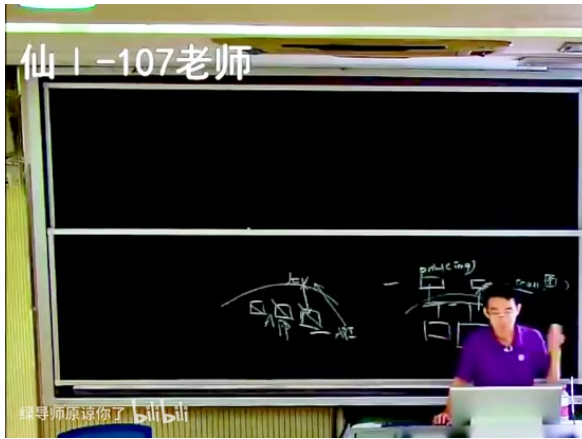
... (30 earlier lines, ctrl+o to expand)
<div align="left">
  <table width="100%" border="0" cellspacing="0"
  cellpadding="0">
    <tr>
      <td><b>ACM Turing Lecture 1

74.9K 42.0K R19K CH96.6% $0.004 0.7%/1.0M (auto) deepseek-fla


```

00:42:21

你看它就谈到了为什么会有软件危机。如果你永远写这个online价值的小程序, 就不会有软件危机的。你就算一个东西, 算完就完了。你一旦把这个程序调对, 你就永远不需要去维护它, 对吧? 但是机器变强以后, 你的软件的需求是不断的膨胀的, 然后你要承担越来越多。就你机器越强, 你承担这个现实世界的intent越多。然后intent越多, 这个不确定性就越多, 然后实现就会变得越复杂, 然后越来越复杂以后, 应该怎么样?



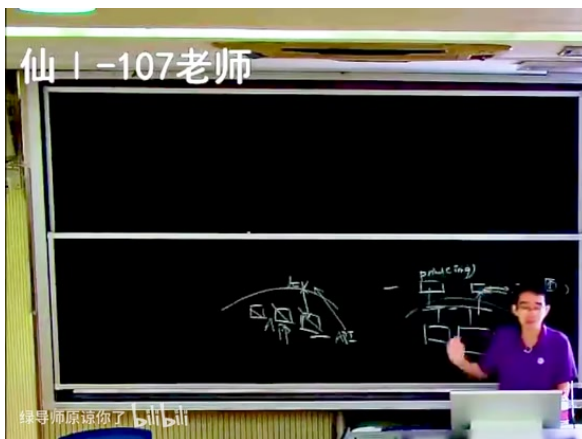
画面 00:43:00

00:43:01

然后他提了一些他对软件的思考，nicely factor的solutions。然后就这样子的想法在你构造任何软件系统的时候都在不断的发现。就比如说你看我就写一个lab mate帮我驱动我的标签打印机和扫描仪，它也是一个 nicely fac的solution？一个良好的分解。我们无时无刻都在用这样的这个想法来来组织我们的软件？然后他还有一些有意思的这个观点？

00:43:37

保守力量主要来自教育机构与舍不得重写旧程序去的用户，这就是放到今天依然是你你觉得能够觉得醍醐灌顶的这个确实大那个时候1972年值一个指一个图形甲，对，然后在那个时候，其实就大概就在那个时期，软件工程就会成为了大家关注的一个焦点。因为软件开始失败了。所以在1968年的时候，有一个natto软件工程的会议，你们也可以去直接去读这个原sum我，我就不读了，我就开一个开这个让他在在后台后台跑起来。然后也差不多在那个时候，这个Hamilton和他的团队提出了软件工程这样的概念。这个帖子我觉得大家也可以读一下，这是一个非常有标志性的照片在。



画面 00:44:00

“软件工程”成为共同议题

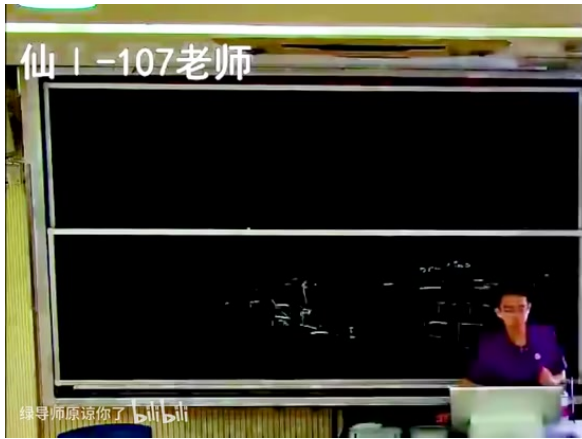
1968年：一场刻意的“挑衅”

- 1968年10月，50多位专家在德国Garmisch参加NATO软件工程会议。[会议报告, Background](#)
- “Software Engineering”被刻意选作带挑衅意味的名称：软件生产也应有成熟工程学科的理论基础与实践纪律。报告前言

连“危机”也有争议

- 会上有人反对一概而论：排序、工资处理已有成熟办法；困难集中在规模和要求超出已有经验的系统。[会议讨论, §7.1.2](#)
- 由此理解“危机”：过去成功的经验，未必能支撑下一种规模与野心。

9/7



画面 00:44:27

When Margaret Hamilton was a child, her father used to take her on long drives through Michigan's Upper Peninsula. They would talk the whole time about "pioneers"—rugged things, the software engineers, now 83, recall. "Like what he and why and why not."

About two decades later, Hamilton held her breath in the MIT Instrumentation Lab's mission monitoring room as one of the greatest "what ifs" in history came to fruition. On July 20, 1969, at 3:37 a.m. Boston time, the lunar module Eagle touched down on the moon. By then, Hamilton was the head of software engineering for the Apollo program. As she likes to say, the Eagle's crew were the first humans to work on the lunar surface—and her team's software was the first to join them.

In more than a decade working with the Apollo missions, Hamilton helped safely shepherd spacecraft into and out of orbit. She led a group of nearly 100 colleagues, bringing rigor—and rescue—to what was then the brand-new field of software engineering. In fact, she came up with the term "software engineer" and pushed for its use. And she did all of it with creativity and dynamism, but to a computer repeatedly broke out that "the never-gonna-happen happens."

Hamilton left college thinking she would be a mathematician, planning to get a job while her husband went to Harvard Law School and then go to grad school herself. In 1959, she started working for the renowned MIT meteorologist Edward Lorenz, who charged her with programming an 800-panel computer called the LGP-30.

Hamilton had never seen a computer. But she took to it right away, finding unorthodox ways to make the hardware doing work more quickly. "I would often say that would be called 'hacker' programming today," she says. For instance, by the book debugging involved slowly feeding the long paper roll that held an incorrect program back through the machine. Hamilton, who had no patience for this, learned binary code and started editing the errors herself—poking a hole in the paper with a sharp pencil to turn a 0 to a 1, and using a small piece of tape for the reverse. "The work done by Hamilton and others eventually helped lead to the success of the Apollo mission," she says.

POPULAR

A fundamental flaw leaves LLMs critically vulnerable to attack

WB Douglas Heaven

AI's recursive self-improvement might not come as quickly after all

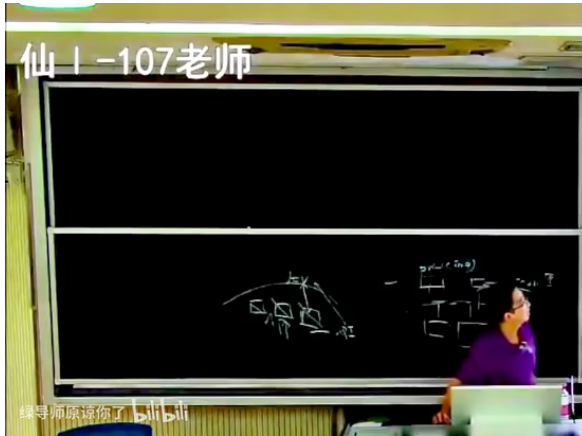
Michelle Kim

AI is more likely than humans to form biases when being

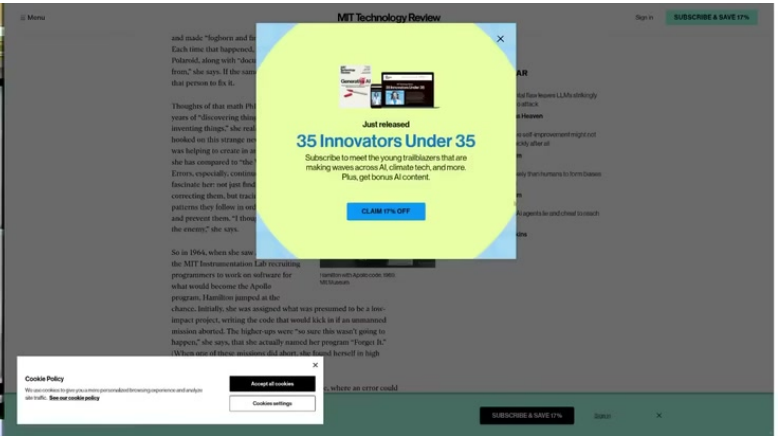
Michelle Kim

Here's why AI agents lie and cheat to reach their goals

Grace Horkins

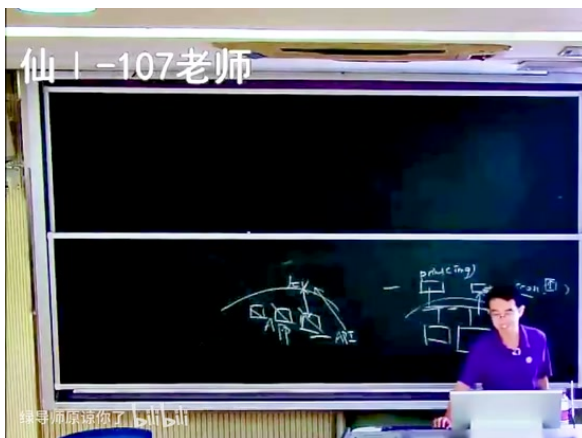


画面 00:44:33



00:44:38

应该是载入史册的照片，with a polo code 1969这个polo的demo，sorry. 然后我们也可以让他来读一下这个可能要读一下这个代表了软件工程学科的诞生，对吧？



画面 00:44:45

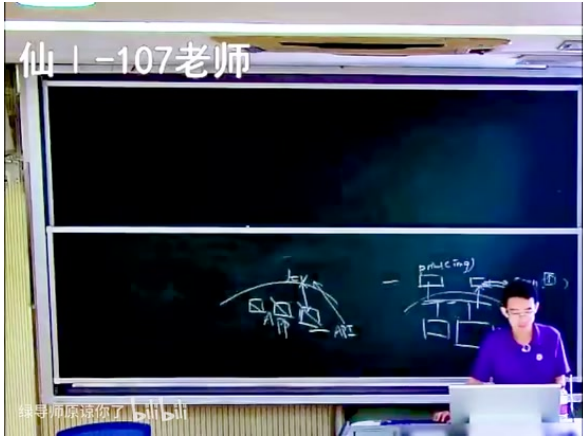


hooked on this strange new field she was helping to create in an atmosphere she has compared to "the Wild West." Errors, especially, continued to fascinate her: not just finding and correcting them, but tracing the patterns they follow in order to predict and prevent them. "I thought of them as the enemy," she says.

So in 1964, when she saw an ad from the MIT Instrumentation Lab recruiting programmers to work on software for what would become the Apollo program,



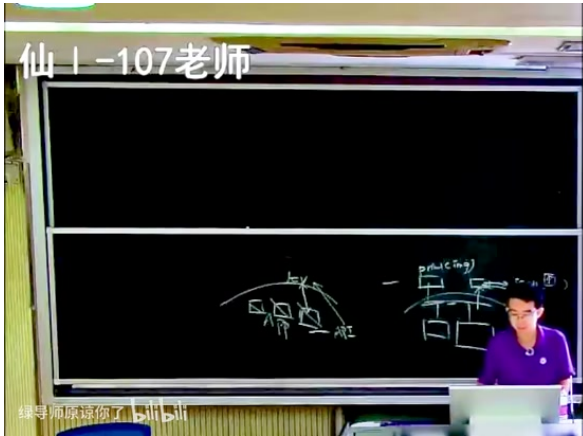
仙！-107老师



但其实曾经的软件工程不是这样的 😊
- 有必要了解一下“构造软件”的历史中大家发现/发明了什么
软件工程的诞生
回到“软件”还是新生事物的 1960s...
- 码农很贵 (越早, 编程工具越不成熟, 码农越贵)
- 高智力需求的工种, 成本极高
- 当然那时候硬件更贵 🤖
- 码农还经常缺少对客户同理心、商业的理解等
“软技能”, 也不懂业务
- 软件企业的任务是**把客户的需求翻译成软件系统**
- 客户不理解计算机能做什么
- Intent-spec-impl gap 天然存在
[[去互联网上找一个1960s杂志对软件的幻想(扫描图), 如果没找到就不放]]
软件工程和其他工程的区别
- 需求是**高度定制化**的
“Software crisis”
- 软件开发进度无法预测
- 软件开发成本崩溃
- 质量、维护都是问题
js/lect/index.md 60,35-26 24%

画面 00:44:49

仙！-107老师



```
$
```

```
Working
```

```
~
```

```
14k 44.7k R78k CH98.7% $0.010 1.9%/1.0M (auto) deepseek-flan
```

```
To resume this session: pi --session 01a8be70-1969-76bc-b677-2
```

```
27f4fbdc17
```

```
$ pi
```

```
pi v0.86.0
```

```
escape interrupt · ctrl+c/ctrl+d clear/exit · / commands · !
```

```
bash · ctrl+o more
```

```
Press ctrl+o to show full startup help and loaded resources.
```

```
Pi can explain its own features and look up its docs. Ask it
```

```
how to use or extend Pi.
```

```
[Context]
```

```
·pi/agent/AGENTS.md
```

```
[Extensions]
```

```
@narunitw/pi-chrome-devtools:dist
```

```
~
```

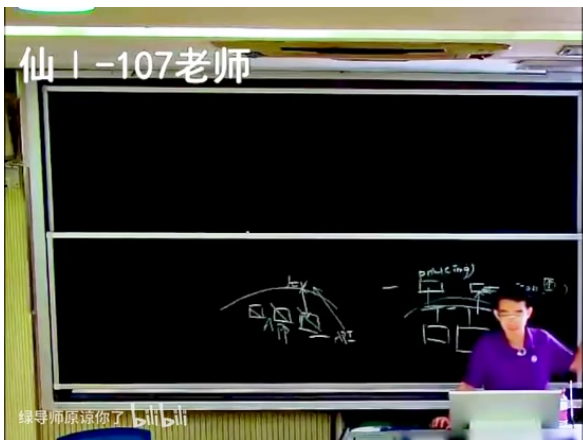
```
0.0%/1.0M (auto) deepseek-flash · high
```

画面 00:45:00

00:45:11

2019年8月, 经过不懈的排查错误, 这位最初的软件工程师把宇航员送上了月球。然后他有一个他的传记, 然后这个软件工程你们看到今天你们都要防御性编程。你们在第一节, 比如说学SICP的时候, 他就告诉你写软件的时候, 你要写它的solutions tions, 写specifications。他把一个绝对不能发生的假设都当成可能一个种防御性的为未来思考的这样的一个智慧, 对吧? 他假设这个软件在任何时候他都要甚至违背违反假设的时候, 这个软件依然要能够能够工作他才设计了一个成功的软件。那我们也可以让他。当然我不确定他不会做web search, 就deep sick flash有没有足够的支持。如果是一个足够大的模型的话, 他可以去做, 他可能去做web search, 他可能在这里就有对, the never going to happen can happen, 对吧?

仙！-107老师



```
$
```

```
误差源的协议 (出错者与相应文档一起拍立得存档), 形成追溯、
```

```
预测和预防错误的思路。她说: “我把错误当作敌人。”
```

```
- 阿波罗计划: 1964 年加入 MIT 仪器实验室。她先写无人任务中止
```

```
时的代码。因认为不会触发而给程序取名 “Forget It”, 结果真派
```

```
上战场。随后晋升到“载人级”软件, 必须能预测并补偿硬件故障或
```

```
宇航员误操作。她提出了“软件工程 (software engineering)”一
```

```
词并力推使用, 领导近 100 名开发者。
```

```
- “Lauren bug”: 她常带 4 岁女儿 Lauren 到实验室, Lauren 在模
```

```
拟器里乱按。触发了任务中途的发射前序列 P01, 导致崩溃。NASA
```

```
起初不以为意, 只允许记录在案。阿波罗 8 号 (首次载人绕月)
```

```
真有宇航员误操作导致导航数据丢失。正是这个隐患——她笑称
```

```
“It’s the Lauren bug!”
```

```
- 阿波罗 11 号: 登月舱即将着陆时, 出现五条算力不足的告警。由
```

```
于软件设计可靠, 休斯顿信任它优先处理最关键的着陆任务。任务
```

```
得以继续。看看软件在降落中自愈, 她说那是“纯粹的恐惧”。
```

```
- 晚年荣誉: 2016 年获奥巴马总统颁发总统自由勋章; 2017 年她站在自
```

```
己团队代码堆旁的照片被做成乐高小人。她创办 Hamilton
```

```
Technologies, 开发 Universal Systems Language, 力求从根本上
```

```
预防常见错误, 而非事后补救。
```

```
意义: 文章借此说明, 把一个“绝不会发生”的假设当成可能, 正是
```

```
可靠软件与航天安全的关键。
```

```
Recap: 本文串起 Hamilton 从“错误即敌人”的理念到阿波罗软件
```

```
工程实践的故事, 重点在于 “the never-going-to-happen can
```

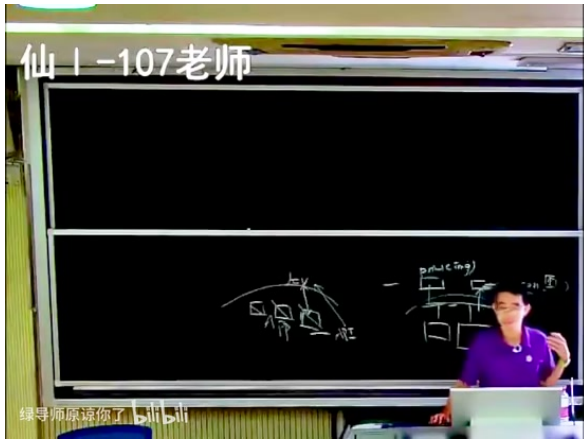
```
happen”这一预防性思维, 以及她首创的“软件工程”概念如何把可
```

```
靠性变成一门学科。
```

画面 00:46:00

00:46:32

我觉得这就是你看软件工程。当当我们回头去看这个原作的时候，你发现这些思想在今天反而更容易理解。比这个教科书上面经过了好多层的一手、二手、三手、四手消化的时候要这个要好的多。然后他比如说你可能要假设这个硬件也是不可靠的对吧？然后我无论如何都要在这个软件里面做一个兜底的机制。当你的程序crash了以后，整个系统依然要能够工作。其实这就是软件工程的起点。所以你看这里有一个能工作，还包括发生意外的时候，这有一个？



仙 I-107老师

画面 00:47:00

一道保护，NASA 觉得太不可能，只允许她“记一笔”。

阿波罗 8 号（首次载人绕月）上，宇航员真的误操作输入了 P01，导致数据当场消失，只能把她叫来恢复。她后来说话时还在笑：“这是 Lauren bug”。

4. 阿波罗 11 号：1201/1202 告警

最惊险的一幕发生在登月舱 Eagle 即将着陆时：告警 1201 和 1202 连续闪出五条，意思是计算机过载、任务被挤掉——一部分原因是交会雷达的开关位置不对，喂进了多余数据。

执行别的系统，这叫“死机”；但这套软件设计时就让执行程序（Executive）能优先级调度：过载时自动砍掉低优先级任务，重启并让最关键的着陆任务继续跑。休斯顿的年轻工程师 Jack Garman 对最关键的着陆任务继续跑。休斯顿的年轻工程师 Jack Garman 对最关键的着陆任务继续跑。休斯顿的年轻工程师 Jack Garman 对最关键的着陆任务继续跑。

Hamilton 说，看着软件在降落过程中当场自愈，那种感觉是“纯粹的恐惧（sheer terror）”。

5. 遗产：从“修错”到“软件工程”

- 她首创并力推 “software engineering”（软件工程）这个词，把当时被当作“手工活”的工作变成一门严肃学科。
- 核心方法：不只修 bug，而是**追踪错误模式、预测并预防**：让代码在硬件故障、人为失误、意外时序下仍能恢复、恢复。
- 她后来的公司 Hamilton Technologies 开发 **Universal Systems Language**，主张“从设计上让常见错误无法发生”，而不是事后补救。
- 2016 年获总统自由勋章；2017 年那张她站在自己团队代码打印稿旁的照片被做成乐高小人。

Recap：用“绝不可能发生”的假设去设计软件，是这段故事的核心方法——从 Forget It, Lauren bug 到 1201/1202 告警，每一次“意外”都验证了错误恢复与优先级调度的价值，也让“软件工程”从

00:47:17

登月appoo 11登月下降的时候发生警报。但是这个软件因为有这个优先级和和recovery，让关键的下降控制任务得以继续。那个时候还没有软件工程的时候，他们就从1st principle去思考，如果如果我们这个时候要构造一个真正好的高质量的软件，我们应该怎么做，它的1st principle就是这个never going to happen，can happen对吧。有意思，软件软软件的工程，然后这就这然后在这个意义下，这个软件工程就诞生了。



仙 I-107老师

画面 00:47:39

连续闪出五条，意思是计算机过载、任务被挤掉——一部分原因是交会雷达的开关位置不对，喂进了多余数据。

执行别的系统，这叫“死机”；但这套软件设计时就让执行程序（Executive）能优先级调度：过载时自动砍掉低优先级任务，重启并让最关键的着陆任务继续跑。休斯顿的年轻工程师 Jack Garman 对最关键的着陆任务继续跑。休斯顿的年轻工程师 Jack Garman 对最关键的着陆任务继续跑。

Hamilton 说，看着软件在降落过程中当场自愈，那种感觉是“纯粹的恐惧（sheer terror）”。

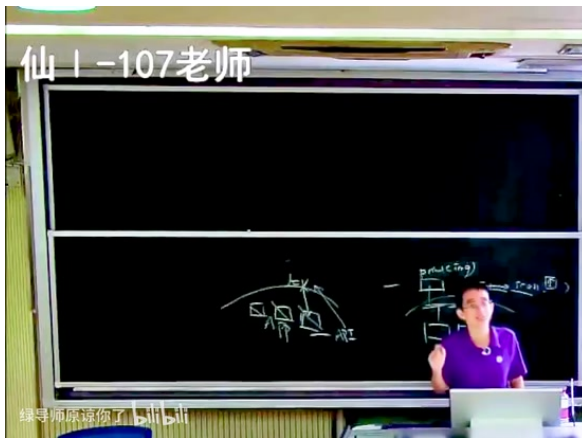
5. 遗产：从“修错”到“软件工程”

- 她首创并力推 “software engineering”（软件工程）这个词，把当时被当作“手工活”的工作变成一门严肃学科。
- 核心方法：不只修 bug，而是**追踪错误模式、预测并预防**：让代码在硬件故障、人为失误、意外时序下仍能恢复、恢复。
- 她后来的公司 Hamilton Technologies 开发 **Universal Systems Language**，主张“从设计上让常见错误无法发生”，而不是事后补救。
- 2016 年获总统自由勋章；2017 年那张她站在自己团队代码打印稿旁的照片被做成乐高小人。

Recap：用“绝不可能发生”的假设去设计软件，是这段故事的核心方法——从 Forget It, Lauren bug 到 1201/1202 告警，每一次“意外”都验证了错误恢复与优先级调度的价值，也让“软件工程”从

00:47:52

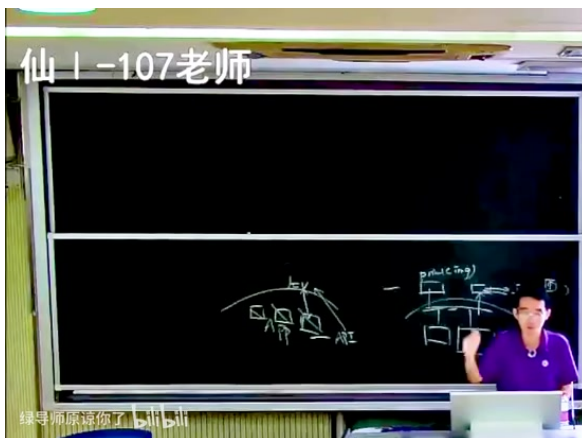
就在差不多那个时间点，因为在成本时间各种各样团队的约束下交付软件在那个时候已经成为了一个很大的挑战。那怎么样做这件事？就是软件工程对吧？在成本时间约束下，把人的意图，你想要设计，你要做的产品或者是客户的要求，变成一个可以工作的，可以维护的、可以生存的？你随着时间的推演的需求可能会变化。这个never going to happen。有点这个感觉吗？你们做O店的时候，你假设题目是对的，假设题目是能做出来的，这个本身假设就是不对的。就当你反复强化的时候，你overfit这个假设，你应该假设的是你面对这个开放的世界，这个题目可能是错的，这个题目的条件随时可能会变化，这个题目甚至可能会变成另外一个题目。



画面 00:48:00

00:48:49

那你怎么做一个软件来解决它，对吧？当然编程是一件事，你通过训练这个online价值，训练的是你的逻辑思维、算法思维，你的cha能力。当然对于设软件理解，比如说理解这些东西是有好处的对吧？所以你看这个H boy他说是把工程a systematic discipline , quantifiable approach to the development, population and maintenance of software, that is application of engineering工程化。他把它把软件变成一个你你不再是一个个人英雄主义，对吧？那个时候Hamilton的那个时代，他需要一个好的leader，就是换一个人，这个项目就失败了，对吧？变成可以拿到一个checklist对着checklist。



画面 00:49:00

00:49:31

如果你1 2 3 4 5 6 7 8 9 10全部都满足了，那你至少这个软件可能不能保证你的软件成功。但是失败的概率会小很多，有纪律可量化覆盖整个软件的整个生命周期。所以在AI I还没有大杀四方的时代，软件工程这个学科从刚开始的大量的失败，或者说是从数亿美金的教训得到的？什么一个火箭这种很多时候都是可能是跟软件工程有关系的。即其实积累了很多的智慧，大家看到成功，看到失败，多多少少就会从中总结出一些经验和教训。

软件工程的诞生

在成本、时间等约束下交付软件

- 在成本、时间等约束下，把人的意图变成可工作的软件系统。
 - 从大到团队组织管理，小到代码风格都会覆盖的学科

IEEE: 把工程应用于软件

“ IEEE: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1).

- 系统化、有纪律、可量化：覆盖开发、运行与维护。

软件工程的诞生

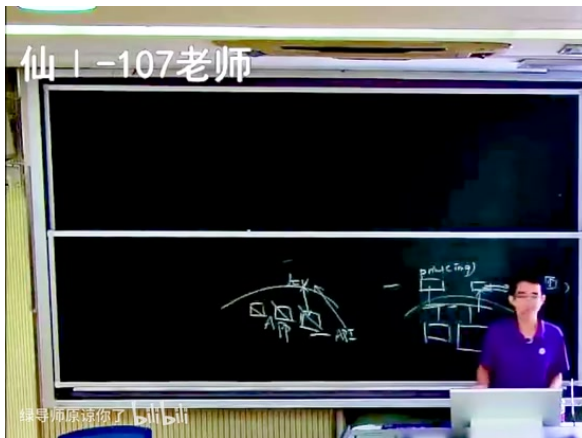
在成本、时间等约束下交付软件

- 在成本、时间等约束下，把人的意图变成可工作的软件系统。
 - 从大到团队组织管理，小到代码风格都会覆盖的学科

IEEE: 把工程应用于软件

“ IEEE: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1).

- 系统化、有纪律、可量化：覆盖开发、运行与维护。



画面 00:50:00

00:50:12

然后当然如果我们回到这个pre-LLM时代，大家其实也非常清楚这个intent speimplementation这样的gap对吧？如果你接到一个开发任务，你肯定不能立即按照去安排说大家去干，对吧？任何一个leader都不应该干这样的事儿。即便最终的产物一定是马农写出来的，因为不同的人他看到的是不一样的部分，对吧？甚至如果这个软件要分开的话，你的leader都需要指派哪一个部分，有哪一个人来做。

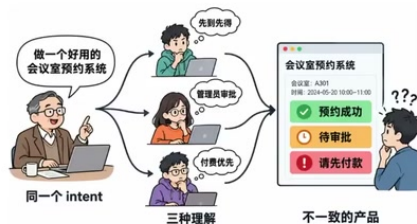
00:50:44

就像我做这两个的时候，打印机会想到你，那个时候我已经同时接到我的电脑上了。那那我就是起了两个agent对吧？开两个窗口，一个去逆向打印机，一个去逆向扫描仪。这很正常，需要一个划分任务的这样的过程，不能是一个self organized的如果我们涉及到划分任务？也就是说刚开始我们从一个空的ra的时候，空的report的时候，那我们怎么样去做呢？是直接去开发，就直接把这个一句话交给他做MVP。那个时候还没有这个minimum，对吧？

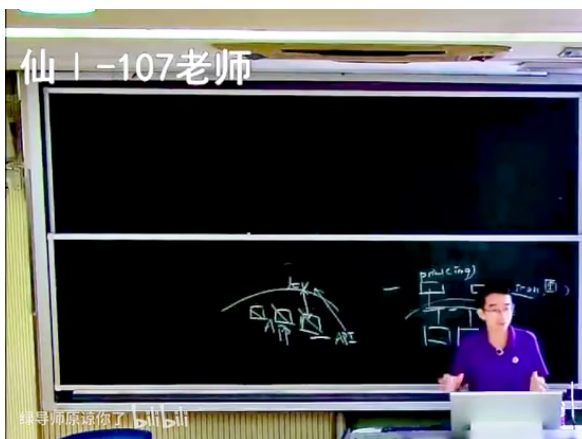
软件工程的智慧

回到 Pre-LLM 时代

- 接到开发任务，肯定不能立即安排码农干活——虽然最终产物是他们生产的
- 如何减少 **intent-spec-impl gap** 带来的影响？



5/8



画面 00:51:00

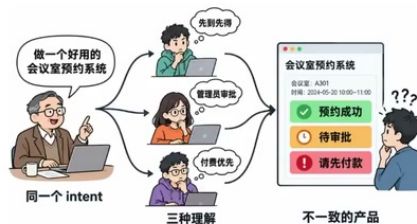
00:51:21

你想1970年，这是1970年的论文。1970年的时候软件工程刚刚诞生，还没有minimum variable product，没有大语言模型对吧？那一个思考怎么按时不超预算的交付大型软件。这有一个软件工程领域的开山之作，这个winroce的madevelopment of large software systems。然后今天你们有些同学可能学过这个软件工程这门课，然后你们可能学过，我可以开个浏览器，应该多多少少一定是我们搜索百度。

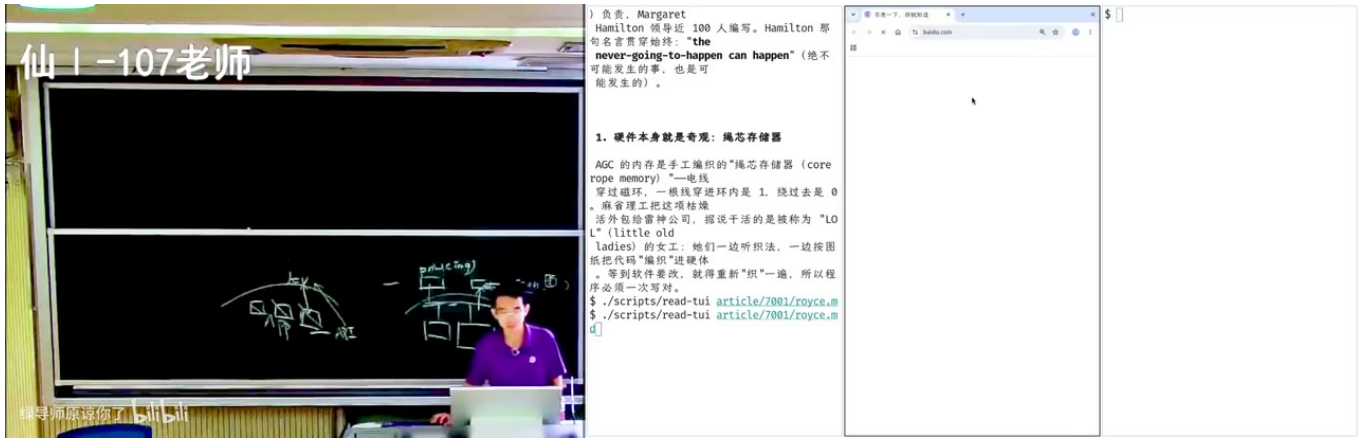
软件工程的智慧

回到 Pre-LLM 时代

- 接到开发任务，肯定不能立即安排码农干活——虽然最终产物是他们生产的
- 如何减少 **intent-spec-impl gap** 带来的影响？



5/8



画面 00:52:00

00:52:05

好，搜索一个瀑布模型。我觉得大家如果学过任何学过一点软件工程，我就不打开苗懂百科了，这个有点搞笑。放大一些对吧？都有一个这样的，你们见过很多时候见过这样的图对吧？大概长这样图，一个waterfall model，从上到下，你们只要是computer science或者software相关的，你们一定在软件工程课上面见过这样的图。



画面 00:52:15



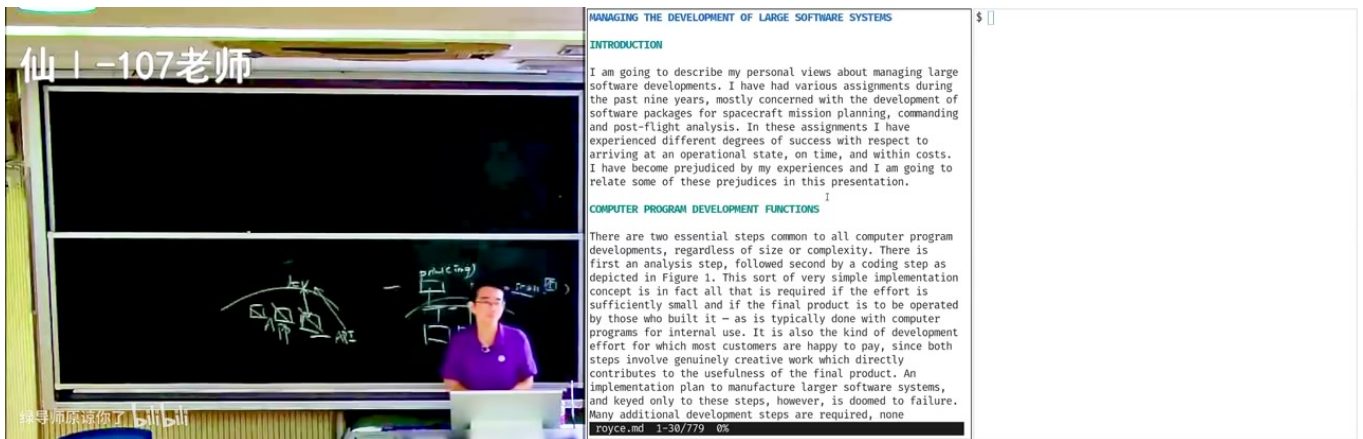
画面 00:52:21



画面 00:52:27

00:52:44

然后这个图来自哪里呢？这个图就来自winroce的这一篇经典的文章。当然你看我读文章的方式和你们读文章的方式有些不一样。这个是我昨天在备课的时候，我需要这篇文章，我就直接找到了这篇文章的链接，然后直接和我的agent说入库这个paper，然后我就得到了这个，这是一个文字整理版的。我记得我给大家可能展示过一次对吧？Read space，我可能没有在这个课好像没有在这个课上展示过。比如说这个article 70对吧？他这样的我暗夜切割，就我的AI I梗同学，然后会把所有的图片mark down的图片都直接嵌入进来。



画面 00:53:00

00:53:33

然后我有一些工具，比如说我有一个命令行的工具，可以去。我有一个命令行的工具，可以去这个有点宽，可以去打印文本，也可以去提取图片等等。也有一个类似于less的工具，这都是web code的。就我什么也没有说，来帮我对吧？帮我去解去去直接在命令行里面读读论文。

00:54:00

然后瀑布模型就来自这里，瀑布模型然后实际上实际上有意思最有意思的是我们都讲，比如说软件工程学软件工程师都学瀑布模型，实际上这篇文章是批判瀑布模型的。所以你还得就这种还得去去原文就当个东西好多次经过了消化以后，你得到的东西可能就不是那个东西了。因为其实在那个时代，1970年大家真的是只什么也没有，只能从1st principle去思考？这个呃use space。

仙！-107老师

[Image #1: Figure 2 - waterfall-style flowchart of implementation steps]

Figure 2. Implementation steps to develop a large computer program for delivery to a customer.

I believe in this concept, but the implementation described above is risky and invites failure. The problem is illustrated in Figure 4. The testing phase which occurs at the end of the development cycle is the first event for which timing, storage, input/output transfers, etc., are experienced as distinguished from analyzed. These phenomena are not precisely analyzable. They are not the solutions to the standard partial differential equations of mathematical physics for instance. Yet if these phenomena fail to satisfy the various external constraints, then invariably a major redesign is required. A simple octal patch or redo of some isolated code will not fix these kinds of difficulties. The

royce.md 89-118/779 11%

画面 00:54:00

00:54:50

我直接让他帮我读这个，对，我有一个read skill试一下。

仙！-107老师

of this is that as the design proceeds the change process is scoped down to manageable limits. At any point in the design process after the requirements analysis is completed there exists a firm and closeup, moving baseline to which to return in the event of unforeseen design difficulties. What we have is an effective fallback position that tends to maximize the extent of early work that is salvageable and preserved.

[Image #1: Figure 2 - waterfall-style flowchart of implementation steps]

Figure 2. Implementation steps to develop a large computer program for delivery to a customer.

I believe in this concept, but the implementation described above is risky and invites failure. The problem is

royce.md 79-108/779 16%

画面 00:55:00

```
Thinking...

$ ls -la .kimi-code/skills 2>/dev/null; find . -maxdepth 3 -name "SKILL.md" -o -maxdepth 3 -type d -name "skills*" | head -20

... (2 earlier lines, ctrl+o to expand)
drwx----- 4 jyy jyy 4096 Sep 19 16:47 ..
drwx----- 2 jyy jyy 4096 Sep 11 19:33 kimi-ocr
drwx----- 2 jyy jyy 4096 Sep 11 19:47 reader
drwx----- 2 jyy jyy 4096 Sep 12 00:19 rewrite
./kimi-code/skills

Took 0.0s

$ find / -maxdepth 6 -path /proc -prune -o -type d -name "skills" -print 2>/dev/null | head -20

Elapsed 1.0s

-- Working

~/Projects/readspace
71.9k 41.1k R29k CH95.6% $0.002 0.5%/1.0M (auto) deepseek-fla
```

00:55:06

需要一点点的时间，没事，我们我们继续读。如果你看到这儿，那这就是一个经典的瀑布模型。你看。需求到系统的需求，软件的需求，分析设计代码测试，然后去上线上线运维。我有一个这个，但这个我还没还可以继续继续调。你看他说后辈称为瀑布模型的原始出处，也是被误读最严重的一篇。所以应该是一个最早可能有一两个article，它错误的就没有描述所有的这个文章里面的观点，然后我给了一个他我还可以重塑在现代意义上重重数，说一个程序小到能被践踏的人，自己用两步就够了。所以你看这就是你们的online line价。

仙！-107老师

coding, and all drive up the development costs. Customer personnel typically would rather not pay for them, and development personnel would rather not implement them. The prime function of management is to sell these concepts to both groups and then enforce compliance on the part of development personnel.

Figure 1. Implementation steps to deliver a small computer program for internal operations.

[Image #1: Figure 1 flowchart - ANALYSIS box with arrow pointing down to CODING box]

Figure 1. Implementation steps to deliver a small computer program for internal operations.

Figure summary: Simple two-step flow diagram. A rectangular box labeled "ANALYSIS" (upper left) has a thick curved arrow leading down and to the right into a second rectangular box labeled "CODING" (lower right), showing that a small computer program for internal operations is produced by a single analysis step followed by a single coding step.

royce.md 32-61/779 4%

画面 00:56:00

问题的根源很朴素：一个程序小到能被建它的人自己用，两步就够了——分析、编码（Fig.1）。但“大系统”不行。于是人们自然会加阶段，得到 Fig.2 那条漂亮的单向阶梯。Royce 承认这条线是对的，然后立刻指出它 **risky, invites failure**。

为什么危险？因为测试是链条上第一个“物理”事件：时序、存储、I/O 传输在这里才从“被分析”变成“被经历”。这些现象不像数学物理里可解的像微分方程——你分析不准，一旦不满足外部约束，不是反复设计，错在晚期就是 100% 超支。他反复强调：分析与编码其实“好登”（轨道力学做完可能就行并行算术），真正吃掉风险预算的是测试。

关键跳跃：既然风险最终在测试处爆炸，而爆炸的代价由回溯距离决定，那么所有补救的本质就是同一件事——把“发现问题”的时间点提前，把“回溯距离”压缩。五条建议都是从这一条推出来的，而非凑数的最佳实践：

1. **设计先行：**让 program designer 在 requirements 之后，analysis 之前就分配存储/时序/数据流。批评者说这是“真空里设计会错”——Royce 承认，但“misses the point”：错在早期只是重做设计，错在晚期就是 100% 超支。他随手给了一个组织学上的巧思：让设计师在分析阶段“抢”资源，逼分析员亲身感受约束的代价，而不是事后被告知。
2. **文档：**早期阶段 documentation = specification = design，三位一体，所以“文档没写出来就等于设计还不存在”。他给了个很工程化的换算：5M 美元硬件 = 30 页规格；5M 美元软件 = 1500 页。理由是金是“下游可替代性”：有文档，测试时不必找原作者的脑子，而谁可以交给运维人员；文档可以不做到完美。
3. **做两遍：**不确定性的本质是 human judgment，而人对工期/成本

00:56:01

如果你站在今天的就是你站在今天你做的所有的软件的practice，你依然很能够很好的对上这个文章里面当时他们的一些思考，对吧？这个ok没问题，但是这个很容易失败。如果如果你把这样的一个流程扩展到这样的一个瀑布模型，是因为测他的观点，因为测试是整个链条上第一个物理事件。就你在设计的时候你有报告，然后你在这个program真正被写出来之前，它都是不能运行的这是个1st principle对吧？也就是说当这种不能运行的东西在的时候，你都会随便做做，这就是人对吧？文档反正文档你我就你叫我写文档，我就写一个。你们想想你们写这个实验文档的时候，你们对它的正确性有guarantee，对吧？

仙！-107老师

scoped down to manageable limits. At any point in the design process after the requirements analysis is completed there exists a firm and closeup, moving baseline to which to return in the event of unforeseen design difficulties. What we have is an effective fallback position that tends to maximize the extent of early work that is salvageable and preserved.

Morning Paper 风格的重新指导

问题的原点很朴素：一个程序小到能被建它的人自己用，两步就够了——分析、编码（Fig.1）。但“大系统”不行，于是人们自然会加阶段，得到 Fig.2 那条漂亮的单向阶梯。Royce 承认这条线是对的，然后立刻指出它 risky, invites failure.

为什么危险？因为测试是链条上第一个“物理”事件：时序、存储、I/O 传输在这里才从“被分析”变成“被经历”。这些现象不像数学物理里可解的偏微分方程——你分析不准，一旦不满足外部约束，不是改一行八进制补丁能救的，而是要动需求层次的重大重设计，于是回到原点，100% 超期/超支（Fig.4）。注意他反复强调：分析与编码其实“好客”（轨道力学做完可能就几行串行算术），真正吃掉风险预算的是测试。

关键跳跃：既然风险最终在测试处爆炸，而爆炸的代价由回溯距离决定，那么所有补救的本质就是同一件事——把“发现问题”的时间点提前，把“回溯距离”压短。五条建议都是从这一条推出来的，而非凑数的最佳实践：

1. 设计先行：让 program designer 在 requirements 之后，analysis 之前就分配存储/时序/数据流。批评者说这是“真空里设计会错”——Royce 承认，但“misses the point”：错在早期只是重做设计，错在晚期就是 100% 超支。他还顺手给了一个组织学上的巧思：让设计师在分析阶段“抢”资源，逼分析员亲身感受约束的代价，而不是事后被告知。
2. 文档：早期阶段 documentation = specification = design，三位一体，所以“文档没写出来就等于设计还不存在”。他给了个很工程化的换算：5M 美元硬件 = 30 页规格；5M 美元软件 = 1500

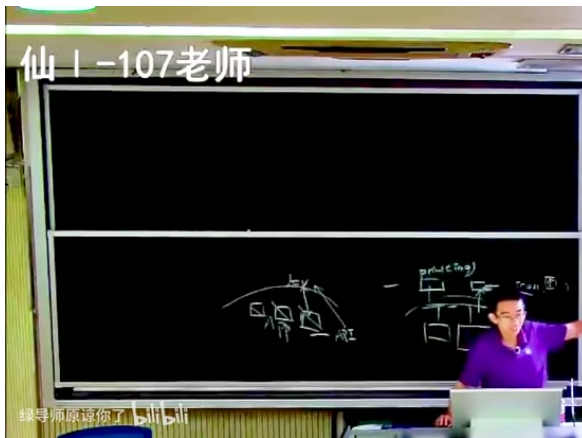
画面 00:57:00

00:57:01

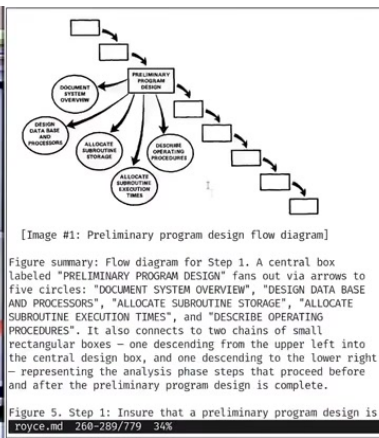
当然有同学会认真一点，或者说我有个best effort的guarantee，但实际上你是没有guarantee的。你文档或者先写就写了，然后写就放在那儿。然后可能你代码改了，设计改了，文档不一致，也就放在那儿。所以这些东西是有一个很强的约束在很强约束下做的。然后只有在你测试的时候，这个程序才真正跑起来，你才真正知道它的行为是什么。虽然你可能在这个编码的时候就已经感觉到可能有一些有有一些不对了。但是一旦你的比如说你的从intent到specification到intation发生了漂移，你到测试的时候再发现的时候，就意味着你要回头去。这我记得应该也有一个图的，你要回头去回到前面去修改，那这个就会带来成本，所以就是超期或者超支的源头，对吧？

00:57:57

你看到你看这些，我觉得如果你真的读过原文，读过原文他很好理解，而且非常的就你站在今，尤其你今天有今天的软件工程实践以后去理解它，其实非常的舒服。既然风险最终都在测试的地方爆炸，那所有的补救的本质都是把发现问题的时间点提前，就非常自然的一个想法？设计先行文档做两遍do twice，这个是它里面的一个非常经典的观点。对，应该是对这个词你看他用了我这里总结了一下叫preliminary pilot program。他说我们先要做一个不完备，但是能够体现这个系统功能的设计。然后再把这个东西可能会失败，但它迭代的成本会变得很低。迭代成本变得很低，它就可以去reflect到这个系统的设计上。然后你看这是什么，这就是minimum variable product对吧？然后他有各种各样的模型，他后面应该在论文的最后可能是这个对，就是他所想的一个software developer model。



画面 00:58:00



[Image #1: Preliminary program design flow diagram]

Figure summary: Flow diagram for Step 1. A central box labeled "PRELIMINARY PROGRAM DESIGN" fans out via arrows to five circles: "DOCUMENT SYSTEM OVERVIEW", "DESIGN DATA BASE AND PROCESSORS", "ALLOCATE SUBROUTINE STORAGE", "ALLOCATE SUBROUTINE EXECUTION TIMES", and "DESCRIBE OPERATING PROCEDURES". It also connects to two chains of small rectangular boxes - one descending from the upper left into the central design box, and one descending to the lower right - representing the analysis phase steps that proceed before and after the preliminary program design is complete.

Figure 5. Step 1: Insure that a preliminary program design is
royce.md 200-289/779 34%

Morning Paper 风格的重新推导

问题的原点很朴素：一个程序小到能被建它的人自己用，两步就够了——分析、编码 (Fig.1)，但“大系统”不行。于是人们自然会加阶段，得到 Fig.2 那条漂亮的单向阶梯。Royce 承认这条线是对的，然后立刻指出它 **risky, invites failure**。

为什么危险？因为测试是链条上第一个“物理”事件：时序、存储、I/O 传输在这里才从“被分析”变成“被经历”。这些现象不像数学物理里可解的偏微分方程——你分析不准。一旦不满足外部约束，不是改一行八进制补丁能救的，而是要动需求层次的重大 redesign，于是 **回溯原点，100% 超期/超支** (Fig.4)。注意他反复强调：分析与编码其实“好客”（轨道力学做先可能就几行串行算术），真正吃掉风险预算的是测试。

关键跳跃：既然风险最终在测试处爆炸，而爆炸的代价由回溯距离决定，那么所有补救的本质就是同一件事——**把“发现问题”的时间点提前，把“回溯距离”压短**。五条建议都是从这一条推出来的，而非凌效的最佳实践：

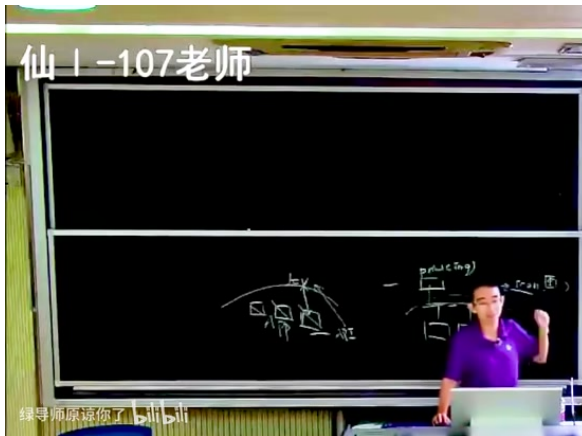
1. **设计先行**：让 program designer 在 requirements 之后，analysis 之前就分配存储/时序/数据流。批评者说这是“真空里设计会错”——Royce 承认，但“misses the point”，错在早期只是重复设计，错在晚期就是 100% 超支。他还顺手给了一个组织学上的巧思：让设计师在分析阶段“抢”资源，逼分析员 **亲身感受** 约束的代价，而不是事后被告知。
2. **文档**：早期阶段 documentation = specification = design，三位一体，所以“文档没写出来就等于设计还不存在”。他给了一个很工程化的换算：5M 美元硬件 + 30 页规格 = 5M 美元软件 + **1500 页**。理由全是“下游可替代性”：有文档，测试时不必找原作者的脑子，运维可以交给运维人员，改版可以不推倒重来。

的是测试。

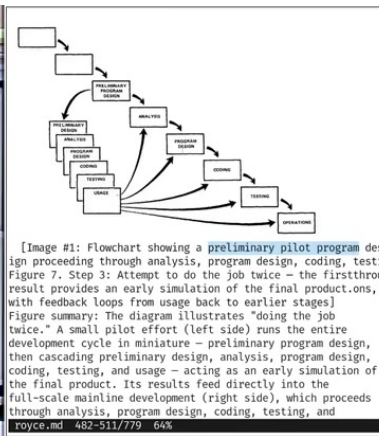
关键跳跃：既然风险最终在测试处爆炸，而爆炸的代价由回溯距离决定，那么所有补救的本质就是同一件事——**把“发现问题”的时间点提前，把“回溯距离”压短**。五条建议都是从这一条推出来的，而非凌效的最佳实践：

3. **做两遍**：不确定性的本质是 human judgment，而人对工期/成本的估计“invariably and seriously optimistic”。所以先做一个缩小版全流程（30 个月的项目重放 10 个月的 pilot），把判断性争议变成可实验的假设。
4. **测试是最大资源消耗者，也是最快、备份最少的一环**，所以视觉走查（让计算机找丢号太贵），每逻辑路径至少数值验证一次，用 **独立** 的 product assurance 团队——他还反过来用这一点做判断：“只有原作者能测”是 **文档不合格的确诊信号**。
5. **客户**：需求定义的“共识”其实高度可歧义，所以要设 PSR/CSR/FSAR 让客户反复书面承诺，而不是交付时第一次见面。

结论：把一条“单向阶梯”变成“带早期反馈的迭代螺旋”，代价是每一条都多花钱——如果简单的办法够用，这笔钱就不该花；但他九年的经



画面 00:59:00



[Image #1: Flowchart showing a preliminary pilot program design proceeding through analysis, program design, coding, testing]

Figure summary: The diagram illustrates "doing the job twice." A small pilot effort (left side) runs the entire development cycle in miniature - preliminary program design, then cascading preliminary design, analysis, program design, coding, testing, and usage - acting as an early simulation of the final product. Its results feed directly into the full-scale mainline development (right side), which proceeds through analysis, program design, coding, testing, and

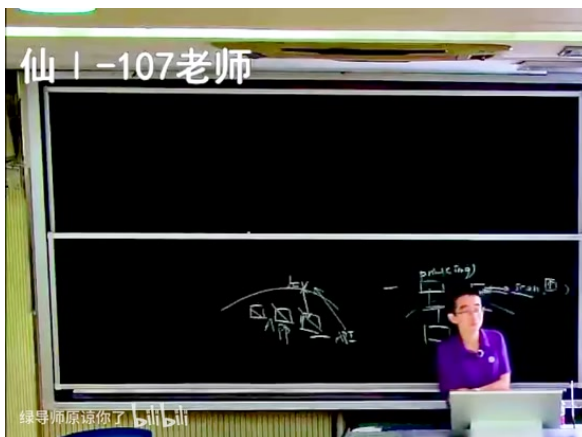
royce.md 482-511/779 64%

00:59:17

然后你就算是在今天这个web coding的时代，你回去它还是提供了很多的软件工程的智慧。即便他在那个时候是在比如说像航天器的任务规划？把人送上月球，在这些背景下写的写的程序对吧？

00:59:35

瀑布模型他先提出了这个瀑布模型，然后说了这个瀑布模型在软件工程中是怎么失败的，他分析了一个具体的失败，然后这个流程在今天开发软件的时候也是完全一样的。因为你总是从空的项目开始，不管你是用什么样的版本控制工具，你都需要精力。先想清楚你要干什么，然后再大概想想我价格应该怎么做，再去写代码对吧？不管写代码的是软件工程师还是AI这个流程依然是成立的，所以他的这个都是成立的。



画面 01:00:00

Royce 与瀑布模型

如何按时、不超预算地交付大型软件？

- Winston Royce: [Managing the development of large software systems](#)

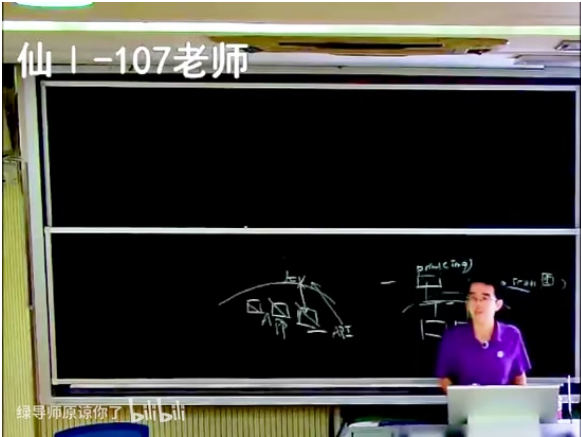
- 这篇论文出了后来被称为“瀑布模型”的经典阶段图
- 1970 年的背景：航天器任务规划、指令与飞行后分析，时间和存储都有硬约束。
- 原文在这张图之后就指出：这样的实施方式风险很高，容易失败。

“瀑布假设”

- 希望迭代只发生在相邻阶段之间
- 实际测试中暴露的问题，却会一路回溯到设计与需求

01:00:10

所以你看到了在现在AI 11的时代，每一个人都可以阅读一手的信息，你可以直接回到这个历史的背景上，而且AI既知道过去又知道现在。你还有历史的原文，然后你会发现有些东西你在书上读不懂的，就你两页纸，那个书看起来不长，两页纸讲瀑布模型，读了两遍还没搞懂，然后你回去看这个原文，马上就看懂了。所以知识的压缩就是过去比如说像我上学的时候或者再早的时候，如果你要查证一个知识网络是几乎不可能的事情，对吧？你要去图书馆翻很多的资料，图书馆的索引机制建的也不好。虽然图书馆确实也有索引机制。如果你要找到那个精确的资料，你要花去无穷多的时间。所以你那个时候你是不可能通过一手资料来学系的，你必须要经过教科书做知识的有损压缩。



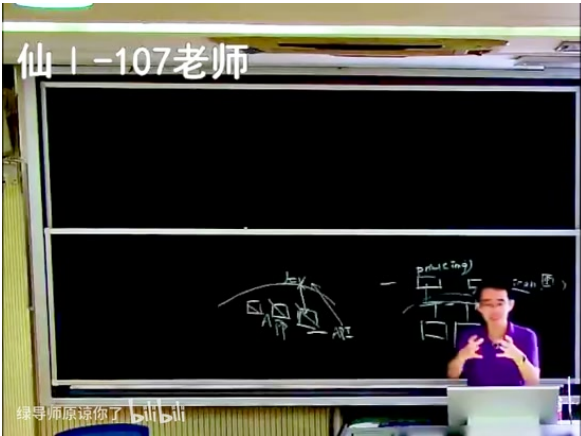
画面 01:01:00

01:01:03

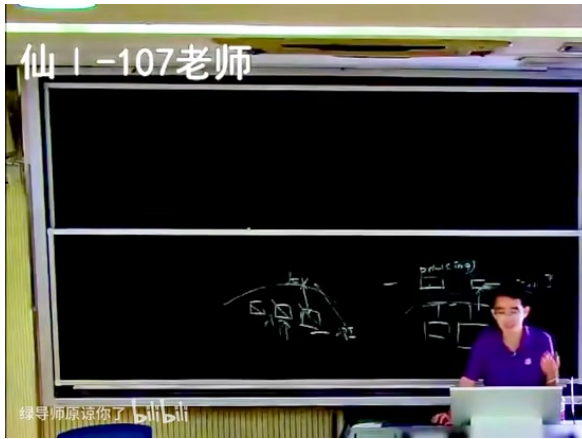
有经验的人会，所以大家就是有的教科书写的好，有的教科书是房自学的。教科书写的好的人他就可以？他读过一手的信息，他也他可能也能共情。我觉得好的好的老师可以共情学生，他知道学生想要什么，学生什么地方会不明白。所以他把这个一手的信息做一个压缩，同时对于阅读者的损失是很小的。但今天这些东西也都不要了，因为你可以直接就说我作为一个老师，我觉得我不需要再做压缩。

01:01:34

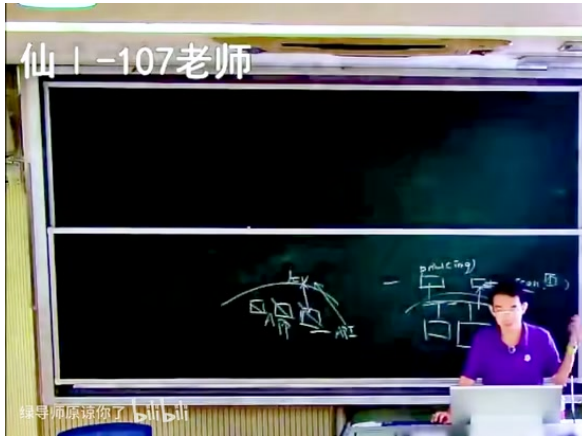
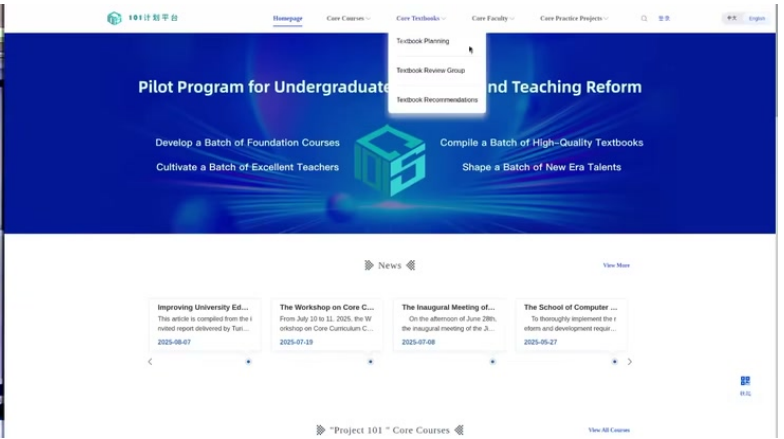
因为我做压缩对每个人的压缩都是优损的，而且对不同的人压缩损失了不同的部分。如果我一对一的教学，我就可以做最合适你的压缩，但不需要你直接对着你的圆。我只要把这个知识的脉络，把知识拆解开来，拆解成很多的这个我好像有一次课的时候讲过，我我怎么理解这个世界的知识，把知识拆成一。个一个的发现点，然后把这个你只需要把这个点和它的原文拿到，然后做一个属于你自己的压缩就可以了，对吧？所以有些东西，比如说我们花了很大的代价去去比如说我我们的我们有很多导师老师也在这个计划里去建设比较先进的课程。然后很然后我觉得比较有趣的一件事儿是这样的对吧？



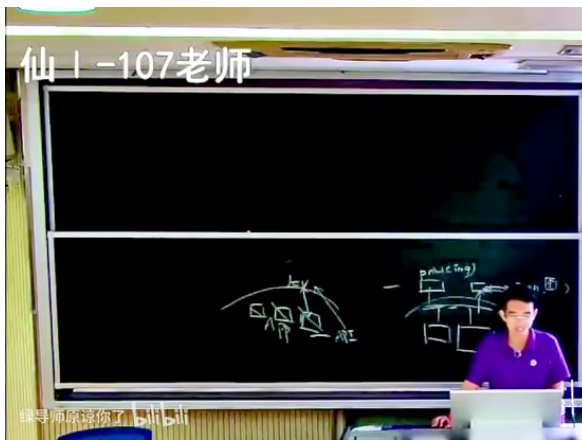
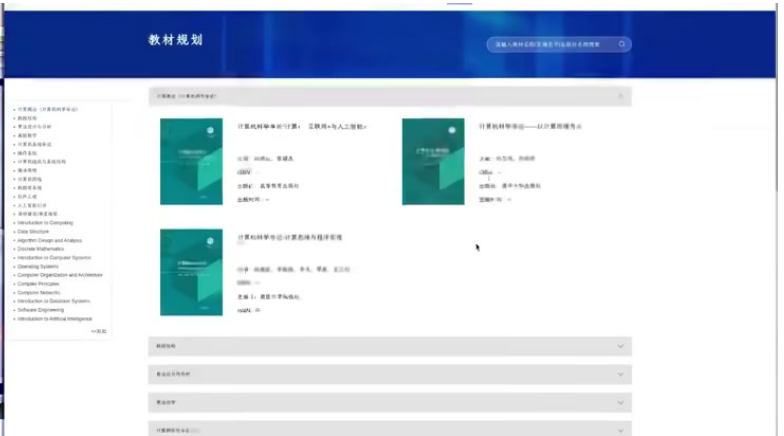
画面 01:02:00



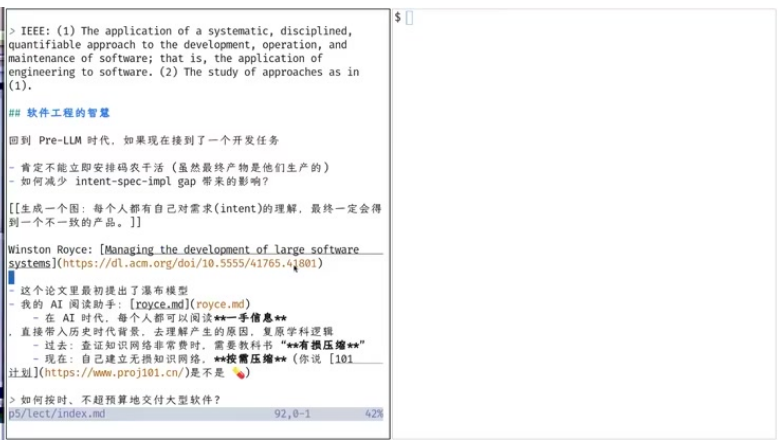
画面 01:02:12



画面 01:02:17



画面 01:02:25



01:02:29

这个其实是一个很好的计划，因为在查证知识网络非常费时的时候，如果有真正的专家能够帮你做好的压缩，那这种毫无疑问是有价值的。但是在AI I的时代，是AI I是摧毁性的对于整个领域？你可能在里面投入了百万、千万甚至上亿的人民币，但最后都是一个模型，就把你轮子就也就碾掉了。一个可能是个deep k这样的？Flash这样的模型刚才在帮我对吧？你有任何的问题我都可以直接问他，比如说你能了解一下，解释一下这个拓布模型在软件工程里到底是为什么，怎么误读了，对吧？

仙！-107老师

画面 01:02:59

emphasizing that the customer participates continuously via requirement updates throughout the development cycle.

Figure 10. Summary

[Image #1: Summary flowchart of the software development process - a large central iterative loop surrounded by review ova Figure summary: This is the closing summary figure of thegh No paper. It presents the author's full recommended software development process as one integrated diagram: a large iterative development loop in the center (preliminary design → analysis → program design → coding → testing → operations, driven by a spiral of "critical software reviews") with feedback arrows looping back from every stage to earlier

royce.md 696-725/779 92%

的是测试。

关键跳跃：既然风险最终在测试处爆炸，而瀑布的代价由回溯距离决定，那么所有补救的本质就是同一件事——把“发现问题”的时间点提前，把“回溯距离”压短。五条建议都是从这一条推出来的，而非满数的最佳实践：

1. 设计先行：让 program designer 在 requirements 之后、analysis 之前就分配存储/时序/数据流。批评者说这是“真空里设计会错”——Royce 承认，但“misses the point”：错在早期只是重做设计，错在晚期就是 100% 超支。他还顺手给了一个组织学上的巧思：让设计师在分析阶段“抢”资源，逼分析员**亲身感受**约束的代价，而不是事后被告知。
2. 文档：早期阶段 documentation = specification = design。三位一体，所以“文档没写出来就等于设计还不存在”。他给了个很工程化的换算：5M 美元硬件 = 30 页规格；5M 美元软件 = 1500 页。理由全是“下游可替代性”：有文档，测试时不必找原作者的脑子，画继可以交给画继人员，改顺可以不继续追问来。
3. 做两遍：不确定性的本质是 human judgment，而人对工期/成本的估计“invariably and seriously optimistic”。所以先做一个缩小版全流程（30 个月的项目里放 10 个月的 pilot），把判断性争议变成可实验的假设。
4. 测试是最大资源消耗者，也是最晚、备份最少的一环：所以视觉走查（让计算机找毛病等太贵），每条逻辑路径至少就值验证一次。用独立的 product assurance 团队——他还反过来用这一点做判据：“只有原作者能测”是文档不合格的**确诊信号**。
5. 客户：需求定义的“共识”其实高度可歧义，所以要设 PSR/CSR/FSAR 让客户反复书面承诺，而不是交付时第一次见面。

结论：把一条“单向阶梯”变成“带早期反馈的迭代螺旋”，代价是每一套新多花钱——如果简单的办法够用，这钱就不该花；但他九年的经

01:03:21

你会产生疑问，然后你就直接跟他对话，然后他可以在这个过程当中注意找到更多的原文吧？找到甚至找到整个误读的链怎么样？这个教材一级，最后到一个你手上拿的这个软件工程泪水教材上面是怎么一步一步的，曲解个他有一H。

01:04:00

这个解释还挺make sense的，虽然可能也许你也许选择不相信AI再去追问，或者用一个更好的模型让它联网更好的。瀑布好画好背好出题可以作为第一个模型讲。而且每一个新方法都说这个？瀑布是史上最好用的反派，他说的很对对吧？就真的是这样，因为本身他就是错的，本身这个1970年的文案已经告诉你不能瀑布，1970就告诉你你要做M1P对吧？而且确实不成属虚构对吧？在这里对，挺好的，有意思。对，这是关于这个文章，当然我已经带着大家读过这个文章了，所以这几页就AI生成的也不那么重要了，对吧？

仙！-107老师

画面 01:04:00

emphasizing that the customer participates continuously via requirement updates throughout the development cycle.

Figure 10. Summary

[Image #1: Summary flowchart of the software development process - a large central iterative loop surrounded by review ova Figure summary: This is the closing summary figure of thegh No paper. It presents the author's full recommended software development process as one integrated diagram: a large iterative development loop in the center (preliminary design → analysis → program design → coding → testing → operations, driven by a spiral of "critical software reviews") with feedback arrows looping back from every stage to earlier

royce.md 696-725/779 92%

3. 未读反过来说是对象，“waterfall”是批评者造的贬义标签，先有贬义，后被回溯挂到 Royce 头上。这和“Big Bang”是 Hoyle 为挖空而造，结果成了宇宙学正式名称完全一样。名字一旦流行，就自带一套“线性、不可回头”的语义，再没人去对原文。
4. 最有杀伤力的一层：制度化和采购。Royce 那 6 份 Document 交付物（需求/初步设计/接口/最终设计/测试计划/操作手册）太好用了——美国国防部的 DOD-STD-2167、MIL-STD-2167A 把它变成了 SRM/PRM/CDM 这类强制制审查卡。他的“文档里程序”被完整继承，他的“迭代螺旋”被完整丢弃，合同和固定价采购天然喜欢“先冻结需求再施工”：管理愿意相信瀑布，Royce 自己就是 TRW 的管理者，他提醒的正是管理者的这种乐观（“invariably and seriously optimistic”）。

三、为什么这个误读活这么久

教学价值：瀑布好画、好背、好出题，可以作为第一个“模型”讲。

完美的稻草人：螺旋、RUP、XP、Scrum、DevOps 每一代新方法都需要一个“旧世界”家反对。瀑布是这个史上最好用的反派——它作为稻草人的功能价值，超过了它作为方法的实用价值。这也是它至今不死的原因：没人真用它，但人人都需要它。

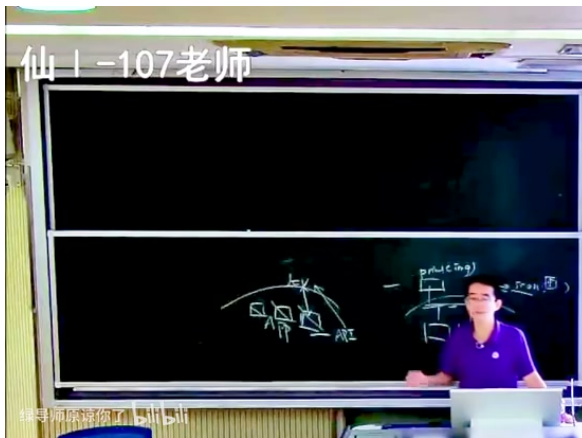
它并不纯属虚构：Fig.3 那句“迭代只限前两步”确实给误读了命法入口，所以不是有人凭空捏造，而是选择性阅读——只引 Fig.3，不引 Fig.4 和 Fig.10。

四、顺势一提

Larman & Basili 的 *Iterative and Incremental Development: A Brief History* (IEEE Computer, 2003) 系统地做了这个考据，并指出迭代式实践早在 1950 年代就存在：Royce 严格说属于迭代派的早期声音之一（“do it twice”就是原型/试点），而不是迭代的敌人

01:04:48

里面文章告诉我们叫do twice一个很重要的工程理解。但今天就是做做MVP。然其实你也可以在不同的各种各样的现在的软件开发的背景下去去了解。比如说如果我们用TDD叫test driven development，叫测试驱动开发，去理解这个工作，如何解释如何用这个工作来解释TDD？它都是互相的。我们今天的概念，就今天正确的practice可以映射到过去，然后过去正确的这个1st principle也可以用来映射到今天。



画面 01:05:00

01:05:30

我们来看看一个700B的小模型是什么样的。说rois他讲了一件事儿，他说缺陷被发现的时间？越晚他带来的返工代价就越大。显然这个是成立的，就跟你你们做OJ都会出现这个事儿。你们一开始的时候脑子一热想一个算法，这个算法不对的，然后就去写代码了。写代码写完以后发现样例不对，然后你对着样例跑了，你的那个算法你发，完了我这个算法整个就错了，这就是这就是一个典型的瀑布模型的失败。

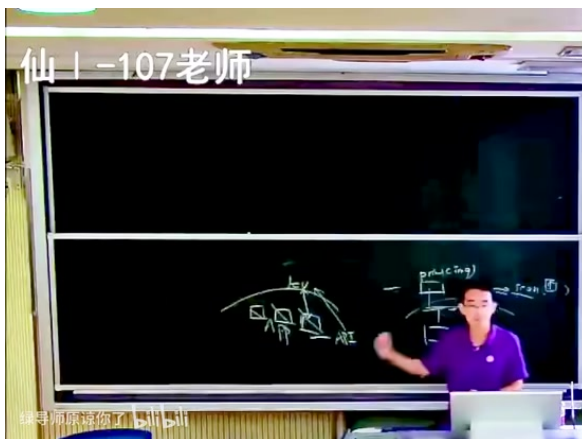
Do It Twice

用今天的 MVP 理解 Royce 的 Pilot Model

- Minimum Viable Product: 先做一遍，从中学习，再完善产品
- Royce 的 **do it twice**: 先用 pilot model 经历关键问题
- Pilot 重点验证技术与运行风险；MVP 更关注用最小投入验证用户与业务假设。

用敏捷开发理解“反复做 Pilot”

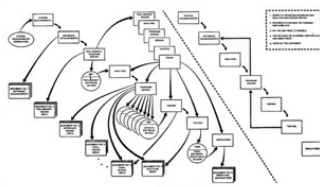
- 不必把 pilot 单独隔成一个大阶段，而是把试做、反馈、修正反复快速做
- 共同点：让关键假设尽早遇到证据；敏捷原则将频繁交付、合作与调整变成持续过程。



画面 01:05:43

emphasizing that the customer participates continuously via requirement updates throughout the development cycle.

Figure 10. Summary



[Image #1: Summary flowchart of the software development process - a large central iterative loop surrounded by review ova Figure summary: This is the closing summary figure of thegh No paper. It presents the author's full recommended softwarereshed development process as one integrated diagram: a large iterative development loop in the center (preliminary design → analysis → program design → coding → testing → operations, driven by a spiral of "critical software reviews") with feedback arrows looping back from every stage to earlier

royce.md 696-725/779 92%

Thinking...

把 Royce 当坐标系，TDD 可以看成同一条判据在另一个时间尺度上的重写。我们先固定判据，再逐条映射，最后说 TDD 越界的地方。

一、先固定 Royce 的判据

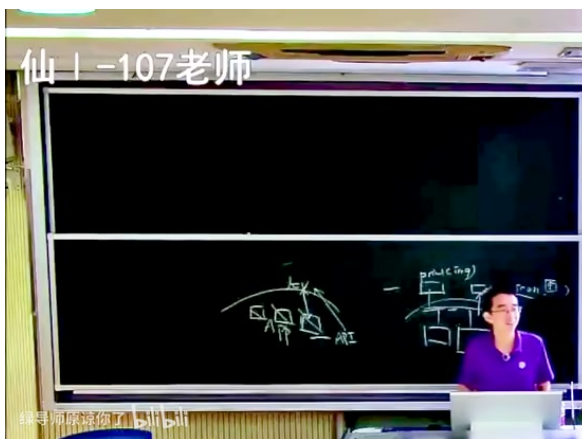
他整篇论文只讲了一件事：

缺陷被发现的时间 → 返工的距离 → 代价。测试是第一个“物理”事件，所以那里爆炸最贵。过程设计的目标，就是把代价积分压到最小。

五条建议全是这条判据的实例。TDD 不是新原理，是把这条判据的回边长度从“月”压到“秒”。

二、逐条映射

Royce	TDD 里的对应物
Step 1 设计先行 ("design at the risk of being wrong")	Red 阶段。写测试就是先定接口、契约、数据形状、调用方式。TDD 常被说是设计方法——正因为这一步就是 Royce 的 preliminary program design，只是粒度从“全系统”降到“一个函数”
Step 2 文档 (doc = spec = design 三位一体)	可执行测试即规格。Royce 的 1500 页规格，在 TDD 里被压成



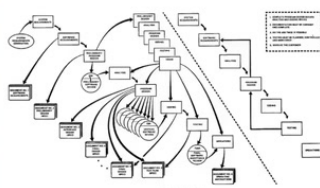
画面 01:06:00

01:06:07

然后你在TDD的意义上，你看设计先行，那你设计先行的时候，你就干脆你就先把测试用例定好，对吧？你输入这个，它就应该经历这个流程，输出这个，然后你马上就对还有一些敏捷开发等等的概念就可以用上了。所以实际上现在做的事情不完全颠覆过去的经验。

emphasizing that the customer participates continuously via requirement updates throughout the development cycle.

Figure 10. Summary



[Image #1: Summary flowchart of the software development process - a large central iterative loop surrounded by review ova Figure summary: This is the closing summary figure of thegh No paper. It presents the author's full recommended softwarereshed development process as one integrated diagram: a large iterative development loop in the center (preliminary design → analysis → program design → coding → testing → operations, driven by a spiral of "critical software reviews") with feedback arrows looping back from every stage to earlier

royce.md 696-725/779 92%

Thinking...

把 Royce 当坐标系，TDD 可以看成同一条判据在另一个时间尺度上的重写。我们先固定判据，再逐条映射，最后说 TDD 越界的地方。

一、先固定 Royce 的判据

他整篇论文只讲了一件事：

缺陷被发现的时间 → 返工的距离 → 代价。测试是第一个“物理”事件，所以那里爆炸最贵。过程设计的目标，就是把代价积分压到最小。

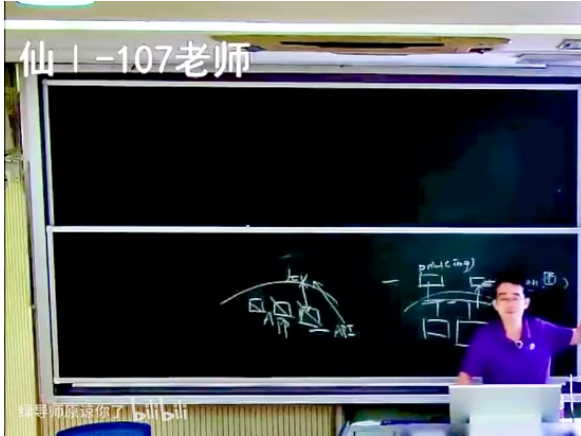
五条建议全是这条判据的实例。TDD 不是新原理，是把这条判据的回边长度从“月”压到“秒”。

二、逐条映射

Royce	TDD 里的对应物
Step 1 设计先行 ("design at the risk of being wrong")	Red 阶段。写测试就是先定接口、契约、数据形状、调用方式。TDD 常被说是设计方法——正因为这一步就是 Royce 的 preliminary program design，只是粒度从“全系统”降到“一个函数”
Step 2 文档 (doc = spec = design 三位一体)	可执行测试即规格。Royce 的 1500 页规格，在 TDD 里被压成

01:06:32

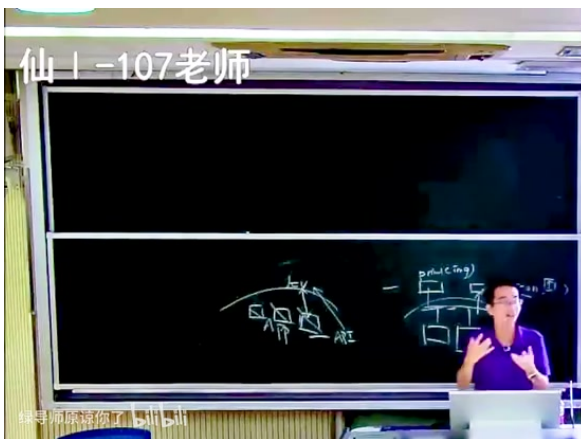
这些老的东西我一直说我很喜欢这些很多图灵奖的工作。他为什么能得图灵奖？就是因为他确实是在一个完全大家不知道的情况下，确立了一个1st principle，告诉大家这件事情应该这么做对吧？不仅是图影讲这个奠定了软件工程基础的rise的这个对吧？瀑布我讲它瀑布模型，开玩笑叫瀑布模型的paper，不是瀑布managing的development。Large software systems这样的一个经典的文章，它就给我们很多的启发，在它上面长出了这个世界的很多分支。



画面 01:07:00

01:07:08

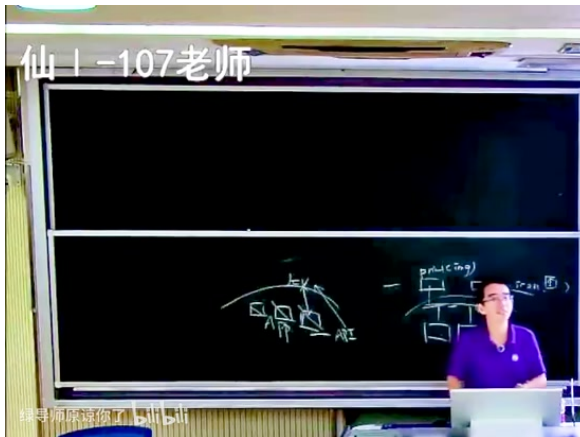
当然也更多好玩的是你有一天发现他一开始有个地方方向想错了，然后把全人类都带歪了。如果全人类从某一点开始能够换一个方向，那这个全人类也许就立即快速走上了正确的道路，比如说直流电和交流电，爱迪生和特斯拉ok这是do twice。然后你也可以用敏捷开发去理解这个do it twice对吧？你不需要把pilot理解成一个完整的流水，对吧？而是所谓的敏捷。就是你的第一个版本，你不是do it twice，你是do it infinite many times。你上来第一个版本，哪怕它只有一好多word，它只是打印一个你的流，或者就是每个模块都自报一下。家门能把这个模块串起来，这也是一个好的product。然后在在这个意义上不断的把软件向后向后向后演进，对吧？



画面 01:08:00

01:08:06

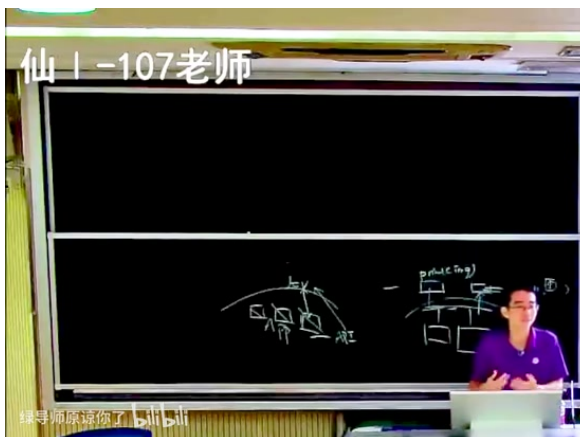
让关键假设尽早遇到证据，你一开始不确定的东西，立即因为你有一个原型了，有一个敏捷的原型了，你就可以立即验证它。所以你看这个1st principle在这么多年软件工程发展的实践当中都还是存在的。所以说AI第一个A来了很焦虑，就是AI I确实学会了AI也许他品味有一天也学会了对吧？就他可以从这些论文里面学到这个品味。我们很焦虑，AI能帮我们做事儿，但是我们仍然或者说现在人类还有一种傲慢，觉得我们还是希望能够理解AI到底做的什么，能够判断A到底做的对不对。就真的到那一天，AI I做的所有东西都是远超人类，都是对的。然后就只有人会犯错误，AI不会犯错误的时候，我们就没有，我们就不要去理解它了，我们也理可能也放弃就去放弃理解它。但至少在今天AI I虽然财政能力在增长，但我们还是至少能驾驭能理解AI I等人能活下来。大家在这个斩杀线上升的基础上，你就知道我们怎么样用好AI I怎么样去理解这个里面的软件工程的智慧。



画面 01:09:00

01:09:16

然后我是怎么理解这个呃ro的这个pilot的model？我因为我是一个的人，我们往前一点对上一系。然后我说我把这个ro模型理解成一种data dependency，然后修改是有代价的，这样可以吗？对吧？你就可以跟他聊，当然这是我自己的理解，我觉得还是自治的。你看这个瀑布的模型，他说从需求分析到系统设计，到系统实现，到测试验证，到交付维护。你可以把它想象成每一个过程都想象成是一个program，就是人，我现在把人想象成程序，人和程序就是等价的，我的人脑我就神经网络，然后我可以做chain of thought，对吧？我也学会了，我先学会了语言，我学会了说话。



画面 01:10:00

Do It Twice

用今天的 MVP 理解 Royce 的 Pilot Model

- Minimum Viable Product: 先做一遍，从中学学习，再完善产品
- Royce 的 **do it twice**: 先用 pilot model 经历关键问题
- Pilot 重点验证技术与运行风险；MVP 更关注用最小投入验证用户与业务假设。

用敏捷开发理解“反复做 Pilot”

- 不必把 pilot 单独隔成一个大阶段，而是把试做、反馈、修正反复快速做
- 共同点：让关键假设尽早遇到证据；敏捷原则将频繁交付、合作与调整变成持续过程。

瀑布模型就是 Data Dependency Graph

需求分析 → 系统设计 → 系统实现 → 测试验证 → 交付维护

- 不做需求 (intent / spec)，设计就会漂移
- 设计 (spec) 错了，实现就会漂移
- 没有实现 (impl)，谁会想去测试？
- 当然，交付之后还得有人擦屁股

TDD：拿 Testcase 当需求与设计

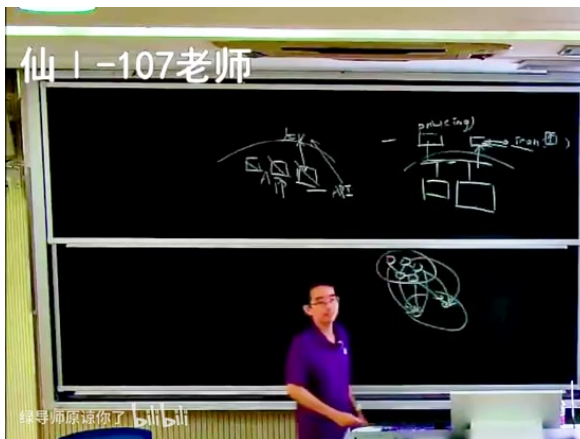
- 后来大家才反应过来：可以先写 testcase
- 写下预期行为只需要需求；验证实现是否满足预期，才需要实现。
- 产物之间有依赖，不等于所有阶段只能完整地做一遍。

01:10:10

然后你们在内心，我不知道你们是怎么样的，至少我自己在想问题的时候，我还是会对齐到语言上。我哪怕不说话，我依然会说ok我要我要这样或者怎么样，这样会有什么样的后果？我还是用语言来对齐我的内脑内的这个思维链的思考。所以change是一个我觉得很有意思的机制。

01:10:29

然后这就是一个如果你把它产生程序和artifact，那就是一个data dependency graph，对吧？它是一个有向无环图，如果我在就我可以把这个稍微展开一点。比如说我在做需求的时候，我需求里面有好几个点，这些当然也有之间可能也有相互的关联。有些可能是没有关联的我在做设计的时候，我的这个设计可能就和这些需求的点是关联的。我做的这个设计，然后我说这个设计又是关联的，然后这个关联关系会传递对吧？关联关系会传递。



瀑布模型就是 Data Dependency Graph

需求分析 → 系统设计 → 系统实现 → 测试验证 → 交付维护

- 不做需求 (intent / spec)，设计就会漂移
- 设计 (spec) 错了，实现就会漂移
- 没有实现 (impl)，谁会想去测试？
- 当然，交付之后还得有人擦屁股

TDD：拿 Testcase 当需求与设计

- 后来大家才反应过来：可以先写 testcase
- 写下预期行为只需要需求：验证实现是否满足预期，才需要实现。
- 产物之间有依赖，不等于所有阶段只能完整地做一遍。

01:11:03

然后我有一个这样的data flow graph。那就意味着如果我这种一烫头下来，一旦发现，所以你看这个跟rois的解释又是一样的。如果一旦发现这个test错了，然后错了，如果我们一路往上追溯发现这几个design错了，那就惨了，所有依赖它的东西全部都毁掉了。所以这就解释了为什么cost很高。

01:11:34

所以你看你可以用自己的每一个人，就我相信每一个人对这个模型的理解都是都可以是不一样的。也就是为什么我们非常需要一个非常需要一个个性化的教训。每一个人都有自己的diversity，他成长的经历不同。然后我就觉得因为我是做还算是一个做过一点R的操作系统人。所以我就用会很自然的用我在science里面习得的概念去解释这个解解释这个世界，这基本就是软件工程后来真正采用的形式法方式，这也是一个1st principle。然后他解释了这个对吧，解释了解释了这个模型。

仙 I -107老师

simulation the project manager is at the mercy of human judgment. With the simulation he can at least perform experimental tests of some key hypotheses and scope down what remains for human judgment, which in the area of computer program design (as in the estimation of takeoff gross weight, costs to complete, or the daily double) is invariably and seriously optimistic.

[Image #1: Flowchart showing a preliminary pilot program design proceeding through analysis, program design, coding, testing, and operation. Step 3: Attempt to do the job twice - the first throw result provides an early simulation of the final product, with feedback loops from usage back to earlier stages]

Figure summary: The diagram illustrates "doing the job twice"

royce.md 4/5-504/779 63%

边，架构与系统性风险仍需提前设计。

我把royce的模型，理解成一种data dependency，然后修改是有代价的，这样可以吗？

Thinking...

可以，而且这不是类比上的小聪明——这基本就是软件工程师后来真正采用的形式化方式。但有两个地方这个框架会漏掉，需要打补丁。

一、它在哪（而且可以推到很强）

如果按编译器的数据依赖读：

Fig.2 = 直线型数据流，每个阶段消费上一个的产物。调度长度 = 关键路径 + 工期。

Royce说的“迭代只限相邻步骤”= 依赖的局部性假设（就像 loop nest 里依赖在 tile 内部）。

Fig.4 = 长程依赖（Long-range dependence）：测试通过一条不受约束的通道依赖需求层。他在 Fig.3 里暗示依赖是局部的，Fig.4 告诉他不是——而长程依赖没法靠“把迭代限制在相邻步骤”tile 掉。

迭代代价 = 依赖传播锥（cone of influence）的大小。100% 延期 = 一条长边触发的金流水钱 flush。

再进一步，把“修改有代价”当成第一性原理，Royce 的整个论证就变成一个期望代价最小化问题。而且他是这么权衡的：Step 1 里那句“批评是对的，but it misses the point”——批评者优化的是出错的概率（设计在真空中会做错），他优化的是出错的代价（那

画面 01:12:00

01:12:23

那如果我们知道这个data dependency一旦这个改，如果这个触及到它上游的东西需要修改，那我们该怎么办呢？那我们很自然的，我们可以把它割小，MVP就来了，我们不要一次性的产生大量的de泛定性，而是我们从一个宽度优先对吧？这是一个breakfast，你的一个用breakfast search的方式构建软件，先做第一层，再做第二层，再做第三层，我们变成一个deficit对吧？这不就是敏捷开发吗？我们先做一个minimal viable product，把一些重要的优先级最高的这个defen流程走通，走通了以后再不断的向右扩展一个D F1个迭代的，这就这就对了这就对了，所以这就是软件工程没有什么没有什么真正困难的。我们越早能够得到反馈，我们就越能把这个路线上面的不确定性这个gap？

仙 I -107老师

瀑布模型就是 Data Dependency Graph

需求分析 → 系统设计 → 系统实现 → 测试验证 → 交付维护

- 不做需求 (intent / spec)，设计就会漂移
- 设计 (spec) 错了，实现就会漂移
- 没有实现 (impl)，谁会想去测试？
- 当然，交付之后还得有人擦屁股

TDD：拿 Testcase 当需求与设计

- 后来大家才反应过来：可以先写 testcase
- 写下预期行为只需要需求；验证实现是否满足预期，才需要实现。
- 产物之间有依赖，不等于所有阶段只能完整地做一遍。

画面 01:13:00

01:13:35

因为我们每一个阶段这个瀑布模型，所以瀑模型里面其实有in的模型里面，它它对比比如说前期的阶段，比如说需求阶段，它就对应的我们要把intent确定，吧？要把intent确定，然后在设计阶段我们要把spiack基本上确定。当然可能比如说这个spspack也会在需这个需求的阶段也会稍微做一点，然后最后到这个编码测试的时候才是硬盘配。然后因为我们知道这个东西是有gap的，然后这个gap就意味着每一个你看都会拆成好多个小点，而且会对后面产生dependency。所以我们应该做这个MVP或者应该做敏捷开发，或者应该做TDD，它是显然是一个正确的事情。



画面 01:14:00

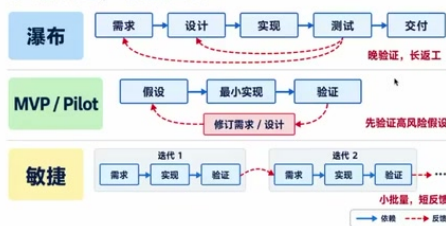
01:14:24

对所以我觉得是阅读这些经典的著作是真的是非常有启发性的，也是一个非常值得大家学习的学习方式，对吧？因为一般经来说，经典都是记录了一个领域最初遇到的，也是最重要的根本性的问题。然后他在方向上找到一个出路，尽管可能我在在事后我可以援引其他的概念比他解决的好。就比如说我就很喜欢这个 data defenancy，一会儿你会看到data defenancy在整个软件工程的发展历史上都是可以你都可以往这个方向去思考，然后去导出更多的概念。这样好，那么这是软件工程的，相当于我觉得这有点像奇点，就是你你你学软件工程这个1st principle是这就是他开发就这个软件到底是什么，对吧？

用 Data Dependency 理解开发过程

MVP 与敏捷：让反馈更早回来

依赖不变，反馈变早



• 比较的不只是阶段顺序，还有：多晚才得到反馈，得到反馈时已经做了多少依赖它的工作。

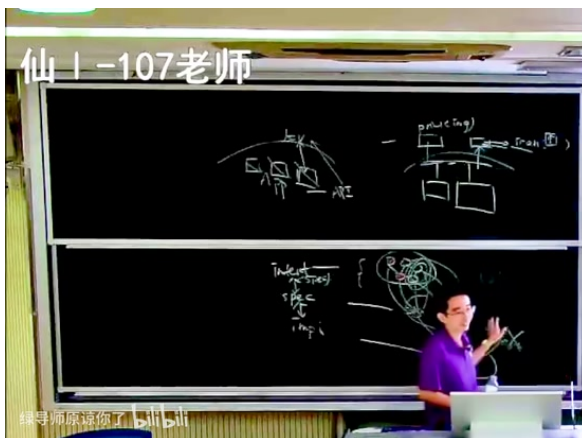
阅读“开天辟地”的经典

即便今天已过时，通常仍然信息量巨大

- 经典记录了一个领域最初遇到的问题，以及怎样找到出路。
- 在今天的背景下，反而非常容易理解

带着今天的经验，回到当年的问题

- 读 Royce 时，可以追问：哪条建议来自当年的硬件约束，哪条来自多人协作，哪条来自“必须运行以后才知道”？



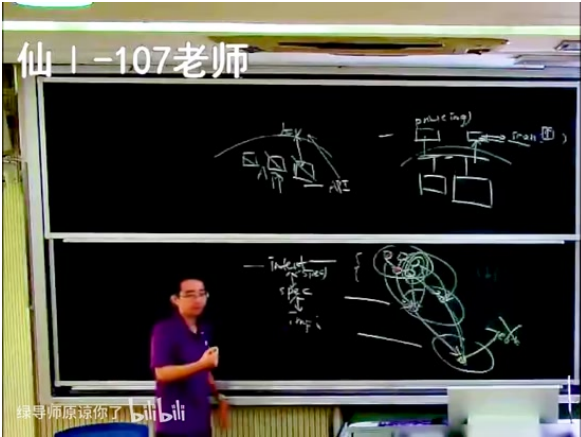
画面 01:15:00

01:15:20

你当然可以用定义说是代码和数据。但当你从这个视角看的时候，你就能继续往后推出更多的东西。然后在现在经历了几十年的发展，软件工程变成什么呢？为了解决这里的问题，他为每一个阶段都创建了方法和工具。然后这些方法和工具可以减少你返工的概率。比如说我不知道这边有没有软练的软件的同学们，软件同学你们学软工课的时候，个在设计阶段他应该是个io标准的。但其实我对这个印象已经不深了，所以我不是做这个方面软件工程的。

01:16:00

比如说这个地方它有一个checklist，你很烦你上那个软件工程课的时候，你老师叫你写一个文档，然后他大概有一个非常长的check list，说这这个有没有做到这个有没有做到这个有没有做到，有没有沟通过。然后你就觉得nonsense，因为你是做OG一体的，你觉得这个checklist nonsense，但checklist实际上是有用的，它减少了你fa的概率。比如说这个checklist的最后总有一个你是不是和这个客户最后沟通过，然后你在学习你在上学的时候会觉得这个很讨厌。对。但是当你真的和客户沟通过并且给他打勾的时候，实际上你大幅减少了这个in不符合客户要求的概率，这就是一个方法，这也是一种工具，这是一个一个有人文关怀的工具，它没有任何的代码。



画面 01:16:00

软件工程的方法和工具

为每个阶段创建方法学和工具

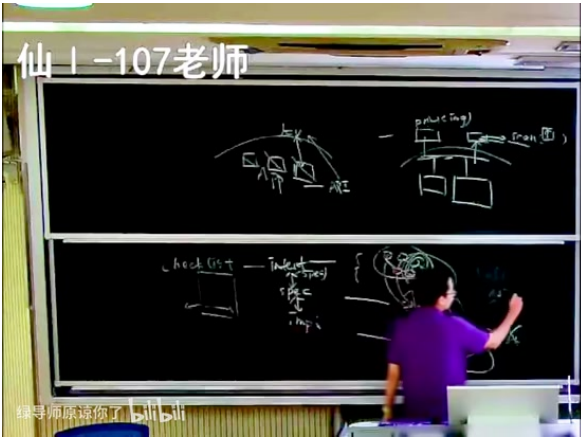
- 软件工程：为每个阶段都创建方法学和工具，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
 - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

01:16:51

然后当然我们现在比如说当我们说比如说现在我们做软件工程研究的时候，更多的时候说是当我们有了代码以后，吧？当然我们比如说有代码，有了代码时能不能做自动的测试，还无穷多的这个灌水的paper。然后有了代码以后，里面有没有这个bad对吧？



画面 01:17:00

软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建方法学和工具，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
 - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

01:17:11

Bad smell这个你们都知道。就比如说你如果看到一个代码，它在等号左右从来不写空格，你会认为这个代码是不好的代码，对吧？这个规定在AI I时代之前都成立。但是我发现在g p 6发学会省token以后，这件事情变了，它会写出这个极为无法理解的代码，然后把很多的语句放到一行里，然后那个代码就是对的。后我不知道他是怎么弄对的，但是反正他是对的。然后也许我们永远不应该去读那样的代码，对吧？

01:17:47

这是一个有意思的观察，但至少我们知道的是什么呢？是软件工程，对吧？当有了这样的一个起点，为了把这个大的工程，系统性的工程扶起来，软件工程里面还有很很多很多的细点。你们如果去读研究生，如果要选这个比如说软件工程的方向，你们大概率会在这个里面的一个非常小的一点，对吧？比如说针对这个软件开发流程当中，编码的这个过程当中，这个Java代码的这种类型的bad sm的。一个什么样更好的检测，去做一个什么东西，对吧？

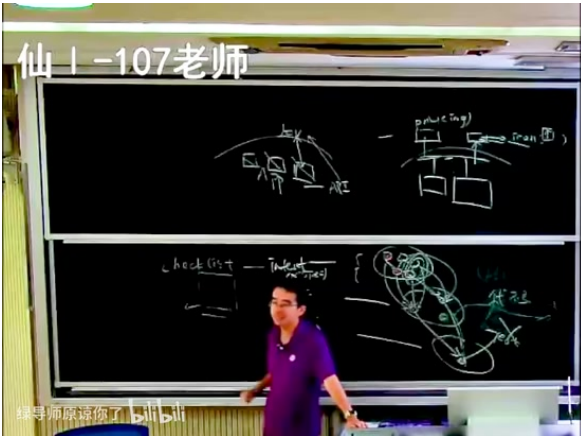


01:18:22

当然你你在就你在做深的过程中，你不要忘记去看这个树林，去看这些经典的东西，你就会有更好的认知。就是对人来说也是深度和广度都是对吧？你要在深度里面训练你才有更长的思维链。你在广度里面训练才有更好的对整个这个世界的感觉。

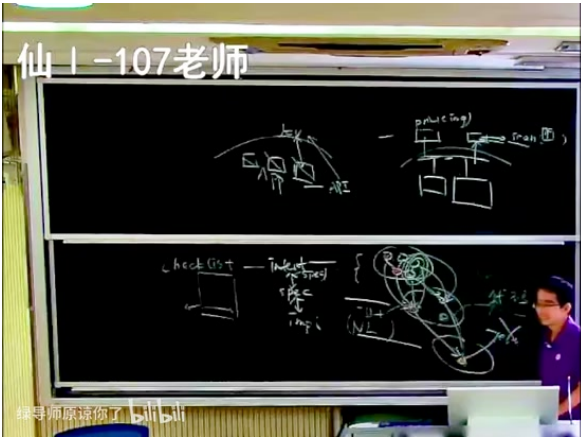
01:18:44

然后很有意思，你就会发现所谓的软件工程，为了在每一个阶段创建方法学和工具，它就需要在不同的阶段用不同的语言，对吧？比如说在checklist的时候，你是不太可能，尤其是在大圆模型时代之，你是不太可能用一个编程语言去做一个checklist的。虽然今天我们都用mark down对吧？但那个时候真的就是一个dock叉的文档。然后你把那个文档打印出来，然后要存档，你可能是手写的跟客户沟通的那个记录。然后你到这个编程的时候又有编程语言，然后在编程语言之前你想马上就看到了一个gap。



01:19:23

那个时候没有大圆模型这种万能的编译器，你只要有自然语言就可以把它编译成任何一种语言。他在不同的阶段你会发现马上你就发现有些在有些比如在设计阶段，我们没有一个语言来帮助你做设计。在比如说在1970年的时候，在设计的时候，那大概用的都是自然语言，对吧？就是一个文档是一个设计文档。你们学软件工程，我记得我那个时候学软件工程就是写设计文档。然后你写先写需求文档，需求文档写完写设计文档follow那个模型，你马上就会想稳当的最大缺点是什么？最大缺点是写文档的是人人会产生幻觉，人会产生style。



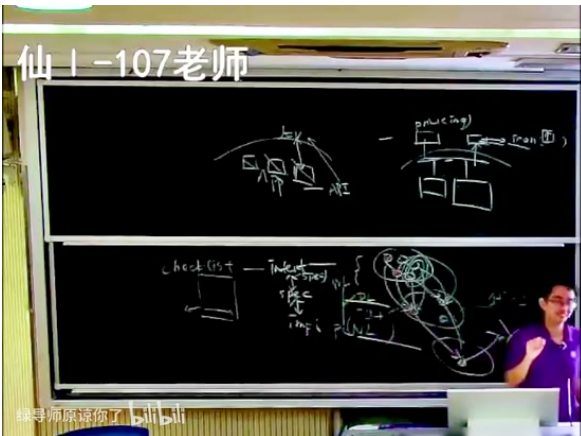
画面 01:20:00

01:20:11

人和人之间当我们讲同一件事的时候是无法对齐的。这件事情在你和室友之间已经发生过无数次了。你说的是这个室友理解成了你花了很多的时间跟他沟通和对齐。当然我觉得跟比如说跟女朋友对齐会比跟室友对齐更困难，这个不重要。

01:20:30

你作为一个软工人？你在1st principle上，如果你想要应用这句话，为设计阶段创建一个方法学，创建工具去减少返工的概率，那你要做什么呢？你要做的是个介于自然语言和编程语言之间的一个设计语言，当然也是一个domain specific language。你们想到什么了，对吧？你们想到了软件工程课上面讲的各种各样的东西。



画面 01:21:00

软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建方法学和工具，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
 - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

1/19

软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建方法学和工具，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
 - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

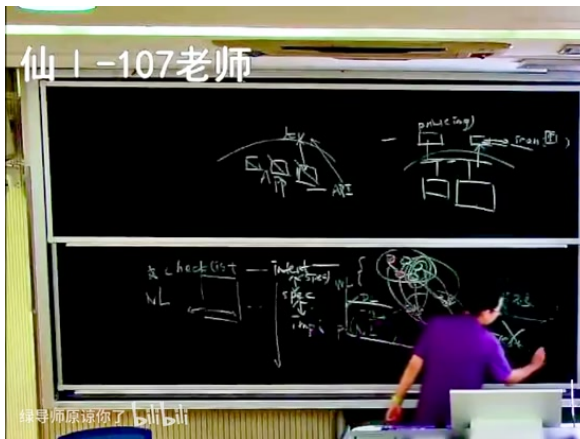
1/19

01:21:01

Checklist就这个checklist虽然它是自然语言，但实际上它也是一种中间语言。因为它有格式的规范，它要求你必须是一条的，每一条必须要打勾或者打叉，对吧？它已经是介于一个自然和形式化，它是有语义的，它是有一定的语义的。这个语义当然是虽然是由人来检查的，但人在于对于勾和差来说的对齐，就比我直接写一个文档的对齐要来的更for一点。所以你看它这个for的程度，就是它有确定性的语义的程度，是从设计的阶段到下面越来越formal的这也迎合了从intent到sp到intation之间的这个gap的解决，所以你就开始理解软件工程这个学科了。

01:21:48

那我们学的时候是什么？怎么学呢？我们是button map的，我们是先学编程对吧？因为你你所有的你看计算机软件工程，你们第一件事是先学工程，对吧？先学怎么做一个码农。因为你不理解代码，你就没有办法理解整个过程。



软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建**方法学**和**工具**，尽可能减少返工概率。

不同阶段，不同的“语言”

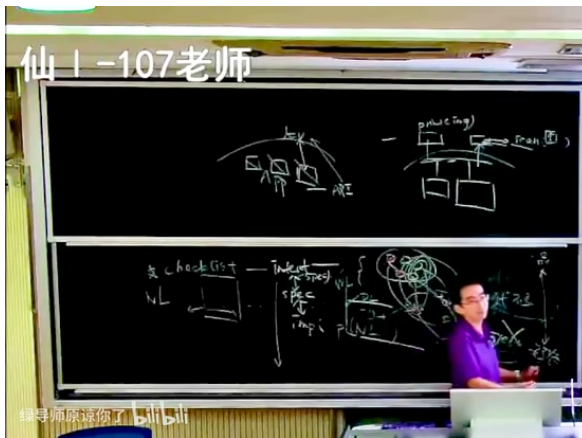
- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
 - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

01:22:06

但是AI来了以后，我觉得变了，你可以先从产品经理开始做产品，完了我脑子卡壳了，产品是这个产吗？不是这个产，我脑子突然卡壳了，就product ok短路的我上年纪了，这个艾way不重要，你是可以先了解整个过程的。就是我觉得我们以后computer science，如果要可能我要来改的话，要产product，就第一个学期就可以做。

01:22:49

因为反正你后面所有的事情都是web部，然后你不去控制它的质量，你也不需要知道对吧？如果我上第一门课我不去，你不需要控制这个质量。因为你你根本就不知道，不理解代码，理解后面是怎么运作。但你依然可以设计好的产品，你只要有有好的想法就可以做对。然后在这个过程当中，其实诞生了很多的有软件工程智慧的方法和和工具对吧？比如说这个check list，然后它不仅 checklist，还有一些像这种决策表，就那种决策数。



画面 01:23:00

01:23:21

就说它有一个很好玩，就是就你们有大家觉得我是一个理科生，我在软件工程学院软件学院学文科。他给我一个checklist说只有当所有的这个勾都打上了，你才能进入下一个阶段，这是什么？对我来说是nonsense，但是这个nonsense赋予了这种形式的语义，避免了遗漏和矛盾。他会要求你，他甚至会有这种checklist，要求你把所有的文档两两摆在一起审查一遍。你会想我靠，我才不想做这个事，我是写程序的对吧？但就是这样的，这个两两摆在一起检查一遍，避免了这个gap的漂移，这都是智慧。但是他失败过了，他失败过。

软件工程的方法和工具

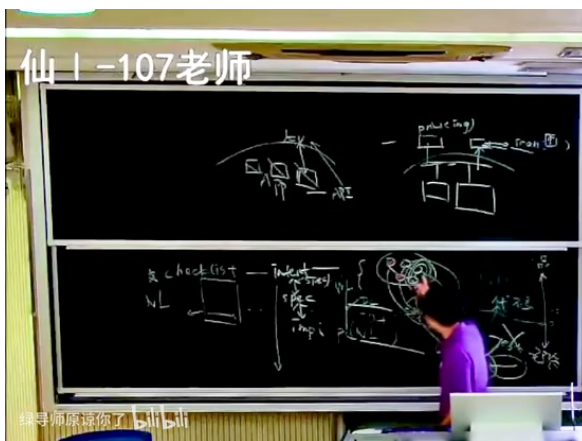
为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建方法学和工具，尽可能减少返工概率。

不同阶段，不同的“语言”

- 在没有 LLM 这种“万能编译器”的时代，不同阶段需要不同的“语言”：
 - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
 - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
 - 代码实现
- Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

5/5



画面 01:24:00

01:24:03

然后在管理团队的过程中形成的管理学的经验，这个软工人也可以学好。然后还有各种是比如说在这个测试上面也有很多经典的工作。比如说pick check，基本上算是我们整个测试领域的开山鼻祖之一了，对吧？

01:24:21

让人描述性质，由工具生成输入寻找返利。然后这是什么？这体现了一个什么逻辑呢？你做任何事都会错，你写代码就是就是你做任何翻译，从上到下的翻译都会错。你从specification翻译到代码，尤其你有些隐含的，比如说你的这个程序不能crash，对吧？不要有no point资源不能泄露这些隐藏的。代码里面的这些specification你也要遵守。那你有没有可能我在代码里面就写一个property，然后这个property可以自动变成好多好多的测试用例。然后如果全部都通过了，我对代码就有更大的信心，对吧？

01:25:00

你做你现在做的软件就是这样的，你写一份代码，你永远不知道它对不对。但是如果你做了测试，你观察过它的结果，你这就对他信心会多一点。当你独立做的质量保障做的越多，你对他的信心越大。除非是你最终写了一个证明秦率的证明，说证明这个程序你的情形可能是更大的对吧？

方法和工具的智慧

需求与测试：把规则变得可检查

- **决策表**：把条件组合与预期结果对应起来，暴露遗漏和矛盾。ISTQB §4.2.3
 - 例：学生优惠遇到满减，究竟能叠加，还是只能选一个？
- **基于性质的测试**：QuickCheck 让人描述性质，由工具生成输入、寻找反例。
 - 例：排序结果既要有序，也要保留所有元素及其出现次数；只检查“有序”，返回空数组也能过关。

维护：改一句需求，会牵动什么？

- **需求追踪矩阵**：维护需求、设计与测试之间的关联，用于检查覆盖情况和分析变更影响。NASA
 - 例：“允许退课”改成“开课后一周内允许退课”，哪些设计和测试需要重新检查？

6/5



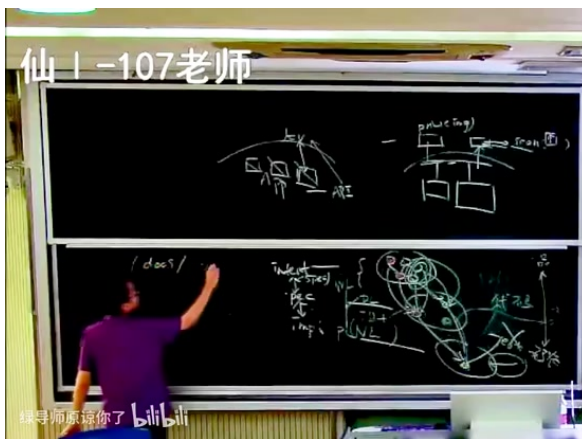
画面 01:25:00

01:25:22

然后这就是软件工程里面人研究的各种各样的有意思的东西。然后再比如说这个需求追踪矩阵，我前面上课的时候讲过一个概念叫traceability，这个是软件里面很重要的一个概念。然后这个traceability做的是什么呢？做的就是把这个data defense graph里面的，以前是没有的对吧？如果你从文件系统或者项目的角度来看这个软件，那这个软件就是好多的文件对吧？

01:25:57

什么杠do下的什么点MD然后source下的一个什么什么点TS，然后又有一些其他的什么东西，还有一个read me对吧？我说这是一个为因为文件系统一开始大家用了而不得不做的一个work around。然后这个东西其实承载了这个东西，它是这个对吧？它承载了整个软件，然后tracability要求你要用链接或者用什么文档，或者用什么样的方式，要求你的每一个代码和每一个需求点都能对得上，需求和需求之间也能对得上，需求和设计之间也能对得上。如果所有这些都能对得上，它有这种独立的互相的交叉检验你对你软件的质量就更有信心了。



画面 01:26:00

01:26:41

他也是没办法，就是人的能力是有限。他假设就是你你我们的每一个人，我们在上大学的时候学这个软件工程，总是觉得我是一个最我我要把自己变成世界上最厉害的马农，那就够了。如果我是世界上最厉害的马农，那我就保证能在软件企业里面找到一个很好的工作，对吧？这是这个社会给你的contract。后但是从资本家的角度来看，或者说从你要完成一项任务的角度来看，他更重要看到的是没有一个码农能够单独完成他的项目。他只能说我码农有好有坏，我得花多少钱雇多少码农，然后最终能把我的项目给交付。所以你看现在每一个人都当产品经理，你第一门课就要学做产品，对吧？

方法和工具的智慧

需求与测试：把规则变得可检查

- **决策表**：把条件组合与预期结果对应起来，暴露遗漏和矛盾。ISTQB 54.2.3
 - 例：学生优惠遇到满减，究竟能叠加，还是只能选一个？
- **基于性质的测试**：QuickCheck 让人描述性质，由工具生成输入、寻找反例。
 - 例：排序结果既要有序，也要保留所有元素及其出现次数；只检查“有序”，返回空数组也能过关。

维护：改一句需求，会牵动什么？

- **需求追踪矩阵**：维护需求、设计与测试之间的关联，用于检查覆盖情况和分析变更影响。NASA
 - 例：“允许退课”改成“开课后一周内允许退课”，哪些设计和测试需要重新检查？

2/3

方法和工具的智慧

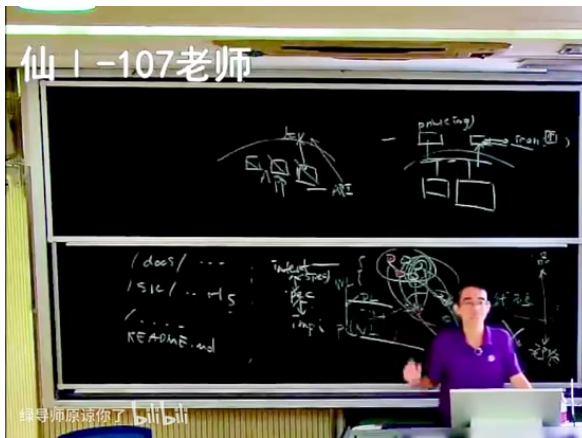
需求与测试：把规则变得可检查

- **决策表**：把条件组合与预期结果对应起来，暴露遗漏和矛盾。ISTQB 54.2.3
 - 例：学生优惠遇到满减，究竟能叠加，还是只能选一个？
- **基于性质的测试**：QuickCheck 让人描述性质，由工具生成输入、寻找反例。
 - 例：排序结果既要有序，也要保留所有元素及其出现次数；只检查“有序”，返回空数组也能过关。

维护：改一句需求，会牵动什么？

- **需求追踪矩阵**：维护需求、设计与测试之间的关联，用于检查覆盖情况和分析变更影响。NASA
 - 例：“允许退课”改成“开课后一周内允许退课”，哪些设计和测试需要重新检查？

2/3



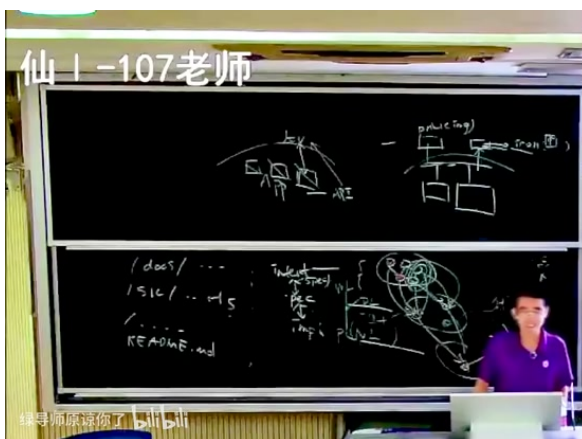
画面 01:27:00

01:27:30

你要调动，假设你有好多码农了，而且你现在马上就有对吧？DK算我觉得deepseeck绝对算是一个相当资深的马农了，绝对不是这种本科毕业生级别的。那你你马上就可以有2500并发的deep Seeck，你手上有2500人你怎么管？对吧？你高中毕业的时候，你做梦都没有想过你刚刚高考完，对吧？你接受了高考的这个训练以后，你马上就能管2500个人。

01:27:57

这个mindset可能要改一改。所以在这个里面就诞生过很多很多好玩的东西。比如说这些都是你们如果上软件工程课可能会讲比如说叫design by contract，叫面向契约的契约的编程，这里有一个文章我就不打开看了。



画面 01:28:00

01:28:19

I feel的这个语言在一应该是1986年左右。1986年他就说我要把这个霍尔三元组直接写到代码里，你看这样的一段上古语法就知道了。你看它叫function方式，然后他它那个赋值还有冒号，等于就是这个是当然这个一路也沿袭下来，有很多语言也采用采用了这条线，很自然语言function is你在这里写，先写文档，这个函数是干嘛的？Require它的前置条件do这是代码的实现。

方法和工具的智慧

需求与测试：把规则变得可检查

- 决策表：把条件组合与预期结果对应起来，暴露遗漏和矛盾。ISTQB 54.2.3
 - 例：学生优惠遇到满减，究竟能叠加，还是只能选一个？
- 基于性质的测试：QuickCheck 让人描述性质，由工具生成输入、寻找反例。
 - 例：排序结果既要有序，也要保留所有元素及其出现次数；只检查“有序”，返回空数组也能过关。

维护：改一句需求，会牵动什么？

- 需求追踪矩阵：维护需求、设计与测试之间的关联，用于检查覆盖情况和分析变更影响。NASA
 - 例：“允许退课”改成“开课后一周内允许退课”，哪些设计和测试需要重新检查？

契约编程：Design by Contract

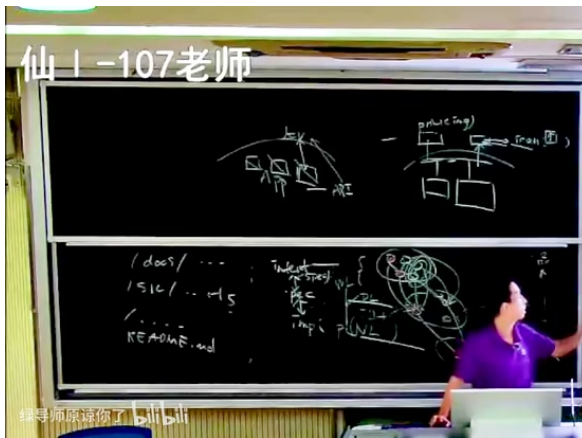
- Applying “design by contract” by Bertrand Meyer

调用者与实现者，各自负责什么？

- 像人类合作中的契约一样，接口明确双方约定：
 - 前置条件：调用者保证，调用前满足使用条件
 - 后置条件：在前置条件成立时，实现者保证

例：不允许透支的账户取款

- 调用 withdraw(amount) 前： $0 < \text{amount} \leq b$
- 正常返回后， $\text{balance} = \text{balance}_{\text{old}} - \text{amount}$
- 类不变量： $\text{balance} \geq 0$ ，在对象对外可用时。
- 这个设计也迫使业务选择：如果“余额”不定。



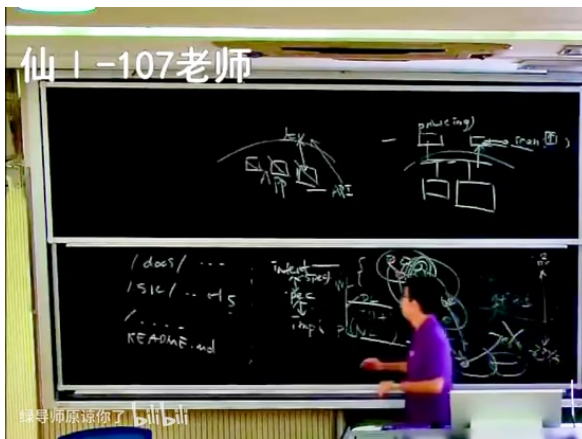
画面 01:29:00

01:29:01

In做完了以后它能够保证什么？他说好，你可以先就他鼓励你先不要写这个度留空，然后把这个软件各个模块文档precondition post condition给写出来，对吧？比如说在你有一个账户的取款，当你在写这个precondition post condition的时候，你就会自然的去关联到你的设计。比如说余额不足到底应该怎么样处理，对吧？

01:29:35

在你你你如果写第一个mv p的时候，你一定不会管这件事。你肯定是说直接让他crash或者怎么。你就是假设这件事情永远不会发生，排除在你的specispace之外。但是当你的软件不断的向后演进的时候，那么ok你的这个contract就变得非常重要。所以他希望能够在编程语言里引入这个。所以你看到这么多年软工人做的事情就是在这个sp上面横跳，对吧？



画面 01:30:00

01:30:05

从自然语言到这个半自然的语言，可以接纳设计的有一些逻辑公式的语言，它可以没有代码单纯的检验这个逻辑。比如说函我中间如果要函数调用，这个逻辑能不能成立。然后再再到现在的这个编程语言，后今天其实依然是一个非常活跃的方向。而且我觉得软件工程未来会发生很大的变化。因为AI来了，自然语言对不齐的问题解决了，checklist不好检查的问题也解决了，我们只剩下这个gap对吧？只有是我没有想清楚我没有说清楚，一旦我想清楚说清楚，可能真的就都能解决了。

把契约写进语言

上古语法: Eiffel

```
function(args) is
  -- Comment
  require
    Precondition
  do
    Routine
  ensure
    Postcondition
end
```

- 契约既是接口文档，也能在开启断言检查时检验本次执行。Eiffel 文档

今天依然活跃的方向

- 今天依旧是个不算太小众的领域: Dafny, Verus, ...
 - Dafny 内置规约与静态验证器; Verus 为 Rust 子集提供静态验证。
 - 它们进一步尝试证明: 在给定假设下，实现满足写出的规约。

4/5

把契约写进语言

上古语法: Eiffel

```
function(args) is
  -- Comment
  require
    Precondition
  do
    Routine
  ensure
    Postcondition
end
```

- 契约既是接口文档，也能在开启断言检查时检验本次执行。Eiffel 文档

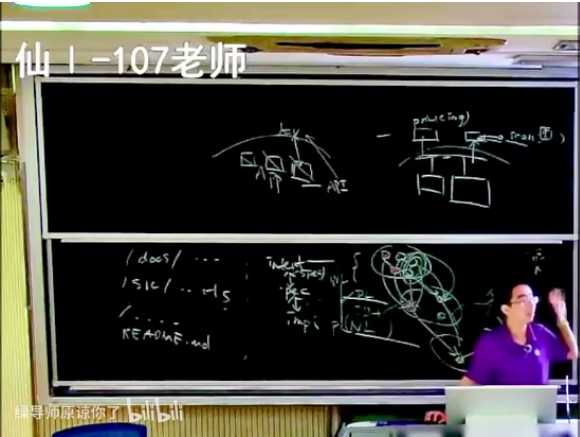
今天依然活跃的方向

- 今天依旧是个不算太小众的领域: Dafny, Verus, ...
 - Dafny 内置规约与静态验证器; Verus 为 Rust 子集提供静态验证。
 - 它们进一步尝试证明: 在给定假设下，实现满足写出的规约。

4/5

01:30:45

因为我们现在，比如说你看像埃菲尔，它催生了很多现在的项目，比如Daph尼us这些形式化验证的工具。你可以就是说大家的梦想，大家梦想都是我如果写一个这样的程序，我可以自动的证明我的函数在前置条件满足的时候，后置条件就会满足，无论我写什么就能证明。而且我还要证明整个系统把所有的前置条件后置条件连起来，它能够满足我的intent。然后我其实在ChatGPT出来之前，我上操作系统课的时候，我就预言那时候我预言30年，说说30年以后，那那可能2020年左右的时候还是2022年左右的时候，你如果你的一个程序过不了编译就错了。如果你一个程序里面有个事T然后这个事T就随便你写事，也可以写这种前置条件，后置条件可以写断言。



画面 01:31:00

把契约写进语言

上古语法: Eiffel

```
function(args) is
  -- Comment
  require
    Precondition
do
  Routine
ensure
  Postcondition
end
```

• 契约既是接口文档，也能在开启断言检查时检验本次执行。Eiffel 文档

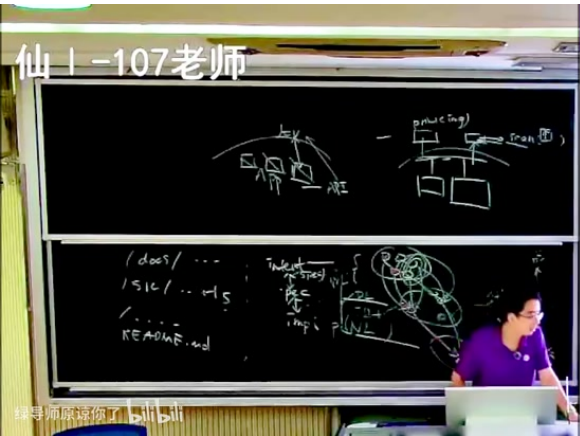
今天依然活跃的方向

- 今天依旧是个不算太小众的领域: Dafny, Verus, ...
 - Dafny 内置规约与静态验证器; Verus 为 Rust 子集提供静态验证。
 - 它们进一步尝试证明: 在给定假设下，实现满足写出的规约。

4/5

01:31:46

就是说我的函数要满足这个Lin左边，右边就Lin往左走再往后走。或者往后走再往前走。总之能回到自己，然后就写个sht然后任何的事T可以和业务逻辑相关的事T这个的只要我写出来它不能证明，或者说就这个东西无法证明的，这个程序不能编译，他编译百分之百保证这个东西才能证明，然后这我就这个软件的ga就代码开发的这种，我还需要这种property based test，我不需要，我只要直接把这个property写进去，他帮我证明了就可以了。然后现在从现在来看可能不需要30年，可能从那个时间点算，十年或者15年就可以就可以解决这样的问题了。然后在这个过程当中，有一个集大成者叫UL，大家绕不过去的一个名字，Unified Modeling Language。



画面 01:32:00

把契约写进语言

上古语法: Eiffel

```
function(args) is
  -- Comment
  require
    Precondition
do
  Routine
ensure
  Postcondition
end
```

• 契约既是接口文档，也能在开启断言检查时检验本次执行。Eiffel 文档

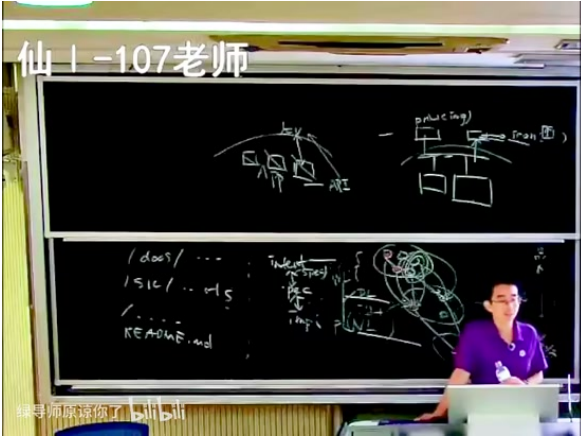
今天依然活跃的方向

- 今天依旧是个不算太小众的领域: Dafny, Verus, ...
 - Dafny 内置规约与静态验证器; Verus 为 Rust 子集提供静态验证。
 - 它们进一步尝试证明: 在给定假设下，实现满足写出的规约。

4/5

01:32:37

他的想法也很简单，既然这个世界这么复杂，对吧？在不同的层次上要用不同的语言。你实现的时候是C或者Java，你在设计的时候，当然实现的时候也可以用这个，I feel也有一些别的专门用来做设计的定义语言，然后在做需求的时候就是DSR。那我们有能不能有一种中间语言统一整个面向对象的世界，对吧？他要做的是一件非常有野心的事情，就是语义的对齐。而这件事情人类其实是就其实人类天生没有做好的事情。因为人类靠自然语言，然后自然语言是直接连接大脑的。



画面 01:33:00

Unified Modeling Language (UML)

能否有一种中间语言，统一整个面向对象世界？

- 人类的发展：自然语言（直接连接大脑 latent space）→ 数学语言（全世界自然对齐）
- UML 汇集了 Booch、OMT、OOSE 等方法：统一记号与语义，让团队和工具能使用共同的建模语言。[OMG 历史说明](#)

在没有代码时，尽量减少语义漂移

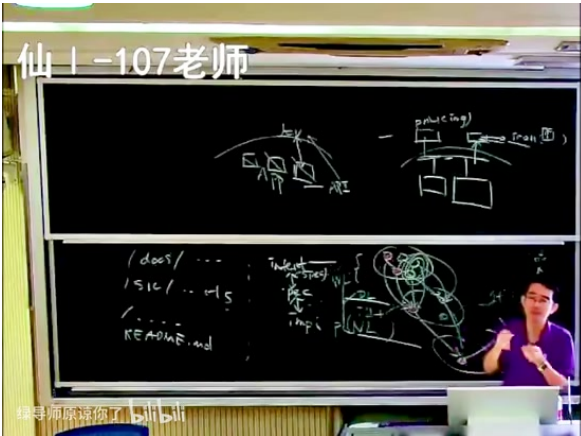
- 通过定义模型元素和（半形式化）语义，最大程度减少在没有代码时的语义漂移。
- 东西可能死了（LLM 时代谁还用 UML？），但设计时候的 first principles 一直指导我们。

01:33:17

Latin space 的当我跟你说的時候，說我還是我那個例子，我今天看到一個美女？在我的 Latin space，你們在每一個人的 Latin space 里面都構造了一個美女，但她都不一樣，這是对不起的。如果你要求我們把那個形象 3D 建模 bl 畫出來，這就就跟我讓這個 GPT 畫一個美女，讓那個 d 畫一個美女用 blender 畫，他們畫出來也是不一樣的？开玩笑但好像那個豆包生成的豆長一樣對吧？

01:33:46

這個語言就是很難對齊的，因為語言直連人類大腦的 Latin space，所以人類發明了數學語言。數學元是一個非常好的，全人類都能自然對齊的。因為我們知道什麼是自然數，我們是從公理一點點構造出來的。然後那個功里非常小，以至於人類還是基本上可以對齊的。一旦對齊完成，那我們用那個皮亞諾的功里就把整個計算機世界全部都建構成起來了。



画面 01:34:00

Unified Modeling Language (UML)

能否有一种中间语言，统一整个面向对象世界？

- 人类的发展：自然语言（直接连接大脑 latent space）→ 数学语言（全世界自然对齐）
- UML 汇集了 Booch、OMT、OOSE 等方法：统一记号与语义，让团队和工具能使用共同的建模语言。[OMG 历史说明](#)

在没有代码时，尽量减少语义漂移

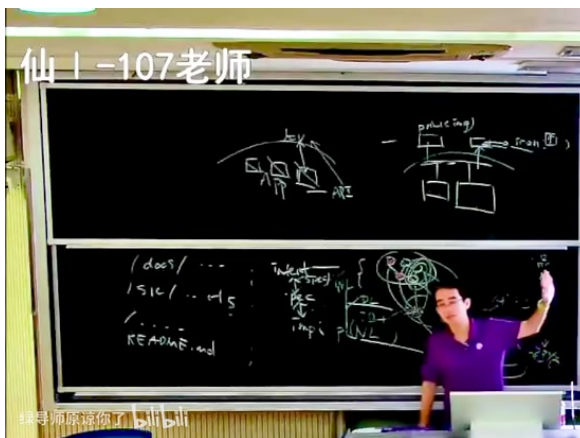
- 通过定义模型元素和（半形式化）语义，最大程度减少在没有代码时的语义漂移。
- 东西可能死了（LLM 时代谁还用 UML？），但设计时候的 first principles 一直指导我们。

01:34:11

在计算机上世界里看到同样的东西，它的语义是什么？就是绝对对齐的。Ok you没想要的是一个非常有野心的目标，他想要去对齐那个自然语言世界里面的东西，比如说需求设计这些说不清的东西。当然你是不可能百分之百对齐的，但是他希望把能对齐的部分和不能对齐的部分给剥离出来。

01:34:36

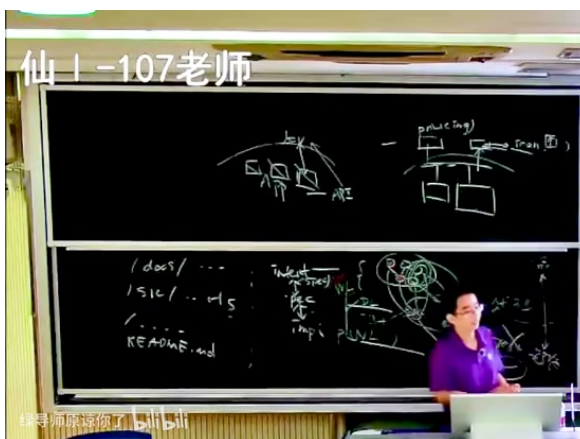
通过定义模型的元素和他们的语义，就有些东西是有语义的。比如说你们可能学过，比如就打比方类图，那类图的语就是确定的。我有个class叫student，student里面有一个field叫name，那它就是student name，它可以变成源代码里面的一个东西，它是有语义的。然后比如说sequence diagram，它的这个顺序图里面，他说我要从前到后按照某一个顺序时序来执行。那这个时序的先后就是绝对对齐的，你不需要用自然语言的对吧？如果你写就没有用，你写自然语言先后，如果当它发生并发的时候，到底谁先谁后能不能并发，他就是有歧义的。但是UM就消除了其在最大程度减少在没有代码时候的这种语义的漂移。



画面 01:35:00

01:35:26

当然我觉得今天我觉得U已经死了，这个就是因为所有它的假设都是我们的自然语言和形式语言是不能对齐的对吧？这是它的假设，所以它需要一层一层的中间语言。然后今天突然间这个自然语言可以和任何东西对齐了，反正AI就可以跟你问。所以U也是一个历史里面的东西了，但是其实它设计时候的这个1st principles，其实一直都在指导我们。对，所以我也特意重温，让这个AI帮我重读了一下这个。



画面 01:36:00

画面 01:36:03

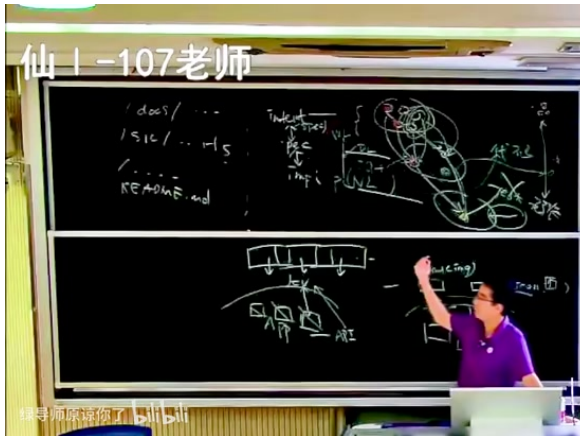
这个就比较恐怖了，一个700页的文档，就是我就就你看到我的工具还是可以的。就是我我特别做了这个可以scale长的文档，几千页的文档也没有也没有问题，这个就跟PDF是差不多的对吧？还是有接近的体验的。对，正好这个k米我就不做，这可能算一个广告对吧？因为他送了我一个699的一个月的套餐，然后说说他说看到了我的视频，后然后送了我一个699的套餐，然后我就直接让这个AI I帮我读了，对吧？就直接让AI I帮我读了这个整个U就是刚才你看到的这个du然后du完了以后我直接让AI I帮我备课，直接对着那个我大概我大概让AI I这样就是分suagent，然后分page ent，每个agent读50页，就我把一个非常长的文档对吧？

画面 01:37:00

所以我觉得这是一个非常恐怖的，我觉得也是一个蛮可怕的事情，就我把这个文档切成这样的带overlap也总会有一点，然后我让它每一个都提取里面的软件工程智慧，不符合AI时代的个不必要的东西。就把我的我把我的lecture no全部给他了，然后把它拿出来，然后总结成课件对吧？然后他就去读，然后花了18%的5小时额度，大概折算成这个699套餐的5.3块。

01:37:47

所以实际上我觉得这是一个值得我们思考的问题。就是当你拥有了更好的模型，我相信比如说你现在如果用 G P 6，可能你的价格会差不多，因为它很省。Token g p 6 的 token 真的非常省，可以得到差不多的价格下可能好得多的质量。这就意味着学生群体，就哪怕你们是学生群体，在拥有好的模型和没有好的模型的时候，你们的学习效率、学习成本也会有很大的差距。然后你的学习就成为了一个如果还好，现在 token 你可能也不算太贵。如果你有一个一百刀的这个 codex，其实我觉得还算是够用了。这个对大家学习来说，因为什么东西烧 token 呢？你去维护大项目，烧 token 大项目它你就会在大上下文里载入很多东西，然后你 100 万上下文马上就填满了，那个很黑 token。



画面 01:38:00

阅读 UML Spec

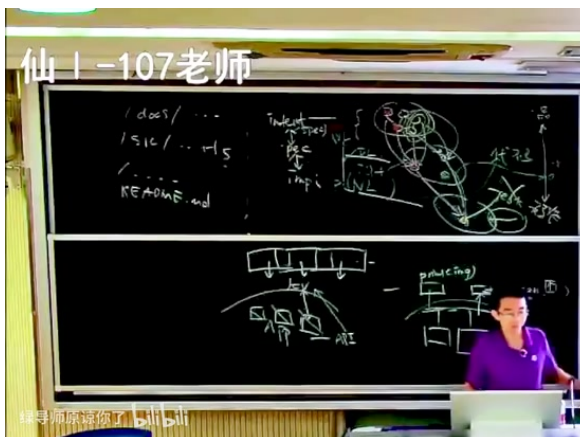
上大学的时候学过，感觉没学懂

- (怀疑老师可能也不太懂。)
- 依稀有一些模糊的印象
- PhD 时候也读了一些 papers
- Kimi 送了 ¥699 的套餐，直接让 AI 备课，花去 18% 五小时额度 (~¥5.3)



01:38:45

但是你现在如果你是一个学习流程的话，你其实上下文你不会像我这样去干这个把整个都放进来，你可以用这个 digested version，然后但这个值得我们思考的是，我们以后还学得起学不起了，这真的是一个非常现实的问题。因为如果让所有人都凭教育平权，那我们没有那么多算力，没有那么多资源，那怎么办？We don't know 对吧？这可能是一个社会性的社会性的问题，但 anyway 这是 UR 他总结的一些但是一句话 796 页的 spec 从头到尾回答一个问题，哪些 intention 必须定死，哪些留白。它构造了一个形式语言，半形式的语言去告诉你哪些东西可以确就是可以在语义上确定下来，哪些可以不用他做解决的，就是对齐的问题。人和人之间，你甚至和下一时刻的自己，你不你今天想的和明天想的不一样，所以你跟你自己对不起。有趣的东西你可以自己大家可以自己回去看一下这个 slides。好，今天下课。



画面 01:38:50



[UML logo - "UNIFIED MODELING LANGUAGE" wordmark with a 3D UML cube emblem in purple, yellow, and red]
Figure summary: Unified Modeling Language logo - the words "UNIFIED MODELING LANGUAGE" beside a 3D cube-shaped "UML" emblem (purple/yellow faces with a red "U" shape on top) and a "TM" symbol.

OMG® Unified Modeling Language® (OMG UML®)

Version 2.5.1

OMG Document Number: | formal/2017-12-05

Date: | December 2017

Normative URL: | <https://www.omg.org/spec/UML/>

Machine Readable: |

Normative: | <https://www.omg.org/spec/UML/20161101/>

Copyright © 2009-2013 88Solutions Copyright © 2009-2010

Artisan Software Tools Copyright © 2001-2013 Adaptive

UML-2.5.1.md 31-60/59767 0%

重的代价不是地重置，文档质量决定了一个变化是局部重置，还是“junk the entire framework, even for relatively modest changes”。拓补只给出可达集合，接口稳定性决定这次重置是付多少钱。你的框架如果不区分这两者，会漏掉 Royce 一半的论点——他花整整一节讲文档，讲的就是这个系数。

三、这个框架会误导你的地方

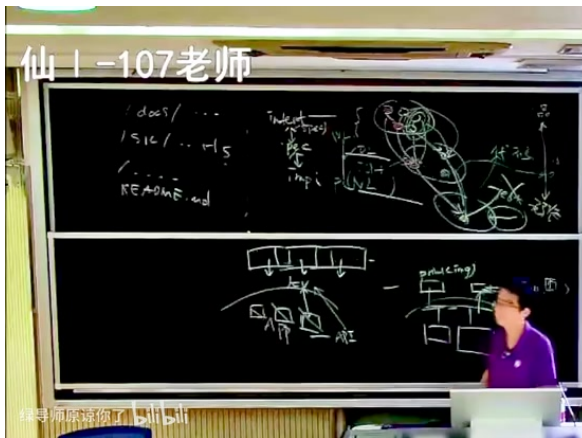
1. “最小化耦合”不是终点。耦合是功能的来源，你不能删边，只能把边改造成到稳定接口上。把“降低耦合”当成唯一目标，会得到一堆什么都没做的模块。
2. 有些修改不是重置，是重新谈判。合同、验收、客户信任是不可逆提交（irreversible commit）：它们不能在图上“重新求值”，只能重新协商。Step 5 的那些评审里程碑之所以重要，正是因为它们把不变量钉死在了合同里。
3. 它会让你以为“做两遍”是冗余重置。不是——pilot model 的价值不是把同样的计算做两次，而是改变图形的形状（消除到底哪个约束会“bind”的不确定性）。它是采样，不是重置。

四、用你的框架重讲 TDD，一句话

你既然不能删掉那条长程回边，剩下的唯一杠杆就是降低它的延迟。于是做推测执行：提前推测验证结果，用一次编辑就能回滚。

- 瀑布 = 分支在流水线末端才发现错误预测，代价是 flush 整个项目
- TDD = 把验证推测到写实现的同一分钟执行，该预测的代价是一次 diff。

这个类比比“文档即规格”更精确地解释了 TDD 为什么有效：它没有减少依赖，只压缩了依赖的传播延迟。而它也和 Royce 的原始判据



画面 01:39:00

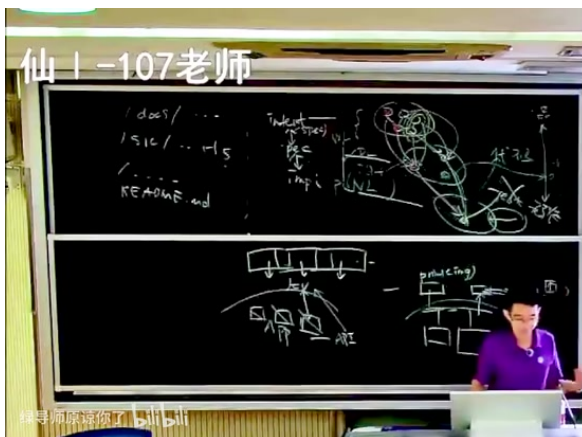
阅读 UML Spec

上大学的时候学过，感觉没学懂

- (怀疑老师可能也不太懂。)
- 依稀有一些模糊的印象
- PhD 时候也读了一些 papers
- Kimi 送了 ¥699 的套餐，直接让 AI 备课，花去 18% 五小时额度 (~¥5.3)



18/19



画面 01:39:58

LLM 时代：Gap 没有消失，只是移位

约束集合与某个具体实现

- UML 把意图从散文析出为形式骨架；LLM 可以从散文直接生成代码
- 模型图定合 & 行为空间：一个实现必须为具体行为作出选择
- 意图：模型图产生多条并发轨迹：满足规约，要检查它可能产生的
- LLM 只能指这一步，也能把歧义消解成没有记录的模糊

模型不得不补全的，可能正是人还没决定的

- “支持超时”之后：继续等待、自动重试，还是告诉用户失败？代码必须
- 一旦把猜出的选择埋进代码，下次重写可能换一个答案，而接口与正常
- 谁维护“已经决定 / 明确禁止 / 尚未决定”的边界，谁就在承接当年元模型与

19/19